



1. Fejezet: Számítógép rendszerek

**The Architecture of Computer Hardware
and Systems Software:
An Information Technology Approach**

**3. kiadás, Irv Englander
John Wiley and Sons ©2003**

**Wilson Wong, Bentley College
Linda Senne, Bentley College**



Tipikus számítógép hirdetés

ONLY
\$999



FASTCAT™ Desktop

Top Performance, Great Value!

- ◊ Intel® 2.7 GHz Pentium® 4 Processor
- ◊ 1 GB up to 2 GB DDRAM
- ◊ 120 GB Ultra ATA 7200 RPM HD 16 MB cache
- ◊ 10/100 PCI Ethernet card
- ◊ 64 MB GeForce 4x AGP Video
- ◊ DVD-ROM/RAM + CD-R/RW
- ◊ 2 USB-2, IEEE-1394 Firewire®
- ◊ 17" Non-Interlaced Full Flat Screen Monitor
.26dp (1024 x 768) Add \$300 for
17" TFT LCD (1280 x 1024)

- Elég gyors a számítógép a fontos programok futtatásához?
- Megéri az árát a számítógép?
- Elavul 6 hónap múlva?



Miért tanulunk számítógép architektúrákat?

- Számítógép felhasználó
 - Megérteni a rendszer képességeit és korlátait
 - Tájékozott döntéseket hozni
 - Fejlessze a kommunikációs készségét az informatikusokkal
- Rendszer analitikus
 - Vázlatot készít, megállapítja a megvalósíthatóságot és meghatározza illetve dokumentálja a felhasználói követelményeket
 - Program rendszerkövetelményeinek meghatározása
- Programozó
 - Hatékony programokat készít sen speciális feladatok megoldásához

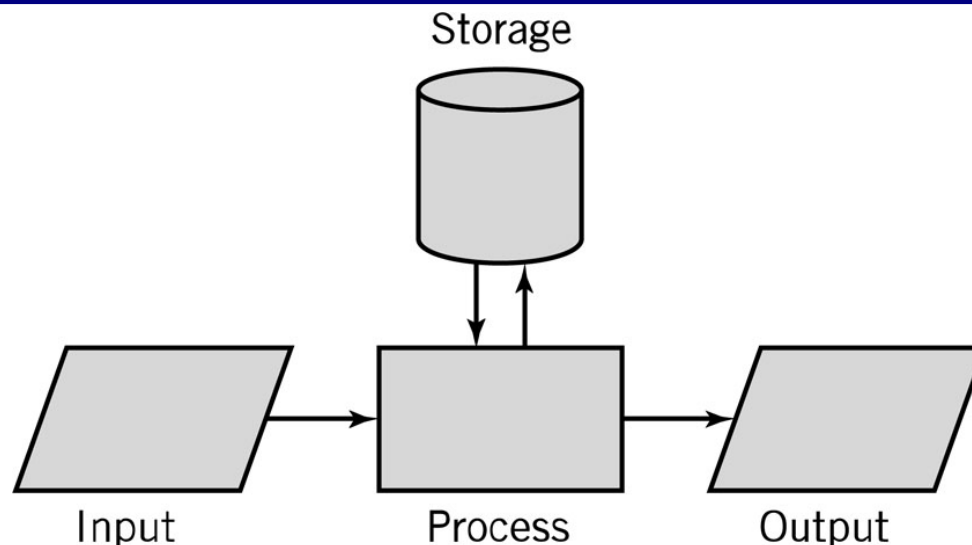


Miért tanulunk számítógép architektúrákat?

- Rendszer adminisztrátor
 - Számítógépes rendszer installálása, beállítása, karbantartása, és felújítása
 - Rendszer rendelkezésre állásának maximalizálása
 - Rendszer teljesítményének optimalizálása
 - Rendszer biztonságának biztosítása
- Web-alkalmazás tervező
 - Optimalizálja az ügyfél hozzáférését a Webes alkalmazásokhoz
 - Webkiszolgáló rendszerek adminisztrálása
 - Kiválassza a megfelelő adatformátumot
 - Hatékony Weboldalak



Input-Process-Output Modell (IPO)



- **„Beolvasás – Feldolgozás – Kiírás” Modell:**

- Feldolgozandó adatok beolvasása (input): billentyűzet, egér, scanner, lyukkártya.
- Feldolgozás (processing): számítógép (CPU) végrehajtja a számítógépprogramot.
- Eredmények kiírása, megjelenítése (output): monitor, nyomtató, fax.
- Tároló eszközök az adatok átmeneti és hosszú-távú tárolására: merevlemez, optikai lemez, hajlékony-lemezek, mágnes kazetta.



Architektúra részei

- *Hardver*
 - Utasítások végrehajtásával adatok feldolgozása képes gép
 - Lehetőséget ad adatok rögzítésére: bevitelre (input) és kiírására (output)
- *Szoftver*
 - Rendszer által végrehajtott utasítások
- Adat
 - Alapvető ábrázolása a tényeknek és az
- Kommunikáció
 - Adatok megosztása és feldolgozás két különböző rendszer között

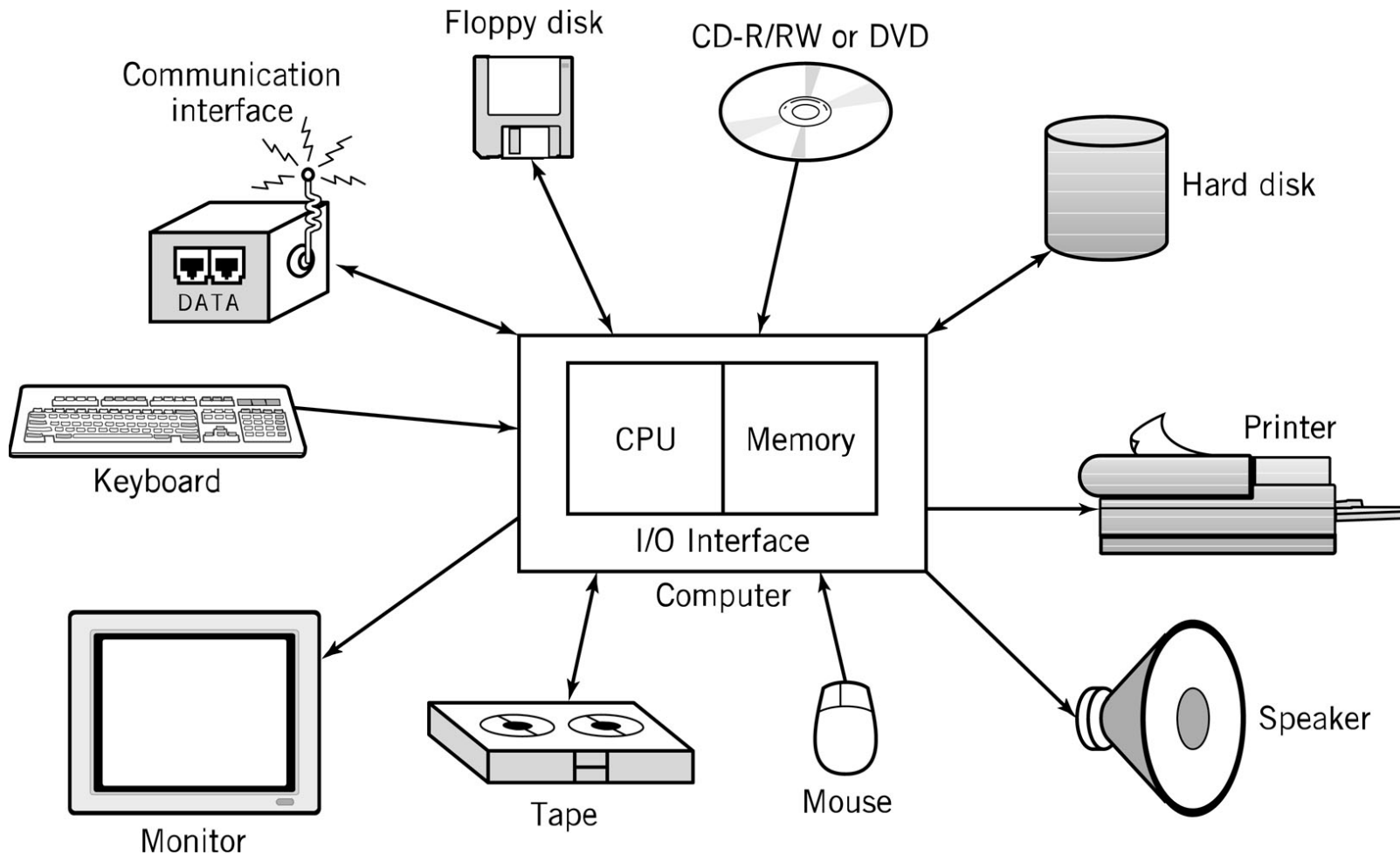


Hardver elemek

- Input/Output eszközök
- Tároló eszközök
- CPU – központi feldolgozó egység (central processing unit)
Részei:
 - ALU: arithmetic/logic unit (aritmetikai/logikai egység)
 - CU: control unit (vezérlőegység)
 - Interface unit (interfészegység)
- Memória
 - CPU által végzett műveletek eredményeinek átmeneti (rövid távú) tárolása



Általános PC





CPU: Központi feldolgozó egység

- ALU: arithmetic/logic unit
 - Aritmetikai és Bool logikai számítások végrehajtása
- CU: control unit
 - Utasítások végrehajtásának irányítása
 - Adatáramok irányítása a CPU-ban
- Interface unit
 - Utasítások és adatok továbbítása a CPU és más hardver egységek között
 - Bus (Sin): kábelköteg, amely képes jeleket és tápfeszültséget szállítani a számítógép különböző elemei között



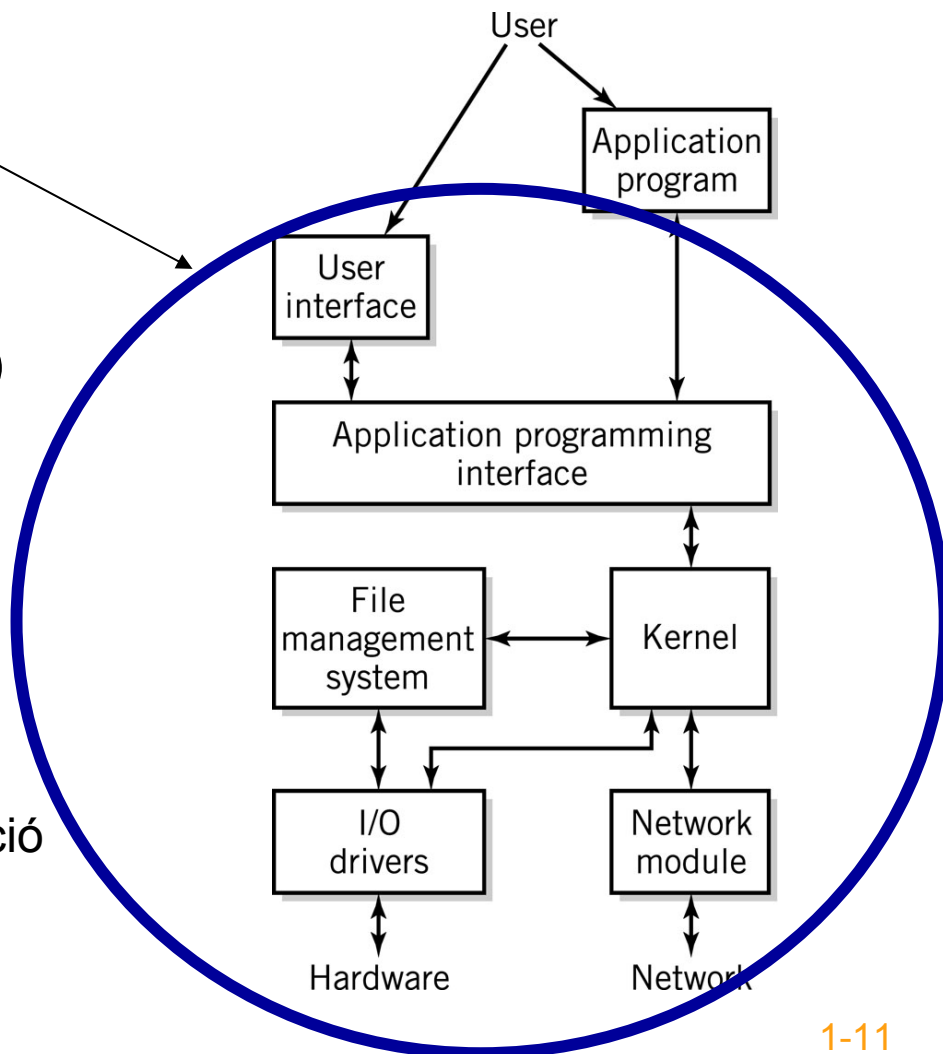
Memória

- Alternatív elnevezések:
 - *elsődleges tároló-*, munkatároló-, *RAM- (random access memory/véletlen hozzáférésű memória)*
- Bit-ekből áll, bit-eket tárol
 - mindegyik felveheti a 0 vagy 1 értéket
 - 8 bit = 1 byte
- Tárolja a számítógép által végrehajtott utasításokat (számítógép program) és az általa feldolgozott adatokat
 - *Neumann János: tárolt program elve (stored program concept)*



Szoftver elemek

- Alkalmazások
- *Operációs rendszer*
 - Felhasználói felület
 - API: application programming interface (programozói interfész)
 - ▣ Alkalmazások által használható hívások
 - Fájl kezelés
 - I/O vezérlők
 - Kernel (mag)
 - ▣ Memória kezelés
 - ▣ Erőforrás ütemezés
 - ▣ Program kommunikáció
 - ▣ Biztonság
 - Hálózati modul





Kommunikációs elemek

- Hardver
 - Kommunikációs *csatornák*
 - Fizikai kapcsolatok számítógépes rendszerek között
 - Például: vezeték, telefon vonal, optikai kábel, infravörös fény, rádió hullámok
 - Interface hardver
 - Közvetlen kommunikáció a számítógép és a kommunikációs csatorna között
 - *Modem* vagy *hálózati kártya (network interface card (NIC))*
- Szoftver
 - Hálózati protokollok: HTTP, TCP/IP, ATAPI



Számítógép rendszerek

Minden számítógéprendszer, bonyolultságától eltekintve tartalmazza a következőket:

- Legalább egy CPU
- Memória a programok és adatok tárolására
- I/O eszközök
- Hosszú távú tároló



Protokollok

- Közös szabályok számítógépek, I/O eszközök, és szoftverek közötti kommunikációra
- Példák
 - HTTP: Web szerver és Web böngésző között
 - TCP/IP: interneten lévő számítógépek és helyi hálózatok között
 - ATAPI: CPU és CD meghajtó között



Szabványok

- Univerzális adat formátum- és protokoll kompatibilitást biztosítanak
- Vagy a szabadalmi hivatal hozta létre vagy tényleges szabvánnyá vált a nagymértékű használat miatt
- Példák:
 - Számítógép nyelvek: Java, SQL, C, JavaScript
 - Megjelenítési szabványok: Postscript, MPEG-2, JPEG, GIF
 - Karakterkiosztás szabványok: ASCII, Unicode, EBCDIC
 - Videó szabványok: VGA, XGA, RGB



Korai történelem

- 1642: Blaise Pascal feltalálta a számológépet
- 1801: Joseph Marie Jacquard megalkotott egy szövőszéket, ami lyukkártyákat használt
- 1800 -as évek:
 - Charles Babbage megkísérelt építeni egy analitikus gépet (mechanikai számítógép)
 - Augusta Ada Byron megalkotott sok alapvető programozási fogalmat
 - George Boole megalkotta a Bool logikát.

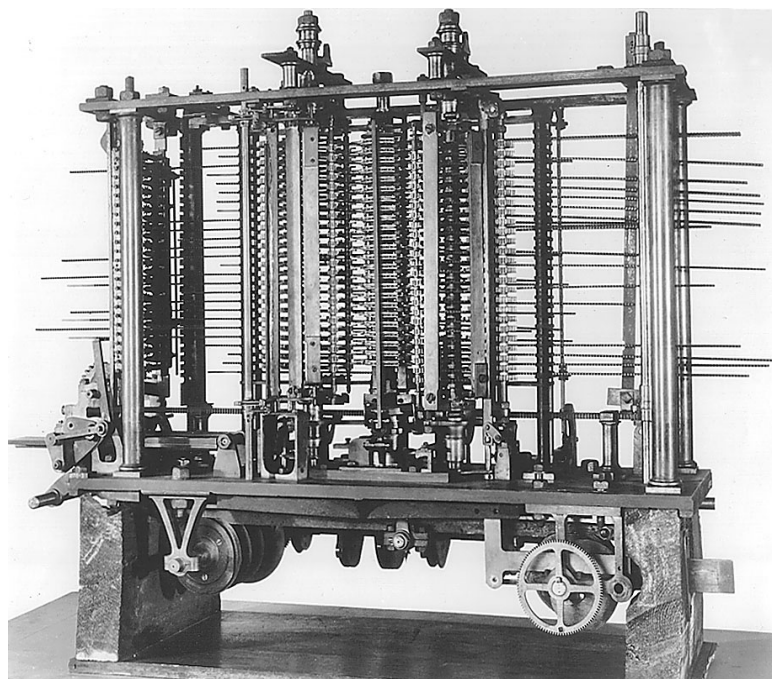


Modern Számítógép fejlesztés

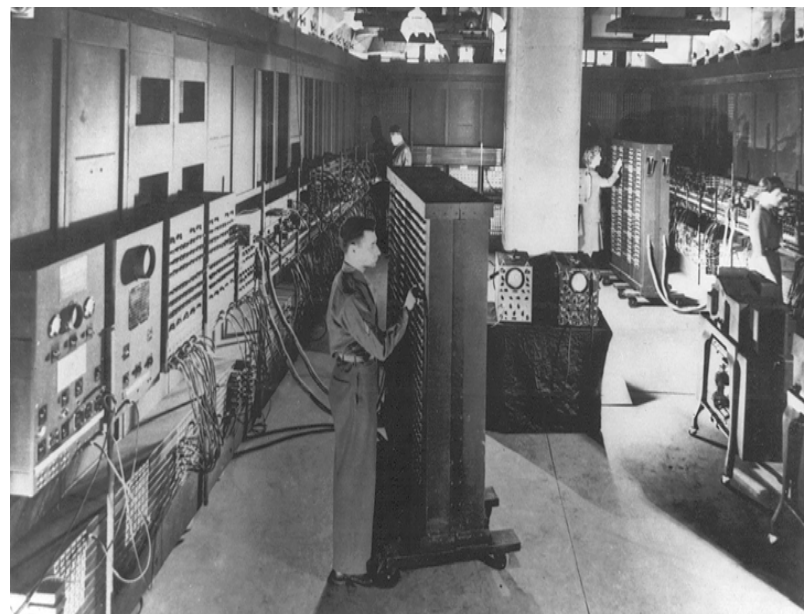
- 1937: Mark I elkészült (Aiken, Harvard University, IBM).
 - Első elektronikus számítógép reléket használva.
- 1939: ABC elkészült
 - Első teljesen elektromos digitális számítógép. Vákuumcsövet használ.
- 1943-46: ENIAC (Mauchly, Eckert, University of Pennsylvania).
 - Első általános tervezésű digitális számítógép
- 1945: Neumann architektúra terv.
 - A mai számítógépek működésének alapelvei:
 - ▣ Tárolt program elve
 - ▣ Bináris aritmetika használata
- 1947: Tranzisztor feltalálása
 - (Bardeen, Shockley, Brattain, Bell Labs).
- 1951: UNIVAC.
 - Az első kereskedelmi számítógép



Korai számítógépek



Babbage analitikus gépe



ENIAC



2. Fejezet : Számrendszerek

**The Architecture of Computer Hardware
and Systems Software:
An Information Technology Approach**

**3. kiadás, Irv Englander
John Wiley and Sons ©2003**

**Wilson Wong, Bentley College
Linda Senne, Bentley College**



Miért bináris?

- Korai számítógépek decimális számábrázolást használtak:
 - Mark I és ENIAC
- Neumann János javasolta a bináris számábrázolást (1945)
 - egyszerűbb a számítógépek megvalósítása
 - utasítások és adatok tárolásához is ezt egyaránt használjuk
- Természetes kapcsolat a kétállású kapcsolók és Bool logikai értékek, ill. műveletek között

	
Be	Ki
Igaz	Hamis
Igen	Nem
1	0



Számolás és aritmetika

- Decimális vagy tízes alapú számrendszer
 - Eredet: számolás az ujjakon
 - “Digit” (számjegy) *digitus* latin szóból ered, jelentése “ujj”
- *Alap*: a számrendszerben használt számjegyek száma, a nullát is beleértve
 - Példa: 10-es alapban 10 számjegy van, 0-tól 9-ig
- *Bináris* vagy *2-es alapú*
- *Bit* (aritmetikai számjegy): 2 számjegy, 0 és 1
- *Oktális* vagy *8-as alapú*: 8 számjegy, 0-tól 7-ig
- *Hexadecimális* vagy *16-os alapú*:
16 számjegy, 0-tól F-ig
 - Példák: $10_{10} = A_{16}$; $11_{10} = B_{16}$



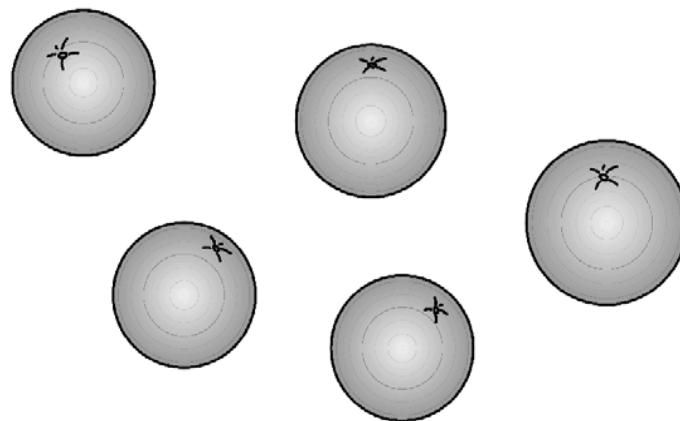
Bitek kezelése a gyakorlatban

- A bit-eket általában csoportokban tárolják és módosítják
 - 8 bit = 1 *byte*
 - 4 byte = 1 word/szó (a legtöbb rendszerben)
- Számítási műveletek során a számábrázolásra használt bit-ek száma
 - befolyásolja az eredmény pontosságát
 - korlátozza a számítógép által kezelhető számok nagyságát (ill. azok számát).



Számok, ill. azok értékének fizikai reprezentációja

- Különböző számjegyek, ugyanaz az érték
 - Rovás írás: IIII
 - Római: V
 - Arab: 5
- Különböző alapok, ugyanazok az értékek
 - 5_{10}
 - 101_2
 - 12_3





Számrendszerek

- Római: pozíció vagy helyiérték *független*, (helyiérték nélküli)
- Modern: helyiértékes jelölésen alapszik
 - Decimális számrendszer: **helyiértékes** számábrázolás, 10 hatványain alapszik
 - Bináris rendszer: **helyiértékes** számábrázolás, 2 hatványain alapszik
 - Oktális rendszer: **helyiértékes** számábrázolás, 8 hatványain alapszik
 - Hexadecimális rendszer: **helyiértékes** számábrázolás, 16 hatványain alapszik



Helyiértékes számábrázolás: 10-es alap

$$43 = 4 \times 10^1 + 3 \times 10^0$$

10-esek



1-esek



Helyi érték	10^1	10^0
Érték	10	1
Számítás	4×10	3×1
Összeg	40	3



Helyiértékes számábrázolás : 10-es alap

$$527 = 5 \times 10^2 + 2 \times 10^1 + 7 \times 10^0$$

100-asok

10-esek

1-esek

Helyi érték	10^2	10^1	10^0
Érték	100	10	1
Számítás	5×100	2×10	7×1
Összeg	500	20	7



Helyiértékes számábrázolás: Oktális

$$624_8 = 404_{10}$$

64-esek

8-asok

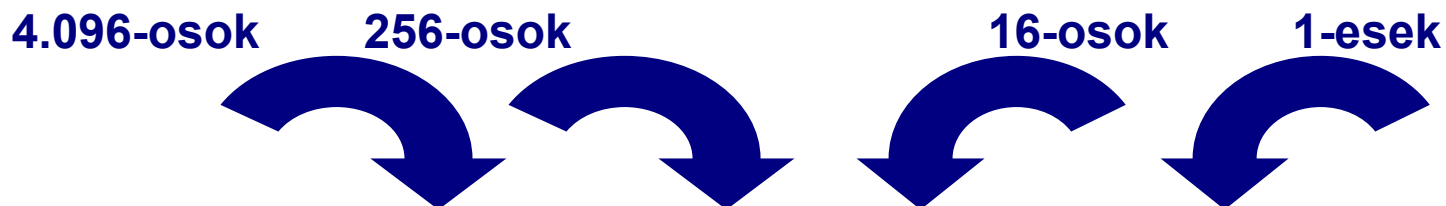
1-esek

Helyi érték	8^2	8^1	8^0
Érték	64	8	1
Számítás	6 x 64	2 x 8	4 x 1
Decimális összeg	384	16	4



Helyiértékes számábrázolás: Hexadecimális

$$6.704_{16} = 26.372_{10}$$



Helyi érték	16^3	16^2	16^1	16^0
Érték	4.096	256	16	1
Számítás	6 x 4.096	7 x 256	0 x 16	4 x 1
Decimális összeg	24.576	1.792	0	4



Helyiértékes számábrázolás: Bináris

$$1101\ 0110_2 = 214_{10}$$

Helyi érték	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Érték	128	64	32	16	8	4	2	1
Számítás	1 x 128	1 x 64	0 x 32	1 x 16	0 x 8	1 x 4	1 x 2	0 x 1
Decimális összeg	128	64	0	16	0	4	2	0



Ábrázolható értékek száma

- $R = B^K$, ahol
 - R = ábrázolható értékek száma (range)
 - B = számrendszer alapja (base)
 - K = számjegyek száma
- 1. Példa: 10-es alap, 2 számjegy
 - $R = 10^2 = 100$ különböző szám (0...99)
- 2. Példa: 2-es alap, 16 számjegy
 - $R = 2^{16} = 65.536$ vagy 64K
 - 16 bit-es PC 65.536 különböző értéket képes tárolni



Bitek decimális terjedelme

Bit-ek	Számjegyek	Terjedelem
1	0+	2 (0 és 1)
4	1+	16 (0-15)
8	2+	256
10	3+	1.024 (1K)
16	4+	65.536 (64K)
20	6+	1.048.576 (1M)
32	9+	4.294.967.296 (4G)
64	19+	$\sim 1,6 \times 10^{19}$ (16×10^{18} [exa])
128	38+	$\sim 2,6 \times 10^{38}$



Számrendszer alapja (base, radix)

- Alap:
 - A számrendszerben számábrázoláshoz szükséges szimbólumok száma
- A *nagyobb* alap *több* számjegyet igényel
 - 10-es alap: 0,1,2,3,4,5,6,7,8,9
 - 2-es alap: 0,1
 - 8-as alap: 0,1,2,3,4,5,6,7
 - 16-os alap: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F



Szimbólumok száma vs. számjegyek száma

- Egy adott szám ábrázolásához a *nagyobb* alap esetén
 - *több* szimbólum szükséges
 - de *kevesebb* számjegy kell
- 1. Példa:
 - 65_{16} 101_{10} 145_8 $110\ 0101_2$
- 2. Példa:
 - $11C_{16}$ 284_{10} 434_8 $1\ 0001\ 1100_2$



Számolás 2-es alappal

Bináris szám	Egyenérték				Decimális szám
	8-asok (2^3)	4-esek (2^2)	2-esek (2^1)	1-esek (2^0)	
0				0×2^0	0
1				1×2^0	1
10			1×2^1	0×2^0	2
11			1×2^1	1×2^0	3
100		1×2^2			4
101		1×2^2		1×2^0	5
110		1×2^2	1×2^1		6
111		1×2^2	1×2^1	1×2^0	7
1000	1×2^3				8
1001	1×2^3			1×2^0	9
1010	1×2^3		1×2^1		10



10-es alapú összeadó tábla

$$3_{10} + 6_{10} = 9_{10}$$

+	0	1	2	3	4	5	6	7	8	9
0	0	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9	10
2	2	3	4	5	6	7	8	9	10	11
3	3	4	5	6	7	8	9	10	11	12
4	4	5	6	7	8	9	10	11	12	13

stb



8-as alapú összeadó tábla

$$3_8 + 6_8 = 11_8$$

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	10
2	2	3	4	5	6	7	10	11
3	3	4	5	6	7	10	11	12
4	4	5	6	7	10	11	12	13
5	5	6	7	10	11	12	13	14
6	6	7	10	11	12	13	14	15
7	7	10	11	12	13	14	15	16

(nincs se 8 se 9 természetesen)



10-es alapú szorzótábla

$$3_{10} \times 6_{10} = 18_{10}$$

x	0	1	2	3	4	5	6	7	8	9
0				0						
1		1	2	3	4	5	6	7	8	9
2		2	4	6	8	10	12	14	16	18
3		3	6	9	12	15	18	21	24	27
4	0	4	8	12	16	20	24	28	32	36
5		5	10	15	20	25	30	35	40	45
6		6	12	18	24	30	36	42	48	54
7		7	14	21	28	35	42	49	56	63

stb.



8-as alapú szorzótábla

$$3_8 \times 6_8 = 22_8$$

x	0	1	2	3	4	5	6	7
0				0				
1		1	2	3	4	5	6	7
2		2	4	6	10	12	14	16
3	0	3	6	11	14	17	22	25
4		4	10	14	20	24	30	34
5		5	12	17	24	31	36	43
6		6	14	22	30	36	44	52
7		7	16	25	34	43	52	61



Összeadás

Alap	Feladat	Legnagyobb egyjegyű szám
Decimális	$\begin{array}{r} 6 \\ +3 \\ \hline \end{array}$	9
Oktális	$\begin{array}{r} 6 \\ +1 \\ \hline \end{array}$	7
Hexadecimális	$\begin{array}{r} 6 \\ +9 \\ \hline \end{array}$	F
Bináris	$\begin{array}{r} 1 \\ +0 \\ \hline \end{array}$	1



Összeadás

Alap	Feladat	Átvitel	Eredmény
Decimális	$\begin{array}{r} 6 \\ +4 \\ \hline \end{array}$	10	10
Oktális	$\begin{array}{r} 6 \\ +2 \\ \hline \end{array}$	8	10
Hexadecimális	$\begin{array}{r} 6 \\ +A \\ \hline \end{array}$	16	10
Bináris	$\begin{array}{r} 1 \\ +1 \\ \hline \end{array}$	2	10



Bináris aritmetika

$$\begin{array}{r} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ + \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ \hline 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{array}$$



Bináris aritmetika

- Összeadás
 - Bool XOR-t és AND-et használ
- Szorzás
 - AND
 - Shift (eltolás)
- Osztás

+	0	1
0	0	1
1	1	10

x	0	1
0	0	0
1	0	1



Bináris aritmetika: Bool logika

- Bool logika véges aritmetika nélkül
 - *EXCLUSIVE-OR (kizáró vagy)*
 - “1” az output ha **nem** mindegyik bemenet értéke “1”
 - *AND (carry bit)*
 - “1” az output ha mindegyik bemenet értéke “1”

1	1	1	1	1			
	1	1	0	1	1	0	1
+			1	0	1	1	0
1	0	0	0	0	0	1	1



Bináris szorzás

- Bool logika véges aritmetika nélkül
 - *AND (carry bit)*
 - “1” az output ha mindegyik bemenet értéke “1”
 - *Shift (eltolás)*
 - Egy szám eltolása bármelyik alapon **balra** egy jeggyel, az egyenlő az alappal való **szorzással**
 - Egy szám eltolása bármelyik alapon **jobbra** egy jeggyel, az egyenlő az alappal való **osztással**
 - Példák:

□ 10_{10} eltolva balra = 100_{10}	□ 10_{10} eltolva jobbra = 1_{10}
□ 10_2 eltolva balra = 100_2	□ 10_2 eltolva jobbra = 1_2



Bináris szorzás

$$\begin{array}{r} \\ x \\ \hline \\ \\ \\ \hline 1 \end{array}$$

Első hely

Második hely

Harmadik hely (szorzandó bitjei *eltolva* egygel feljebb a harmadik helyen)

Eredmény (*AND*)



Bináris szorzás

$$\begin{array}{r} \boxed{1\ 1\ 0\ 1\ 1\ 0\ 1} \\ \times \quad \quad 1\ 0\ 0\ 1\ \textcircled{1}\ 0 \\ \hline 1\ 1\ 0\ 1\ 1\ 0\ 1 \end{array}$$

Második helyiértékhez tartozik (szorzandó bitjei **eltolva** egy helyiértékkel balra ~ 2-vel megszoroztuk)

1 1 0 1 1 0 1

3. helyiértékhez tartozik (2^4)

1 1 0 1 1 0 1

6. helyiérték (2^5)

1 0 0 0 0 0 0 1 0 1 1 1 0

Eredmény (**AND**)

Megjegyzés: a 0 a végén, az első helyen nem lett leírva.

megjegyzés: többszörös carry lehetséges.



Bináris szorzás

			1	1	1	1	1						
	1	1	1	1	1	1	1	1					
	1	0	1	1	1	1	1	0	0	1	x	1	1
		1	0	1	1	1	1	1	0	0	1		
			1	0	1	1	1	1	1	0	0	1	
1	0	1	0	0	1	1	0	0	1	1	1	1	



Konvertálás 10-es alapból

■ Hatvány tábla

Kitevő Alap	8	7	6	5	4	3	2	1	0
2	256	128	64	32	16	8	4	2	1
8				32.768	4.096	512	64	8	1
16					65.536	4.096	256	16	1



10-es alapból 2-esbe

$$42_{10} = 101010_2$$

Kitevő Alap	6	5	4	3	2	1	0
2	64	32	16	8	4	2	1
		1	0	1	0	1	0
Egész		$42/32 = 1$	$10/16 = 0$	$10/8 = 1$	$2/4 = 0$	$2/2 = 1$	$0/1 = 0$
Maradék		10	10	2	2	0	0



10-es alapból 2-esbe

10-es alap **42**

Hányados
$$\begin{array}{r} 2 \) \ 42 \\ \hline 2 \) \ 21 \\ \hline 2 \) \ 10 \\ \hline 2 \) \ 5 \\ \hline 2 \) \ 2 \\ \hline 2 \) \ 1 \end{array}$$

Maradék

Legkisebb helyi-értékű bit

Legnagyobb helyi-értékű bit

2-es alap **101010**



10-es alapból 16-osba

$$5.735_{10} = 1667_{16}$$

Kitevő Alap	4	3	2	1	0
16	65.536	4.096	256	16	1
		1	6	6	7
Egész		5.735 / 4.096 = 1	1.639 / 256 = 6	103 / 16 = 6	7
Maradék		5.735 – 4.096 = 1.639	1.639 – 1.536 = 103	103 – 96 = 7	



10-es alapból 16-osba

10-es alap 5.735 Maradék

Hányados

$$\begin{array}{r} 16 \overline{) 5.735} \quad (7 \text{ Legkisebb helyi-értékű bit} \\ 16 \overline{) 358} \quad (6 \\ 16 \overline{) 22} \quad (6 \\ 16 \overline{) 1} \quad (1 \text{ Legnagyobb helyi-értékű} \\ 16 \overline{) 0} \quad \text{bit} \end{array}$$

16-os alap 1667



10-es alapból 16-osba

10-es alap 8.039

Maradék

Hányados

16)	8.039	(7	Legkisebb helyi-értékű bit
16)	502	(6	
16)	31	(15	
16)	1	(1	Legnagyobb helyi-értékű bit
16)	0		

16-os alap 1F67



8-as alapból 10-esbe

$$7263_8 = 3.763_{10}$$

Kitevő	8^3	8^2	8^1	8^0
	512	64	8	1
	x 7	x 2	x 6	x 3
Decimális összeg	3.584	128	48	3



8-as alapból 10-esbe

$$7263_8 = 3.763_{10}$$

$$\begin{array}{r} 7 \\ \times 8 \\ \hline \end{array}$$

$$56 + 2 = 58$$

$$\begin{array}{r} \times 8 \\ \hline 464 \end{array} + 6 = 470$$

$$\begin{array}{r} \times 8 \\ \hline 3760 \end{array} + 3 = 3.763$$



16-os alapból 2-esbe

- A **nibble** (négy bites szó) megközelítés
 - Hexadecimálist könnyebb írni és olvasni, mint binárist

16-os alap	1	F	6	7
2-es alap	0001	1111	0110	0111

- Miért hexadecimális?
 - Operációs rendszer, ill. hálózati hibák esetén a rendszer a hibaeseményhez tartozó adatokat hexadecimális formában jelenít meg. (~ ismeretlen memóriatartalom)



Törtek

- Tizedespont vagy alappont
 - Tizedespont a 10-es alapban
 - Binárispont a 2-es alapban
- Nincs tökéletes alakja a törteknek különböző számrendszerekben
 - Pontos átalakítás nem mindig lehetséges



Tizedestörtek

- Rakjuk egygel jobbra a tizedespontot
 - Jelenség: a szám megszorzódik a számrendszer alapjával
 - Példa: $139.0_{10} \longrightarrow 1390_{10}$
- Rakjuk egygel balra a tizedespontot
 - Jelenség: a szám osztódik a számrendszer alapjával
 - Példa: $139.0_{10} \longrightarrow 13.9_{10}$



Törtek: 10-es és 2-es alap

0,2589₁₀

Helyi érték	10^{-1}	10^{-2}	10^{-3}	10^{-4}
Érték	1/10	1/100	1/1000	1/10000
Számítás	2 x 1/10	5 x 1/100	8 x 1/1000	9 x 1/1000
Összeg	0,2	0,05	0,008	0,0009

0,101011₂ = 0,671875₁₀

Helyi érték	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
Érték	1/2	1/4	1/8	1/16	1/32	1/64
Számítás	1 x 1/2	0 x 1/4	1 x 1/8	0 x 1/16	1 x 1/32	1 x 1/64
Összeg	0,5		0,125		0,03125	0,015625



Törtek: 10-es és 2-es alap

- Nincs általános összefüggés az $1/10^k$ és $1/2^k$ típusú törtek között
 - Ennek következtében néhány 10-es alapon ábrázolható szám nem ábrázolható 2-es alapon
 - De minden $1/2^k$ formájú törtet tudunk ábrázolni 10-es alapon
- Tört átalakítása két alap között akkor ér véget
 - Ha van racionális megoldás vagy
 - Ha a kívánt pontosságot elértük



Kevert számalakítás

- Egész és tört részeket külön kell konvertálni
- Alappont (tizedes pont): rögzített viszonyítás az átalakításhoz
 - A bal oldalon lévő számjegy egységnyi minden alapban
 - B^0 mindig 1, alaptól függetlenül



4. Fejezet : Az egész számok (integer) ábrázolása

**The Architecture of Computer Hardware
and Systems Software:
An Information Technology Approach**

**3. kiadás, Irv Englander
John Wiley and Sons ©2003**

**Wilson Wong, Bentley College
Linda Senne, Bentley College**

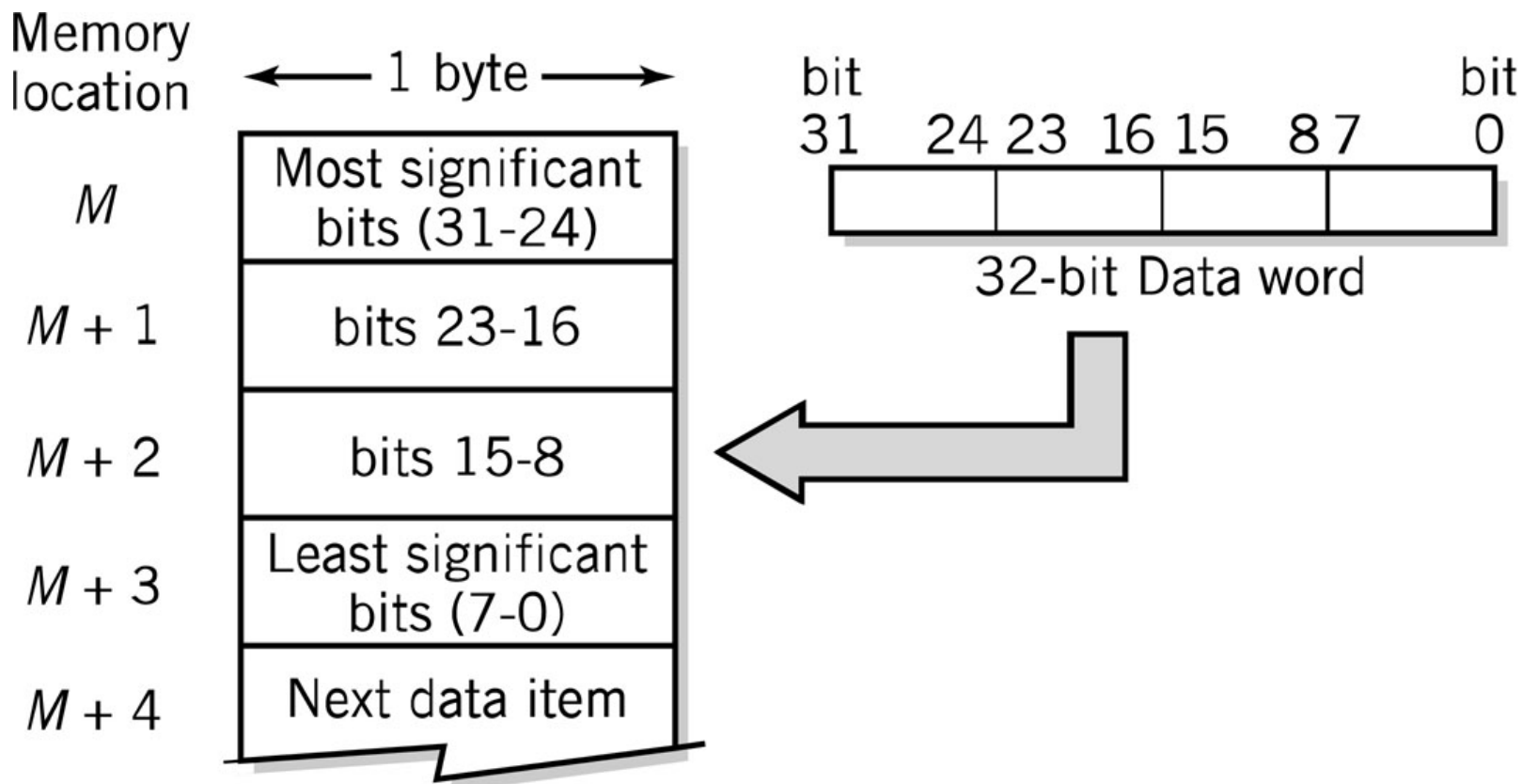


Számábrázolás

- Egésszszámok ábrázolásához a következő információt kell tárolnunk:
 - Egésszszám abszolút értéke (value, magnitude)
 - Előjel (plusz vagy mínusz)



32 bit-es adat szó





Előjel nélküli egész számok: Integerek

- Bináris ábrázolás: Előjel nélküli egész szám vagy *integer*
- BCD ábrázolás: Decimális egész közvetlen *bináris* megfelelője (Binary Coded Decimal)
 - 4 bit: 0-tól 9-ig
 - 8 bit: 0-tól 99-ig
 - 16 bit: 0-tól 9.999-ig
 - 32 bit: 0-tól 99.999.999-ig

Decimális érték	Bináris ábrázolás	BCD ábrázolás		
68	$= 0100\ 0100$ $= 2^6 + 2^2 = 64 + 4 = \mathbf{68}$	$= 0110$	1000	
99 (legnagyobb 8 bit-en ábrázolható szám BCD kódolással)	$= 0110\ 0011$ $= 2^6 + 2^5 + 2^1 + 2^0 =$ $= 64 + 32 + 2 + 1 = \mathbf{99}$	$= 1001$	1001	
255 (legnagyobb 8 bit-en ábrázolható szám bináris kódolással)	$= 1111\ 1111$ $= 2^8 - 1 = \mathbf{255}$	$= 0010$	0101	0101
		$= 2^1$	$2^2 + 2^0$	$2^2 + 2^0$
		$= \mathbf{2}$	$\mathbf{5}$	$\mathbf{5}$



Ábrázolható értékek összehasonlítása: Bináris vs. BCD

- Ábrázolható értékek BCD kódolással < ábrázolható értékek hagyományos bináris ábrázolás
 - Bináris: 4 bit 16 különböző értéket képes tárolni (0-tól 15-ig)
 - BCD: 4 bit csak 10 különböző értéket tud tárolni (0-tól 9-ig)

Bit-ek száma	BCD határ		Bináris határ	
4	0-9	1 számjegy	0-15	1+ számjegy
8	0-99	2 számjegy	0-255	2+ számjegy
12	0-999	3 számjegy	0-4.095	3+ számjegy
16	0-9.999	4 számjegy	0-65.535	4+ számjegy
20	0-99.999	5 számjegy	0-1 million	6+ számjegy
24	0-999.999	6 számjegy	0-16 million	7+ számjegy
32	0-99.999.999	8 számjegy	0-4 billion	9+ számjegy
64	0-(10 ¹⁶ -1)	16 számjegy	0-16 quintillion	19+ számjegy



Hagyományos bináris vs. BCD

- Bináris ábrázolás kedveltebb
 - Nagyobb értéket képes tárolni adott számú bit-en (helyiértéken)
 - Számolási műveletek elvégzése egyszerűbb
- BCD gyakran használatos vállalati rendszereknél, főként a tizedes kerekítés és a tizedes pontosság miatt



4-7



Előjeles egész számok ábrázolása

- Nincs nyilvánvaló, közvetlen ábrázolása az előjelnek bináris ábrázolásban
- Lehetőségek:
 - Előjel-és-érték ábrázolás
 - 1-es komplementens
 - 2-es komplementens (gyakoribb)



Előjel-és-érték

- A baloldali legszélső bit használata előjelként
 - 0 = pozitív; 1 = negatív
- Az ábrázolható értékek száma nem változik
 - Számok fele pozitív, fele negatív
 - A legnagyobb ábrázolt érték feleakkora lesz, mint előjel nélkül
- Példa 8 bit-en:
 - Előjel nélküli: $1111\ 1111 = (+)255$
 - Előjeles: $0111\ 1111 = +127$
 $1111\ 1111 = -127$
 - Megjegyzés: kettő érték a 0-ra:
 $+0 = 0000\ 0000$ és
 $-0 = 1000\ 0000$



Bonyolult számítási algoritmusok

- Előjel-és-érték algoritmusok összetettek és bonyolultak ahhoz, hogy hardver hajtsa végre
 - Vizsgálni kell a 0 mindkét értékét
 - BCD kódolás esetén használható
 - Az előjeles szám és a carry/átvitel sorrendje hibát okoz
- Példa: Decimális összeadó algoritmus

Összeadás: 2 pozitív szám	Összeadás: 1 előjeles szám		
$\begin{array}{r} 4 \\ +2 \\ \hline 6 \end{array}$	$\begin{array}{r} 4 \\ -2 \\ \hline 2 \end{array}$	$\begin{array}{r} 2 \\ -4 \\ \hline -2 \end{array}$	$\begin{array}{r} 12 \\ -4 \\ \hline 8 \end{array}$



Komplemens ábrázolás

- A szám előjelét nem kell elkülönítve kezelni
- Megegyezik minden különböző előjelű bemeneti számkombinációra
- Két módszer
 - Alapszám: alap (abszolút) érték
 - Csökkentett alap: alap értéke mínusz 1
 - ▣ 1-es komplement: a 2-es alap csökkentett alapja



Modulus aritmetika (számolás)

- A műveletek eredménye megadja a (számolás alapjaként használt értékkel) modulus történő osztás maradékát
- Példa:
 - $4 \bmod 4 = 0$
 - $5 \bmod 4 = 1$
 - $1000 \bmod 999 = 1$
- A modulusos aritmetika legfontosabb jellemzője
 - A számolást nullától újrakezdjük, ha a számolás során túllépnénk a *moduluson*



Számábrázolás választásának indokai

- Legyen konzisztens (hasonló logikájú) a megszokott aritmetikával
 - $-(-\text{érték}) = \text{érték}$
- Ha egy értéknek kétszer kiszámoljuk a komplementjét, akkor az eredeti értéket kapjuk vissza:
 - $\text{komplement} = \text{alap} - \text{érték}$
 - Egy komplement értékre még egyszer alkalmazzuk a komplement számolást:
 - ▢ $\text{alap} - \text{komplement} = \text{alap} - (\text{alap} - \text{érték}) = \text{érték} (!!!)$



Túlcsordulás

- Rögzített szó (word) méretnek rögzített a terjedelme
- Túlcsordulás: számok azon kombinációja, amelyek összege a határon kívül esik
- Túlcsordulás átvitelével, ezt a problémát kerüli ki
- Komplementes aritmetika: *határon kívüli* számoknak ellentétes előjelük van
 - Teszt: Ha összeadásnál mindkét bemeneti szám azonos előjelű és az eredmény előjele más, akkor túlcsordulás lépett fel



1-es bináris komplementens

- *Komplementens számítás*: az érték kivonása az alap eredeti értékéből
 - Bináris (2-es alap) csökkentett alapú komplementense
 - Alap mínusz 1 = $2 - 1 \rightarrow 1$ mint az alap
- *Invertálás: egyesek és nullák felcserélése (1→0, 0→1)*
 - 0-val kezdődő számok pozitívak
 - 1-el kezdődő számok negatívak
 - Két értéke van a nullának (-0, +0)
- Példa 8 bit-es bináris számokkal

Számok	Negatív		Pozitív	
Megjelenítési mód	Komplementens		Szám önmaga	
Decimális szám terjedelme	-127₁₀	-0₁₀	+0₁₀	127₁₀
Számolás	Invertálás		Nincs	
Ábrázolási példa	10000000	11111111	00000000	01111111



1-es bináris komplementens

- 1. Példa: $0101\ 1111_{1kB} = +95_D$
- 2. Példa: $1010\ 0000_{1kB}$
 - negatív
 - a negáltja amely egyben az abszolút értéke is(!!!): $0101\ 1111_B = |95_D|$
 - tehát az eredmény: -95_D



Összeadás

- Adjunk össze 2 db 8 bit-es pozitív számot

$$0010\ 1101 = +45$$

$$0011\ 1010 = +58$$

$$\hline 0110\ 0111 = +103$$

- Adjunk össze 2db 8 bit-es különböző előjelű számot

- Vegyük az 58 egyes komplementjét (invertáljuk)

0011 1010

1100 0101

$$0010\ 1101 = +45$$

$$1100\ 0101 = -58$$

$$\hline 1111\ 0010 = -13$$

Invertáljuk, hogy megkapjuk az értéket

0000 1101

$$8 + 4 + 1 =$$

|13|



Összeadás átvitelrel (carry-vel)

$$\begin{array}{rcl} 0010\ 1101 & = & +45 \\ 1110\ 0001 & = & -30 \\ \hline 1|0000\ 1110 & = & + \\ \text{└───────────} +1 & = & 15 \\ & & +15 \end{array} \quad \begin{array}{rcl} 1110\ 0001 & = & -30 \\ \hline 0001\ 1110 & = & |30| \end{array}$$



Összeadás átvitelrel (carry-vel)

- 8 bit-es szám
- Negatív érték hozzáadása:

- Invertálás

0000 0010 (2_{10})

1111 1101

- Összeadás

- 9 bit

Túlcsordulás átvitelrel

$$0110\ 1010 = +106$$

$$1111\ 1101 = -2$$

$$\begin{array}{r} 10110\ 0111 \\ \hline \end{array}$$

$$\begin{array}{r} \text{L} \rightarrow +1 \\ \hline \end{array}$$

$$0110\ 1000 = +104$$



Kivonás átvitelrel (carry-vel)

- 8 bit-es számábrázolás
- szám kivonása = szám
-1 szeresének
hozzáadása (negatív alak!)

- invertálás

0101 1010 (90_{10})

1010 0101



- Összeadás

- 9 bit

Túlcsordulás átvitelrel!

$$0110\ 1010 = +106$$

$$-0101\ 1010 = -(+90)$$

$$0110\ 1010 = +106$$

$$+1010\ 0101 = -90$$

$$\textcircled{1}0000\ 1111$$

$$\text{L} \rightarrow +1$$

$$0001\ 0000 = +16$$



Kivonás

- 8 bit-es számábrázolás
- szám kivonása =
szám -1 szeresének
hozzáadása (negatív alak)

- invertálás

1111 0110 (-9_{10})

0000 1001

- összeadás

$$0110\ 1010 = +106$$

$$- 1111\ 0110 = -(-9)$$

$$0110\ 1010 = +106$$

$$+ 0000\ 1001 = +9$$

$$0111\ 0011 = +115$$



Kivonás

- 8 bit-es szám

- invertálás

1010 0101 (90_{10})

0101 1010

- Összeadás

$$0110\ 1010 = +106$$

$$- 1010\ 0101 = -90$$

$$0110\ 1010 = +106$$

$$+ 0101\ 1010 = +90$$

$$1100\ 0100 = -$$

$$0011\ 1011 = |59|$$

Túlcsordulás! Rossz eredmény!!!



Túlcsordulás

- 8 bit-es szám
 - 256 különböző szám
 - Pozitív számok: 0 to 127
 - Összeadás
 - *Túlcsordulás* tesztelése
 - 2 pozitív bemenetre negatív eredményt adott
- ➔ *túlcsordulás!*
- **Rossz eredmény!!!**

$$\begin{array}{rcl} 0100\ 0000 & = & +64 \\ 0100\ 0001 & = & +65 \\ \hline 1000\ 0001 & & -126 \\ 0111\ 1110 & & \nearrow \\ 126_{10} & & \end{array}$$

Invertáljuk,
hogy
megkapjuk az
értéket

- Programozók óvakodjatok: néhány magas-szintű nyelv pl.: a BASIC néhány verziója nem ellenőrzi a túlcsordulást megfelelően!



2-es komplement

- Modulus = 2-es alapú „1”-es után nullák
 - 8 bit-en a modulus = 1000 0000
- Két módszer van megtalálni a komplementst
 - Kivonjuk a szám értékét a modulusból vagy invertálunk

Számok	Negatív		Pozitív	
Megjelenítési mód	Komplement		Szám önmaga	
Decimális szám értéke	-128_{10}	-1_{10}	$+0_{10}$	127_{10}
Számolás	Invertálás		Nincs	
Megjelelenítési példa	10000000	11111111	00000000	01111111



1-es vs. 2-es komplement

- A választás a számítógép tervezőitől függ
- 1-es komplement
 - Egyszerű előjelet váltani
 - Összeadásnak szüksége van egy extra túlcsondulás átvitelre
 - Algoritmusnak tesztelnie és konvertálnia kell a "-0"-át
- 2-es komplement egyszerűbb
 - Negálás után egy $|1|$ hozzáadás szükséges



2-es bináris komplementens

- 1. Példa: $0101\ 1111_{2kB} = +95_D$
- 2. Példa: $1010\ 0000_{2kB}$
 - negatív
 - a negáltja amely nem az abszolút értéke(!!!):
 $0101\ 1111_B$
 - |1| hozzáadása a negálthoz
 - így:
$$\begin{array}{r} 0101\ 1111_B \\ + \quad \quad \quad 1_B \\ \hline 0110\ 0000_B = |96_D| \end{array}$$
 - tehát az eredmény: -96_D



Egy összetett példa

- Határozza meg a decimális és hexadecimális értékét a következő 8 bit-es bináris számoknak, ha közönséges binárisként, vagy 1-es komplementként illetve 2-es komplementként értelmezzük őket. A hexadecimális számoknál az előjeleket kezelje a decimális számoknál megszokott módon!
- $0111\ 0100_B$: közönséges: 116_D és 74_H
1-es: $+116_D$ és $+74_H$
2-es: $+116_D$ és $+74_H$
- $1101\ 0101_B$: közönséges: 213_D és $D5_H$
 $|0010\ 1010_B|$ 1-es: -42_D és $-2A_H$
 $|0010\ 1011_B|$ 2-es: -43_D és $-2B_H$



Egy még összetettebb példa

- Végezze el a következő **kivonást** 8 bit-es bináris számokkal úgy, hogy a számpár első tagját **1-es komplementesként** a második tagját **2-es komplementesként** értelmezzük. Az **eredményt** adja meg bináris **1-es komplementesként**, majd váltsa át azt **decimális** számrendszerbe:
- $1010\ 0101_{B1K} - 1001\ 1010_{B2K} = 1010\ 0101_{B1K} - (0110\ 0101_B + |1|_B = 0110\ 0110_B \Rightarrow) 1001\ 1001_{B1K}$
- $$\begin{array}{r} 1010\ 0101_{B1K} \\ + 0110\ 0110_B \\ \hline 1\ 0000\ 1011_{B1K} \\ \text{└───────────┬───────────┐} \\ \text{└───────────┘} \quad 1_B \\ \hline 0000\ 1100_{B1K} = +12_D \end{array}$$



Integer mérete

- Pozitív számok 0-val kezdődnek
- Kicsi, negatív számok (közel a 0-hoz) több 0-val kezdődnek
 - $1111\ 1110_{B2K} = -2$ 8 bit-es 2-es komplement
 - $1000\ 0000_{B2K} = -128$, nagyobb negatív szám 2-es komplemente
 - Cseréljük fel minden 1-et és 0-át és határozzuk meg az értéket



Túlcsordulás és Átvitel közötti különbség

- *Átvitel (carry) bit:*
 - Jelzi, ha egy **adott** helyiértéken elvégzett művelet eredménye meghaladja az ott ábrázolható értéket.
- *Túlcsordulás (Overflow):*
 - ha egy összeadás vagy kivonás művelet eredménye kívül esik az aktuális számábrázolási tartományon.



Túlcsordulás/Átvitel példák

■ Példa 1:

- Helyes eredmény
- Se túlcsordulás, se átvitel

$$0100 = (+4)$$

$$\begin{array}{r} 0010 \\ \hline \end{array} = +(+2)$$

$$0110 = (+6)$$

■ Példa 2:

- **Helytelen** eredményt kapunk
- Van túlcsordulás, és átvitel is

$$0100 = (+4)$$

$$\begin{array}{r} 0110 \\ \hline \end{array} = +(+6)$$

$$1010 = (-6)$$

Négy bit-es 2-es komplement számábrázolás!



Túlcsordulás/Átvitel példák

■ Példa 3:

- Az átvitelt elhagyva az eredmény helyes
- Van átvitel, de nincs túlcsordulás

$$\begin{array}{rcl} 1100 & = & (-4) \\ 1110 & = & +(-2) \\ \hline \cancel{1}1010 & = & (-6) \end{array}$$

■ Példa 4:

- **Helytelen** eredmény
- Van túlcsordulás, átvitel mellőzve

$$\begin{array}{rcl} 1100 & = & (-4) \\ 1010 & = & +(-6) \\ \hline \cancel{1}0110 & = & (+6) \end{array}$$

Négy bit-es 2-es komplement számábrázolás!



5. Fejezet : Lebegőpontos számok

**The Architecture of Computer Hardware
and Systems Software:
An Information Technology Approach**

**3. kiadás, Irv Englander
John Wiley and Sons ©2003**

**Wilson Wong, Bentley College
Linda Senne, Bentley College**



Lebegőpontos számok

- Valós számok
- Akkor használjuk a számítógépeknél, ha
 - Kívül esik a számítógép integer határán (túl kicsi vagy túl nagy)
 - Tizedes törtet tartalmaz



Exponenciális jelölés

- Hívják még *tudományos jelölésnek*
 - 12345
 - 12345×10^0
 - 0.12345×10^5
 - 123450000×10^{-4}
- 4 alkotóelem kell egy számhoz
 1. Előjel (pl.: "+")
 2. Érték vagy *mantissza* (pl.: 12345)
 3. Kitevő előjele (pl.: "+" a 10^5 -en)
 4. Kitevő értéke (pl.: 5)
- Plusz
 5. Kitevő alapja (pl.: 10)
 6. Tizedesvessző helye (vagy más bázisé) alappont



Szabályok összegzése

Mantissza előjele

Kitevő előjele

-0.35790 x **10**^{**-6**}

Tizedes-
vessző
helye

Mantissza

Alap

Kitevő



Formai leírás

- Előre definiált forma, általában 8 bit
 - Megnövelt értékhatár (két jegyű kitevő) kisebb pontosság árán (ötjegyű mantissza)

Mantissza előjele

SEEMMMM

2 jegyű kitevő

5 jegyű mantissza



Forma

- Mantissza: előjel karakter az előjel-érték formában
- Tizedespont a mantissza elején helyezkedik el
- Excess-N jelölés: komplement ábrázolás
 - Vegyük az eltolást a középpértéknek, ahol N a középpérték

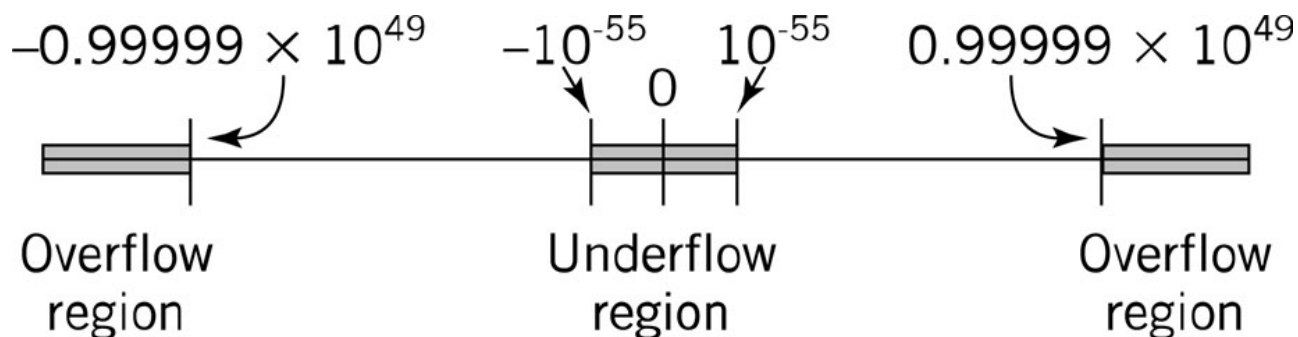
Ábrázolás	0	49	50	99
Ábrázolt kitevő	-50	-1	0	49

– Növekvő érték +



Túlcsordulás és alulcsordulás

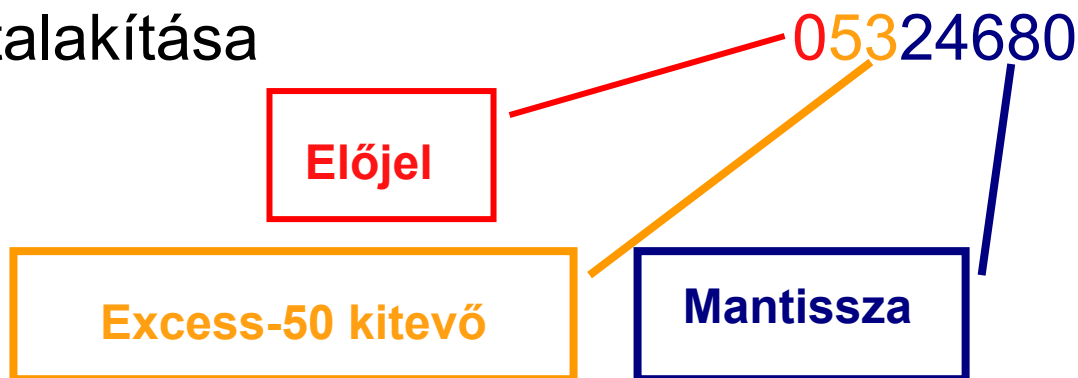
- Akkor fordul elő, ha az adott számábrázolást tekintve túl kicsi vagy túl nagy számokat szeretnénk kezelni





Példa 1: 246.8035

1. Adjunk kitevőt 246.8035×10^0
2. Pozícionáljuk a tizedesvesszőt a normál alakhoz $.2468035 \times 10^3$
3. Vágjuk le 5 jegyűre $.24680 \times 10^3$
4. Szám átalakítása





Példa 2: 1255×10^{-3}

- | | |
|-----------------------------------|-------------------------|
| 1. Már exponenciális formájú | 1255×10^{-3} |
| 2. Pozícionáljuk a tizedesvesszőt | $.1255 \times 10^{+1}$ |
| 3. Normál alakú | |
| 4. Adjunk hozzá 0-t az 5. jegyhez | $0.1255 \times 10^{+1}$ |
| 5. Szám átalakítása | 05112550 |



Példa 3: - 0.00000075

1. Exponenciális jelölés $- 0.00000075 \times 10^0$
2. Tizedespont a helyén
3. Normál alakra hozás $- 0.75 \times 10^{-6}$
4. 0 hozzáadása az 5. jegyhez $- 0.75000 \times 10^{-6}$
5. Szám átalakítása **54475000**



Átalakítás példák

$$05324567 = 0.24567 \times 10^3 = 245.67$$

$$54810000 = -0.10000 \times 10^{-2} = -0.0010000$$

$$55555555 = -0.55555 \times 10^5 = -55555$$

$$04925000 = 0.25000 \times 10^{-1} = 0.025000$$



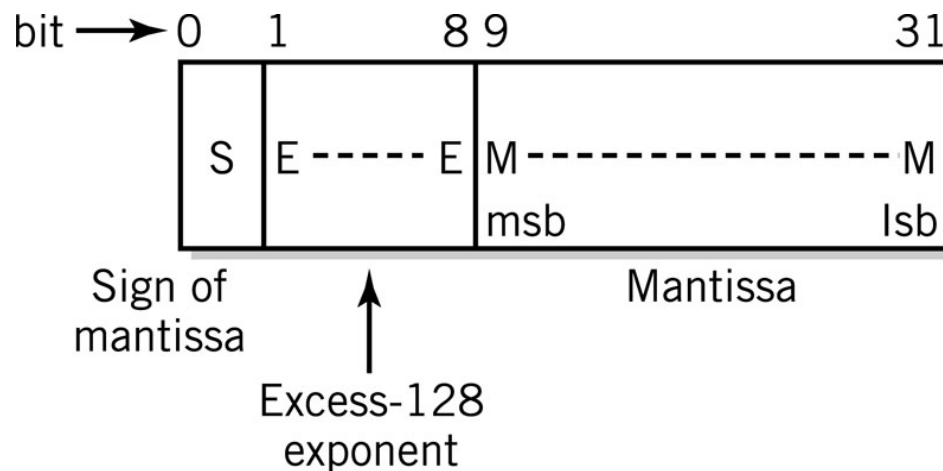
Normál alakra hozás

- Toljuk el balra a számokat növelve a kitevőt, míg a kezdő nullák el nem tűnnek
- Decimális számok konvertálása általános formába
 1. Lássuk el számmal a kitevőt (0 ha még nincs meghatározva)
 2. Növeljük/csökkentsük a kitevő értékét a tizedes pont eltolásával a helyes irányba
 3. Csökkentsük a kitevőt, hogy eltüntessük a nullákat a mantisszából
 4. Javítsunk a pontosságon 0-kat írva vagy elhagyva/kerekítve a kevésbé fontos számjegyeknél



Lebegőpont a számítógépben

- Tipikus lebegőpontos forma
 - 32 biten a mérete $\sim 10^{-38}$ -tól 10^{+38} -ig terjed
 - 8 bites kitevő = 256 szint
 - ▣ Excess-128 jelölés
 - 23/24 bites mantissza: körülbelül 7 decimális jegy pontosságú





Lebegőpont a számítógépben

Excess-128 kitevő

mantissza előjele

mantissza

0

1000 0001

1100 1100 0000 0000 0000 000

+1.1001 1000 0000 0000 00 azaz: **+0.M** x 2^{+1}

1

1000 0100

1000 0111 1000 0000 0000 000

-1000.0111 1000 0000 0000 000 azaz: **-0.M** x 2^{+4}

1

0111 1110

1010 1010 1010 1010 10101 101

-0.0010 1010 1010 1010 1010 1 azaz: **-0.M** x 2^{-2}



IEEE 754 szabvány

Pontosság	Egyszerű (32 bit)	Dupla (64 bit)
Előjel	1 bit	1 bit
Kitevő	8 bit	11 bit
Jelölés	Excess-127	Excess-1023
Mutató alap	2	2
Terjedelem	2^{-126} -tól 2^{127} -ig	2^{-1022} -tól 2^{1023} -ig
Mantissza	23	52
Decimális pontosság	≈ 7	≈ 15
Értékhatár	$\approx 10^{-45}$ -tól 10^{38} -ig	$\approx 10^{-300}$ -tól 10^{300} -ig



IEEE 754 szabvány

- 32 bit-es lebegő pontos érték definíció

Kitevő	mantissza	Érték
0	± 0	0
0	Nem 0	$\pm 2^{-126} \times 0.M$
$1-254$	Bármilyen	$\pm 2^{(E-127)} \times 1.M$
255	± 0	$\pm \infty$
255	Nem 0	Speciális eset



Átalakítás: 10-es és 2-es alap

- Két lépés
 - A számok egész és tört részét külön kell átalakítani, beágyazott tizedes vagy bináris ponttal
 - Az átalakítás végrehajtása előtt az exponenciális alakban lévő számokat vissza kell alakítani egyszerű decimális vagy bináris kevert számmá vagy törtté



Átalakítás: 10-es és 2-es alap

- 253.75_{10} átalakítása bináris lebegőpontos formába

- Szám szorzása 100-zal 25375

- Binárisra alakítás 110 0011 0001 1111 vagy
1.1000 1100 0111 11 $\times 2^{14}$

- IEEE ábrázolás 0 1000 1101 10 0011 0001 1111

Előjel

Excess-127 kitevő =
 $127 + 14$

mantissza

- Az eredeti decimális értéket visszacapjuk, ha elvégzünk egy 100_{10} -zal való osztást.



Excess 128-as illetve 127-es alak

- 128-as esetén:
 - valódi kitevő = szimbolikus kitevő - 128
- 127-es esetén:
 - valódi kitevő = szimbolikus kitevő - 128+1 azaz:
 - valódi kitevő = szimbolikus kitevő - 127



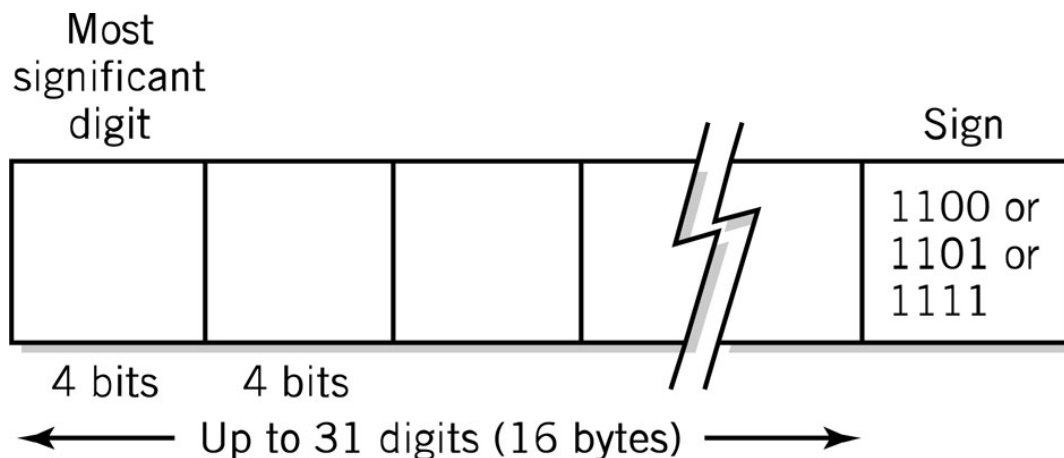
Excess 128-as illetve 127-es alak

- Például, ha a pozitív szimbolikus kitevő:
 - **1000 0001** akkor a:
 - $v_k = 129 - 128 = +1 \Rightarrow (0.M * 2^{+1})$ 128-as alak esetén,
 - $v_k = 129 - 127 = +2 \Rightarrow (1.M * 2^{+2})$ 127-es alak esetén.
- Például, ha a negatív szimbolikus kitevő:
 - **0111 1110** akkor a:
 - $v_k = 126 - 128 = -2 \Rightarrow (0.M * 2^{-2})$ 128-as alak esetén,
 - $v_k = 126 - 127 = -1 \Rightarrow (1.M * 2^{-1})$ 127-es alak esetén.



Tömörített decimális forma

- Dollárok és centek ábrázolása valós számokkal
- A vállalat-orientált nyelvek támogatják, mint a COBOL
- IBM 370/390 és Compaq Alpha





Programozási megfontolás

- Integer előnyei
 - A számítógép könnyebben használja
 - Precízebb
 - Gyorsabb
 - Kisebb tárhely és idő igény
- A legtöbb magas szintű nyelv kettő vagy több formát szolgáltat
 - Rövid egész (16 bit)
 - Hosszú egész (64 bit)



Programozási megfontolás

- Valós számok
 - Változónak vagy konstansnak van tört része
 - A nagyon kicsi vagy nagyon nagy értékű számok kívül esnek az integer ábrázolási határán
 - A program lehet, hogy nem lesz elég precíz a feladat végrehajtásához
 - Tömörített decimális számábrázolás vonzóbb a vállalati alkalmazásoknál



3. Fejezet : Adat formátumok

**The Architecture of Computer Hardware
and Systems Software:
An Information Technology Approach**

**3. kiadás, Irv Englander
John Wiley and Sons ©2003**

**Wilson Wong, Bentley College
Linda Senne, Bentley College**



Adat formátumok

- Számítógépek
 - Minden adatformátumot bináris formában kezel és tárol
- Emberi kommunikáció
 - Beleértve a nyelveket, képeket és hangokat
- Adat formátumok:
 - Előírások az adatok számítógép által kezelhető formába konvertálásához
 - Különböző módok emberi adat ábrázolására a számítógép által feldolgozva és tárolva



Adatok forrása

- Bináris bemenet
 - Egyedi inputként indul
 - Példa: billentyűzeti bemenet **$1+2=3$ számolás**
 - Billentyűzet generál egy bináris számot minden gombhoz
- Analóg
 - Összefüggő adatok mint a hang vagy a képek
 - Szükséges egy hardver az adatok bináris számokká konvertálásához

**$1+2=3$
számolás**



**Input
eszköz**



Számítógép

1101000101010101...



Általános adatábrázolás

Adat típus	Szabvány(ok)
Alfanumerikus	Unicode, ASCII, EDCDIC
Kép (bit térképes)	▪ GIF (graphical image format) ▪ TIF (tagged image file format) ▪ PNG (portable network graphics)
Kép (vektorgrafikus)	PostScript, SWF (Macromedia Flash), SVG
Grafikai megjelenés, ill. betűkészlet leírás	PostScript, TrueType
Hang	WAV, AVI, MP3, MIDI, WMA
Formátumozott dokumentum leírás	PDF (Adobe Portable Document Format), HTML, XML
Video	Quicktime, MPEG-2, RealVideo, WMV



Belső adatábrázolás

- Kifejezi az
 - Input forrás összetettségét
 - A szükséges feldolgozás módját
- Optimális ábrázolás szempontjai
 - pontosság és felbontás
 - ▣ egyszerű fénykép szemben egy festmény-albummal
 - tömörség (tárolás és továbbítás)
 - ▣ több adat szükséges a jobb pontossághoz és felbontáshoz
 - ▣ *Tömörítés*: adatábrázolás tömörebb formában
 - ▣ *Metadata*: adat, ami leírja vagy meghatározza az adat értelmét
 - Módosítások egyszerűsödése:
 - ▣ Egyszerű audió szemben a Hi-Fi minőségű hanggal
 - Szabványosítás
 - ▣ *Szabadalmazott formátumok* adattároláshoz és feldolgozáshoz (WordPerfect vs. Word)
 - ▣ Tényleges szabványok: szabadalmazott szabványok az általánosan használt formákon alapulva (PostScript)



Adattípusok: Számadat

- Matematikai számításokhoz használják
 - Összeadás, kivonás, szorzás, osztás
- Típusok
 - Integer (egész számok)
 - Real (tartalmaz egy tizedespontot)



Adattípusok: Alf numerikus

- Alf numerikus:
 - Karakterek: *b T*
 - Számok: *7 9*
 - Írásjelek: *! ;*
 - Speciális karakterek: *\$ &*
- Számjegy karakterek és számok
 - Mindegyiket általános karakterként vesszük be
 - Számítógép konvertálja számokká a számításokhoz
 - ▢ Példák: változók deklarálása számokként a programozó által (int fut=0)
 - Karakterként kezeli, ha szöveggént kerül feldolgozásra
 - ▢ Példák: telefonszámok, irányítószámok



Alfanumerikus kódok

- Tetszőleges bit-sorozatok választása a karakterek ábrázolására
 - Követelmény: input és output eszköznek ismernie kell a kódolást
 - Bináris szám értékének karakter ábrázolása összhangban van az abc-ben való elhelyezkedésével
 - ▣ Elősegíti a rendezést és keresést



A 852-es kódlap

0→	20→¶	40→{	60→<	80→P	100→d	120→x	140→î	160→á	180→ı	200→Ł	220→■	240→-
1→@	21→\$	41→)	61→=	81→Q	101→e	121→y	141→ž	161→í	181→Ā	201→Œ	221→ı	241→~
2→0	22→_	42→*	62→>	82→R	102→f	122→z	142→Ĥ	162→ó	182→Ĥ	202→Œ	222→Ů	242→
3→♥	23→↑	43→+	63→?	83→S	103→g	123→{	143→Č	163→ú	183→Ě	203→Œ	223→■	243→
4→♦	24→↑	44→,	64→@	84→T	104→h	124→	144→É	164→Ā	184→\$	204→Œ	224→Ó	244→~
5→♠	25→↓	45→-	65→A	85→U	105→i	125→}	145→Ĺ	165→a	185→Œ	205→=	225→B	245→\$
6→♣	26→→	46→.	66→B	86→V	106→j	126→~	146→Í	166→Ž	186→Œ	206→Œ	226→Ů	246→÷
7→·	27→+	47→/	67→C	87→W	107→k	127→û	147→ô	167→ž	187→Œ	207→D	227→N	247→
8→■	28→_	48→0	68→D	88→X	108→l	128→Ç	148→ö	168→Ě	188→Œ	208→đ	228→ñ	248→ó
9→o	29→+	49→1	69→E	89→Y	109→m	129→ü	149→Ĺ	169→e	189→ž	209→D	229→ñ	249→"
10→o	30→▲	50→2	70→F	90→Z	110→n	130→é	150→Ĺ	170→	190→ž	210→Ď	230→Š	250→.
11→♂	31→▼	51→3	71→G	91→[	111→o	131→â	151→Š	171→ž	191→Œ	211→Ě	231→š	251→Ů
12→♀	32→	52→4	72→H	92→\	112→p	132→ä	152→š	172→Č	192→Œ	212→ď	232→Ř	252→Ř
13→♪	33→!	53→5	73→I	93→]	113→q	133→û	153→ö	173→š	193→Œ	213→Ň	233→Ů	253→ř
14→♫	34→"	54→6	74→J	94→^	114→r	134→ć	154→Ü	174→«	194→Œ	214→Ī	234→ř	254→■
15→×	35→#	55→7	75→K	95→	115→s	135→č	155→Ī	175→»	195→Œ	215→Ī	235→Ů	255→
16→►	36→\$	56→8	76→L	96→	116→t	136→ł	156→t	176→	196→-	216→ě	236→ý	
17→◄	37→%	57→9	77→M	97→a	117→u	137→ë	157→ł	177→	197→Œ	217→Ĵ	237→Ů	
18→↑	38→&	58→:	78→N	98→b	118→v	138→ő	158→x	178→	198→Ā	218→Œ	238→t	
19→!!	39→'	59→;	79→0	99→c	119→w	139→ö	159→č	179→	199→ä	219→■	239→	



Karakterek ábrázolása

- ASCII – legszélesebb körben használt
- EBCDIC: IBM nagyszámítógép (hagyományos)
- Unicode: világméretű használatra kifejlesztve



ASCII

- ANSI (American National Standards Institute) által kifejlesztett
- Ábrázolás
 - Latin betűk, arab számok, általános írásjelek
 - Plusz néhány ékezetes és más európai speciális karakterek
- ASCII
 - 7-bites kód: 128 karakter



ASCII referencia tábla

MSD \ LSD	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P		p
1	SOH	DC1	!	1	A	Q	a	W
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACJ	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

74₁₆
111 0100



EBCDIC

- Extended Binary Coded Decimal Interchange Code az IBM által kifejlesztve
 - Főként IBM vagy IBM kompatibilis nagyszámítógépek körében használatos
 - ASCII-ba/ból konvertáló szoftver hozzáférhető
 - Gyakori az archív adatoknál
 - Karakterkódok eltérnek az ASCII-től

	ASCII	EBCDIC
Space	20 ₁₆	40 ₁₆
A	41 ₁₆	C1 ₁₆
b	62 ₁₆	82 ₁₆



Unicode

- Leggyakoribb, 16-bites forma, 65.536 karakter ábrázolására alkalmas
- ASCII Latin-I Unicode része
 - 0-tól 255-ig a Unicode táblában
- Soknyelvű: kódokat határoz meg
 - Majdnem az összes betű alapú karakterhez
 - Nagy része a kínai, japán és koreai „szóképeknek”
 - Vegyesen magánhangzók és szótag csoportok néhány nyelv igénye szerint
- Támogatja a szoftvermódosítást helyi nyelvekre



Összehasonlító sorozat

- Alfabetikus rendezés ha a szoftver kezeli a vegyes kis- és nagybetű kódokat
- ASCII-ban a számokat helyezi előre; EBCDIC-ben utoljára
- ASCII rendezési sorrend karakterláncokra

Betűk							
Adam	A	d	a	m			
Adamian	A	d	a	m	i	a	n
Adams	A	d	a	m	s		

Szám karakterek				
1	011	0001		
12	011	0001	011	0010
2	011	0010		



A kódok kettő osztálya

- *Nyomtatható* karakterek
 - Megjelenik a képernyőn vagy a nyomtatón
- *Vezérlő* karakterek
 - Pozícionálja a kimenetet a képernyőn vagy a nyomtatón
 - ▣ VT: vertical tab (tabulátor) ▣ LF: Line feed (újsor)
 - Tevékenységet vált ki
 - ▣ BEL: bell rings (csipogás) ▣ DEL: aktuális karakter törlése
 - Állapotot közöl a számítógép és az I/O eszköz között
 - ▣ ESC: a soron következő karakter értelmezésének megváltoztatása



Vezérlőkódok meghatározása

NUL	(Null) No character; used to fill space	DLE	(Data Link Escape) Similar to escape, but used to change meaning of data control characters; used to permit sending of data characters with any bit combination
SOH	(Start of Heading) Indicates start of a header used during transmission	DC1, DC2, DC3, DC4	(Device Controls) Used for the control of devices or special terminal features
STX	(Start of Text) Indicates start of text during transmission	NAK	(Negative Acknowledgment) Opposite of ACK
ETX	(End of Text) Similar to above	SYN	(Synchronous) Used to synchronize a synchronous transmission system
EOT	(End of Transmission)	STB	(End of Transmission Block) Indicates end of a block of transmitted data
ENQ	(Enquiry) A request for response from a remote station; the response is usually an identification	CAN	(Cancel) Cancel previous data
ACK	(Acknowledge) A character sent by a receiving device as an affirmative response to a query by a sender	EM	(End of Medium) Indicates the physical end of a medium such as tape
BEL	(Bell) Rings a bell	SUB	(Substitute) Substitute a character for one sent in error
BS	(Backspace)	ESC	(Escape) Provides extensions to the code by changing the meaning of a specified number of contiguous following characters
HT	(Horizontal Tab)	FS, GS, RS, US	(File, group, record, and united separators) Used in optional way by systems to provide separations within a data set
LF	(Line Feed)	DEL	(Delete) Delete current character
VT	(Vertical Tab)		
FF	(Form Feed) Moves cursor to the starting position of the next page, form, or screen		
CR	(Carriage return)		
SO	(Shift Out) Shift to an alternative character set until SI is encountered		
SI	(Shift In) see above		



Billentyűzet bemenet

■ *Scan kód*

- Két különböző scan kód van a billentyűzeten
 - ▢ Egyik akkor keletkezik ha egy gombot lenyomunk, a másik, ha felengedünk egy gombot
- A szoftver által terminálon vagy PC-n Unicode -dá, ASCII –vé vagy EBCDIC –ké konvertálva

■ Előny

- Könnyen alkalmazható különböző nyelveknél vagy billentyűzetkiosztásoknál
- Különböző scan kód a gombok megnyomásához/felengedéséhez és a billentyűzetkombinációkhoz
 - ▢ Példák: shift és control gombok



Más alfanumerikus bemenet

- **OCR** (optical character reader/optikai jelolvasó)
 - Nyomtatott szöveget olvas be és szöveges adattá konvertálja
 - Különlegesen kódolt karakterek olvasására használják
 - Általános használat korlátozott a nagymértékű hibaarány miatt
 - ▢ mágneses felületek „optikai” felismerése (pl. csekkeken vízjel helyett)
- **Vonalkód olvasók**
 - Olyan alkalmazásokban használják, ami gyors, pontos, ismétlődő inputot igényel minimálisan képzett alkalmazottakkal
 - Példák: bevásárlóközpont pénztár- és leltár kezelése
 - Alfa numerikus adat a vonalkód olvasóban optikailag vonalakat használ
- **Mágneskártya olvasók:** alfanumerikus adat bankkártyákról
- **Hang**
 - Digitalizált audió rögzítése egyszerű, de alfanumerikus adattá konvertálása nehéz
 - ▢ Ismerni kell a hangmintákat egy nyelvben (*fonéma/beszédhang*) plusz a kiejtés-, nyelvtan- és mondattan szabályait



Kép adat

- Fényképek, ábrák, ikonok, rajzok, táblázatok és grafikonok
- Kétféle megközelítés:
 - *Bit-térképes* vagy *raszteres (rácsos) képek*
 - Képek és festmények összefüggő váltakozása
 - *Objektumos* vagy *vektorgrafikus*
 - *grafikai objektumokból* áll össze, mint a vonalak és görbék, geometriailag meghatározva
- Különbségek:
 - Kép minősége
 - Tárhely igény
 - Továbbadás időtartama
 - Módosítás könnyedsége



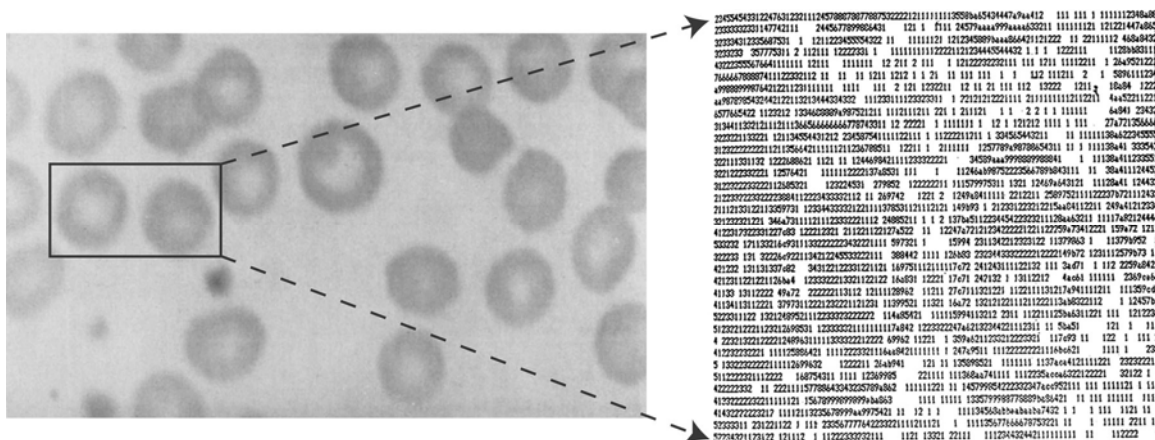
Bit-térképes képek

- Valóságghű, folytonosan változó árnyékú, színű, élességű és szerkezetű képeknél használják
 - Példák:
 - ▣ Scanner-elt képek
 - ▣ Képtár egy *rajz*-program által létrehozva
- Főként nagymértékben részletezett és meglehetősen egyszerű feldolgozás igényű képeknél használják
- Input eszközök:
 - Scanner-ek
 - Digitális kamerák és videó felvevő eszközök
 - Grafikus input eszközök, mint az egér és a toll
- *Képszerkesztő*- vagy *rajz program* kezeli
 - Szerkesztő eszközök, amelyek könnyebbé teszik a fárasztó bit-ről bit-re való feldolgozást



Bit-térképes képek

- Egy ábrában minden egyes *pixel* (*pi(x)cture element* / képpont) bináris számként van tárolva
 - Pixel: kis terület koordinátákkal meghatározva
 - Példa: minden 4 bites kódon ábrázolt pontnak képes megfeleltetni a 16 szürke árnyalat egyikét





Bit-térkép megjelenítés

- Monochrome: fekete vagy fehér
 - 1 bit pixelenként
- Szürke: fekete, fehér illetve 254 árnyalata a szürkének
 - 1 byte pixelenként
- Színes képek: 16 szín, 256 szín, vagy 24 bit-es true color (16,7 millió szín)
 - 4, 8, és 24 bit-es (3 x 8 bit-es) képek



Bit-térképes képek tárolása

- Gyakran nagy fájlok
 - Példa: 800×600 pixel, mindhárom szín (RGB) 1 byte-on tárolva ➡ ~1.5MB –os file-t ad (1.37 MB)
- A file méret függ a
 - *Felbontástól* (pixelek száma inch-enként)
 - ▢ Kihatással van a kép élességére és részletességére
 - *Mélységtől*: szürke vagy összetett színek megjelenítéséhez használt bit-ek száma
 - ▢ *Paletta (színválaszték)*: színtábla, ami minden pixelhez kódot használ az aktuális szín értéke helyett
 - Adattömörítéstől

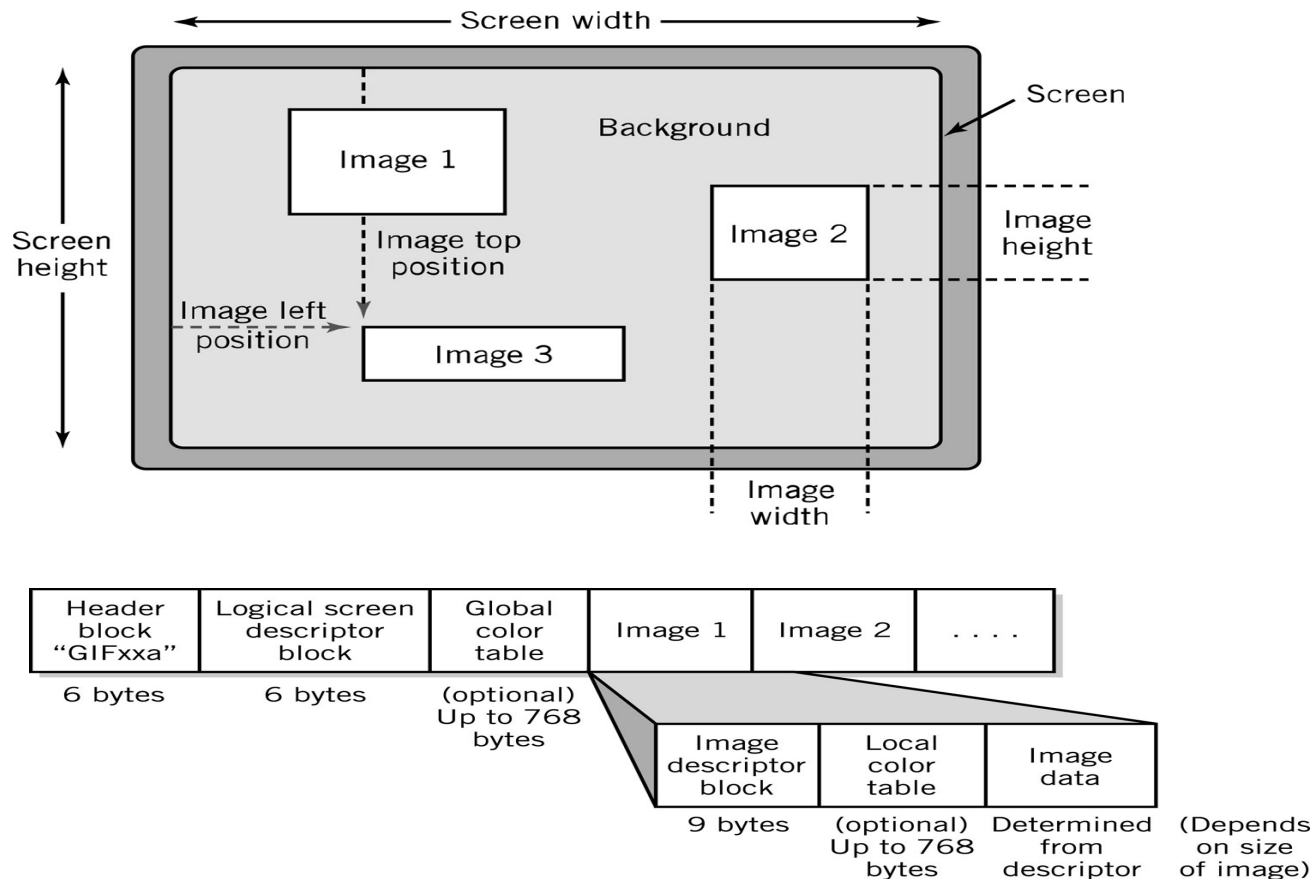


GIF (Graphics Interchange Format)

- 1987-ben fejlesztette ki a CompuServe
- GIF89a animált képek lehetősége
 - Lehetővé teszi a képek egymást követő megjelenítését rögzített időközönként
- Színkorlát: 256
- LZW (Lempel-Zif-Welch) algoritmussal tömörítve
- Főként vonalrajzoláshoz, művészi és nagyméretű azonos színű területeket tartalmazó képekhez
- *Veszteségmentes tömörítés*



GIF (Graphics Interchange Format)





JPEG

(Joint Photographers Expert Group)

- Több mint 16 millió színre képes
- Megfelelő nagy részletességű fényképekhez és festményekhez
- *Veszteséges tömörítési* algoritmust alkalmaz, amely
 - Adatokat eldobva csökkenti a file méretét és az átviteli sebességet
 - Csökkentheti a kép felbontását, elmoshatja az éles vonalakat
 - Minden egyes mentés "rontja" a kép minőségét



Más bit-térképes formátumok

- TIFF (Tagged Image File Format/címkézett kép file formátum): *.tif*
 - Kiváló minőségű képek feldolgozása, különösképp kiadásnál
- BMP (BitMapped/Bittérképes): *.bmp*
 - Eszköz-független formátum Microsoft Windows környezethez: pixel színek a kimeneti eszközöktől függetlenül tárolva
- PCX: *.pcx*
 - Windows Paintbrush formátuma
- PNG: (Portable Network Graphics): *.png*
 - Internetes alkalmazásoknál a GIF és JPEG lecserélésére tervezték
 - Szabadalom mentes
 - Továbbfejlesztett *vesztéségmentes* tömörítés
 - Nincs animáció támogatás



Vektorgrafikus képek

- Csomagok *rajzolásával* vagy ábratáblázattal készítik
- Különböző színű vonalakból és alakzatokból áll össze
- A számítógép lefordítja a geometriai formulákat a kép megalkotásához
- Tárhely mérete függ a kép összetettségétől
 - Utasítások száma egy vonal rajzolásához, alakzatok, minták mennyisége
- Filmek, így a *Shrek* és *Toy Story* megalkotásához is vektorgrafikát használtak



Vektorgrafikus képek

- Matematikai képleteken alapul
 - Könnyű mozgatni, méretezni és elforgatni az élesség elvesztése nélkül, a bit-térképesek képekkel szemben
- Kevesebb helyre van szüksége, mint a bit-térképes képeknek
- Nem tud képeket vagy festményeket ábrázolni
- Nem lehet közvetlenül megjeleníteni vagy nyomtatni
 - Bit-térképesé kell konvertálni, mivel a rajzgép bit-térképes



Ismertebb vektorgrafikus programok

- A legtöbb vektorgrafikus formátum szabadalmazott
 - File kiterjesztéseket is beleértve .wmf, .dxf, .mgx, és .cgm
- Macromedia Flash: alacsony sávszélességű animáció
- Micrographx Designer: tervrajzkészítés, a termékek illusztrálásához
- CorelDraw: vektoros illusztráló, elrendező, bit-térkép készítő, képszerkesztő, festő és animáló program
- Autodesk AutoCAD: építészeknek, mérnököknek, tervezőknek, és tervfüggő szakembereknek
- W3C SVG (Scalable Vector Graphics) XML Web leíró nyelven alapul
 - Nem szabadalmazott



PostScript

- *Oldalleíró nyelv*: műveletek és tulajdonságok listája, amely jellemzi az összes objektumot a nyomtatandó oldalon
 - ASCII-ben vagy Unicode-ban tárolva
 - Az értelmező program a számítógépen vagy a kimeneti eszközön olvassa a PostScript-et a kép konvertálásához
- Méretezhető betű támogatás
 - Többi objektumhoz hasonló meghatározás



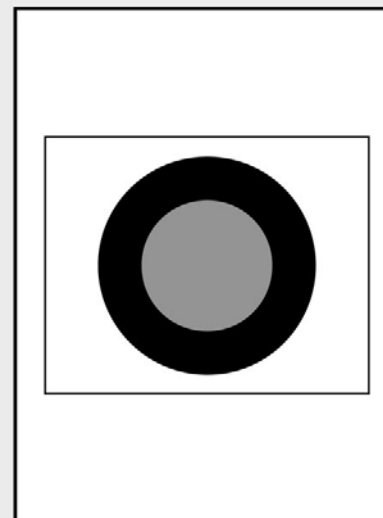
PostScript program

```
288 396 translate % move origin to center of page
0 0 144 0 360 arc % define 2" radius black circle
fill
```

```
0.5 setgray % define 1" radius gray circle
0 0 72 0 360 arc
fill
```

```
0 setgray % reset color to black
-216 -180 moveto % start at lower left corner
0 360 rmoveto % and define rectangle
432 0 rmoveto % ...one line at a time
0 -360 rmoveto
closepath % completes rectangle
stroke % draw outline instead of fill
```

```
showpage % produce the image
```





Karakterek ábrázolása

- Karakterek tárolása Unicode-hoz vagy ASCII-hez hasonló formátumokban
 - A szöveg elsősorban tartalom szerint kerül feldolgozásra és tárolásra
- Megjelenítési feltételek, mint a betűkészlet tárolása karakterrel
 - Szövegmegjelenés az elsődleges tényező
 - ▣ Példa: betűkészlet a Windows-ban
- *Glyph-ek*: Macintosh kódolási minta, amely tartalmazza a karakterek azonosítási és megjelenési feltételeit



Bit-térképes és vektorgrafikus képek összehasonlítása

Bit-térképes	Vektorgrafikus
Pixel térkép	Geometriailag meghatározott formák
Fénykép minőség	Összetett rajzok
Festő program	Rajz program
Nagyobb tárhely igény	Nagyobb számítási igény
Kép méretének növelése élettenséget okoz	Objektumok méretezése egyenletes
A kimenet felbontását a kép felbontása korlátozza	A kimenet felbontását a kimeneti eszköz korlátozza



Videó képek

- Nagymennyiségű adatra van szükség
 - A videokamera 720x576/704x480 (PAL/NTSC) felbontású 24 bit-es képet rögzít 25/30 kép/másodperc sebességgel.
 - Ez 248 Mbit/sec a CCD-n, a szalagon átlagosan 28.8 Mbit/sec.
 - 1 perces film 1.7 Gbit és egy 1 órás film közel 13 Gbyte!
- Lehetőségek a file méret csökkentésére: csökkenti a kép méretét, korlátozza a színek számát, csökkenti a képkocka-arányt
- A videohoz való hozzájutási módtól függően
 - *Videó folyam*: a videó úgy jelenik meg, ahogy letöltöttük a Web szerverről
 - Példa: videó konferencia
 - Helyi adat (file a DVD-re vagy a rendszerre letöltve) a jobb minőségért
 - MPEG-2: kódolási példa: IBBPBBPBBI
 - Intra, Bidirectional, Predictive, film minőségű képek, magas tömörítéssel, tekintélyes feldolgozási képességet igényel,
 - $3 \div 10$ Mbit/sec tömörítési arányra optimalizálva, ez a DVD szabványa is. A file méret = 95% videó-adat + 5% tömörítetlen audió-adat.

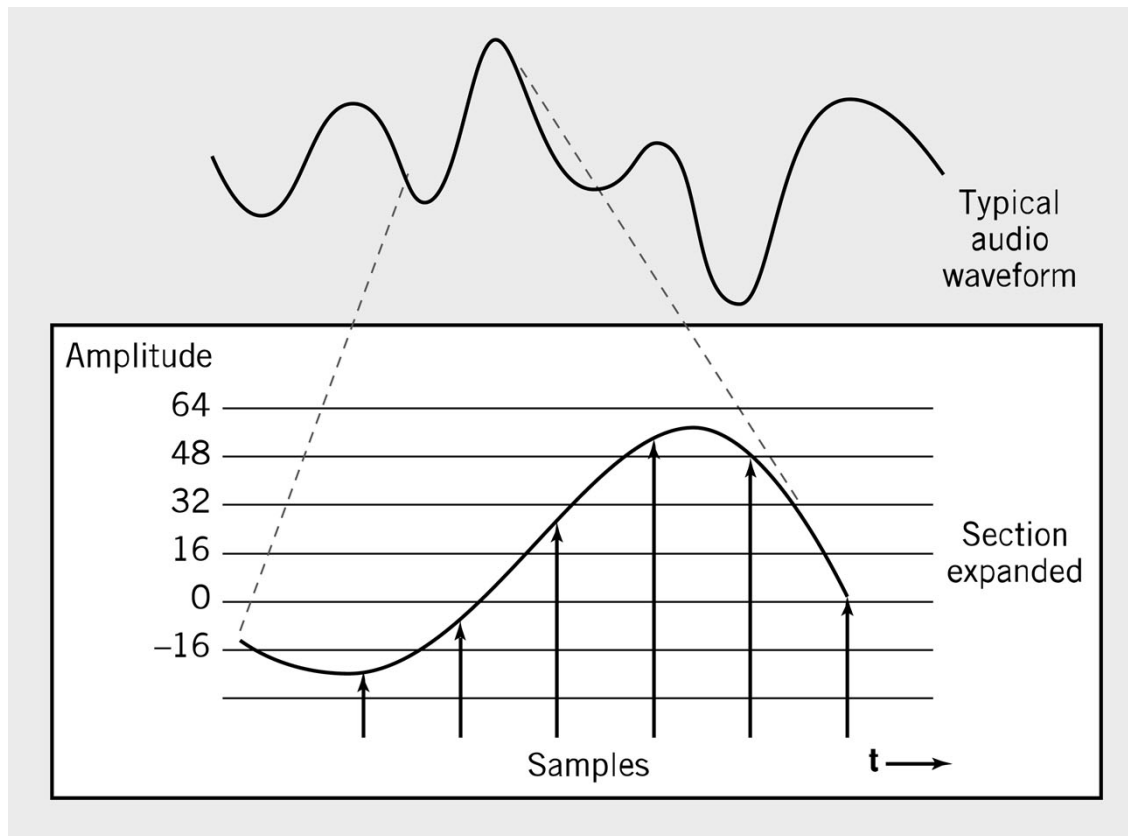


Audió adat

- Átviteli és feldolgozási igényei kisebbek, mint a videoé
- *Waveform audio*: a hang digitális ábrázolása
- *MIDI* (Musical Instrument Digital Interface): utasítások a hangok előállításához
- Az *A-D (analóg-digitális) konverter* analóg hangot konvertál a digitális értékekre



Waveform Audio



Mintavételezési
arány
rendszerint
50kHz



Mintavételezési arány

- A hangból vett mintavételek száma másodpercenként a rögzítési folyamat alatt
 - 1000 mintavétel másodpercenként = 1 kHz (kilohertz), azaz ezred másodpercenként 1.
 - Példa: Audió CD mintavételezési aránya = 44.1 kHz, azaz 226 tízmilliomod másodpercenként 1.
- Különböző minőségű rögzítések:
 - 8 bit-es kvantáltság a rádió minőségű rögzítéshez,
 - 16 bit-esnél 44.1 kHz CD, 48 kHz a Hi-Fi minőségű rögzítéshez, 2 sztereó csatornán,
 - 12 bit-es kvantáltság 32 kHz-en 2 x 2 sztereó csatornán rögzíti a hangot,
 - 24 bit-es kvantáltság 192 kHz-en 5.1 hangkép audió DVD-n.



MIDI

- Zene jelölési rendszer, amely lehetővé teszi a számítógépek kommunikációját a szintetizátorokkal
- Utasítások, melyeket a MIDI műszerek és a MIDI hangkártyák használnak a hang visszajátszásához.
 - Nem alkalmas beszéd- vagy énekhang tárolására vagy visszajátszására
 - Sokkal tömörebb, mint a waveform
 - 3 perc = 10 kB



Audió formátumok

- *MP3*

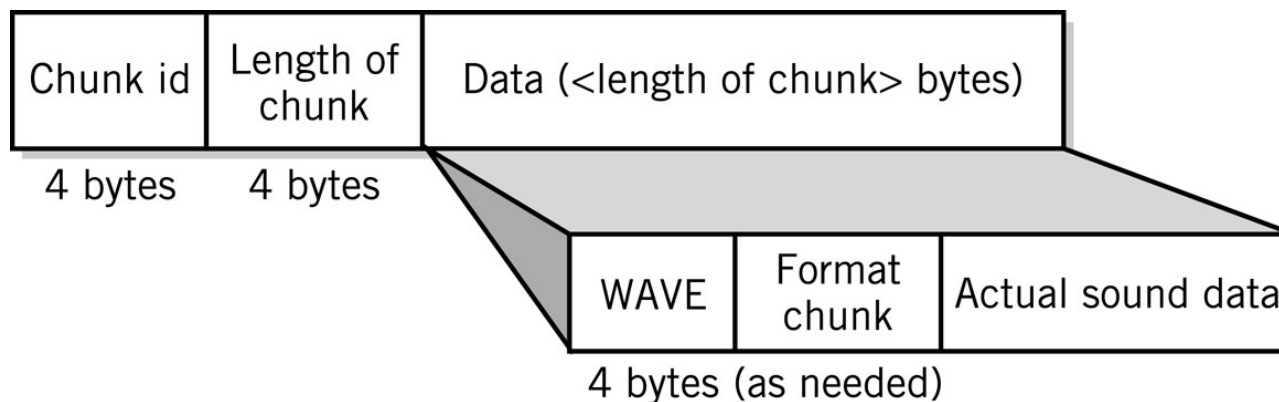
- Az MPEG-2 leszármazottja (ISO *M*oving *P*icture *E*xperts *G*roup)
- *Psychoacoustic*–kus tömörítési technikát alkalmaz a tárhelyigény csökkentéséhez
- Eldobja az ember számára hallhatatlan hangokat: veszteséges tömörítés

- *WAV*

- Microsoft fejlesztette ki a multimédiás környezetének a részeként
- Általános felhasználású veszteségmentes tömörítés



.WAV hang formátum





Adattömörítés

- *Tömörítés*: adatrögzítés, amelynek kevesebb tárhelyre van szüksége
- *Tömörítési arány*: méretes file kicsinyítése
- *Veszteség mentes*: a fordított algoritmus visszaállítja az adatot eredeti formájába
 - Példák: GIF, PCX, TIFF, WAV
- *Veszteséges*: adatvesztés lép fel a kisebb méret és a gyorsabb letöltés kedvéért
 - Sokkal nagyobb tömörítési arányok, általában 1-től 10-ig
 - Példa: JPEG
 - Gyakori a multimédiában
- MPEG-2: mindkettőt használja 100:1-es arányhoz



Tömörítő algoritmusok

- Ismétlődés

- 0 5 8 7 0 0 0 0 3 4 0 0 0 → 0 1 5 8 7 0 4 3 4 0 3

- Példa: nagy területek ugyanazzal a színnel

- Minta helyettesítés

- Mintákat keres az adatban

- Behelyettesíti az új mintákat, szótárat hoz létre

- Példa: 45-ből 30 byte plusz szótár

⌘	Pe	✱	pi	✧	ed
✱	er	◎	ck	⊕	pe
☆	Pi				

- Peter Piper picked a peck of pickled peppers.

- ⌘t✱☆p✱✱✱◎✧ a ⊕◎ of ✱◎|✧ ⊕pp✱s.



Belső számítógép adatformátum

- Minden adat bináris számként van rögzítve
- Értelmezés alapjai
 - A számítógép képes a műveletek végrehajtására
 - Programozási nyelvek adattípus támogatása az alkalmazások készítéséhez



Öt egyszerű adattípus

- *Boolean*: kettő értékű változó vagy konstans, amelynek értéke igaz vagy hamis
- *Char*: változó vagy konstans, amelynek értéke alfanumerikus karakter
- *Enumerated* (felsorolt):
 - Felhasználó által meghatározott adattípusok, a definícióban megadott értékeket veheti fel
 - Type DayOfWeek = Mon, Tues, Wed, Thurs, Fri, Sat, Sun
- *Integer*: pozitív vagy negatív egész számok
- *Real*:
 - Számok tizedesponnttal
 - Számok, amelyek értéke olyan nagy vagy kicsi, hogy meghaladja a számítógép integer tárolási határát



Adatok ábrázolása, adattípusok

Összefoglalás



Adatok ábrázolása, adattípusok

- Számítógépes rendszerek
 - működés: információfeldolgozás
 - IPO: input-process-output modell
 - információ tárolása adatok formájában
 - adatformátum választása:
 - ▣ gépi feldolgozás, tárolás egyszerű legyen – Neumann János → bináris
 - ▣ adatformátum választása: az adatokon műveleteket egyszerűen SZISZTEMATIKUSAN



Adatokon végzett műveletek (IPO modell)

- Információ számítógép által feldolgozható formába rögzíteni:
 - számítógép perifériák (bemeneti egységek)
- Információ feldolgozása
 - CPU, memória
- Eredmények megjelenítése:
 - számítógép perifériák (kimeneti egységek)



6. Fejezet: Little Man Computer (A kisember számítógép)

**A számítógép hardver és szoftver
architektúrája:**

Egy informatikai megközelítés

3. kiadás, Irv Englander

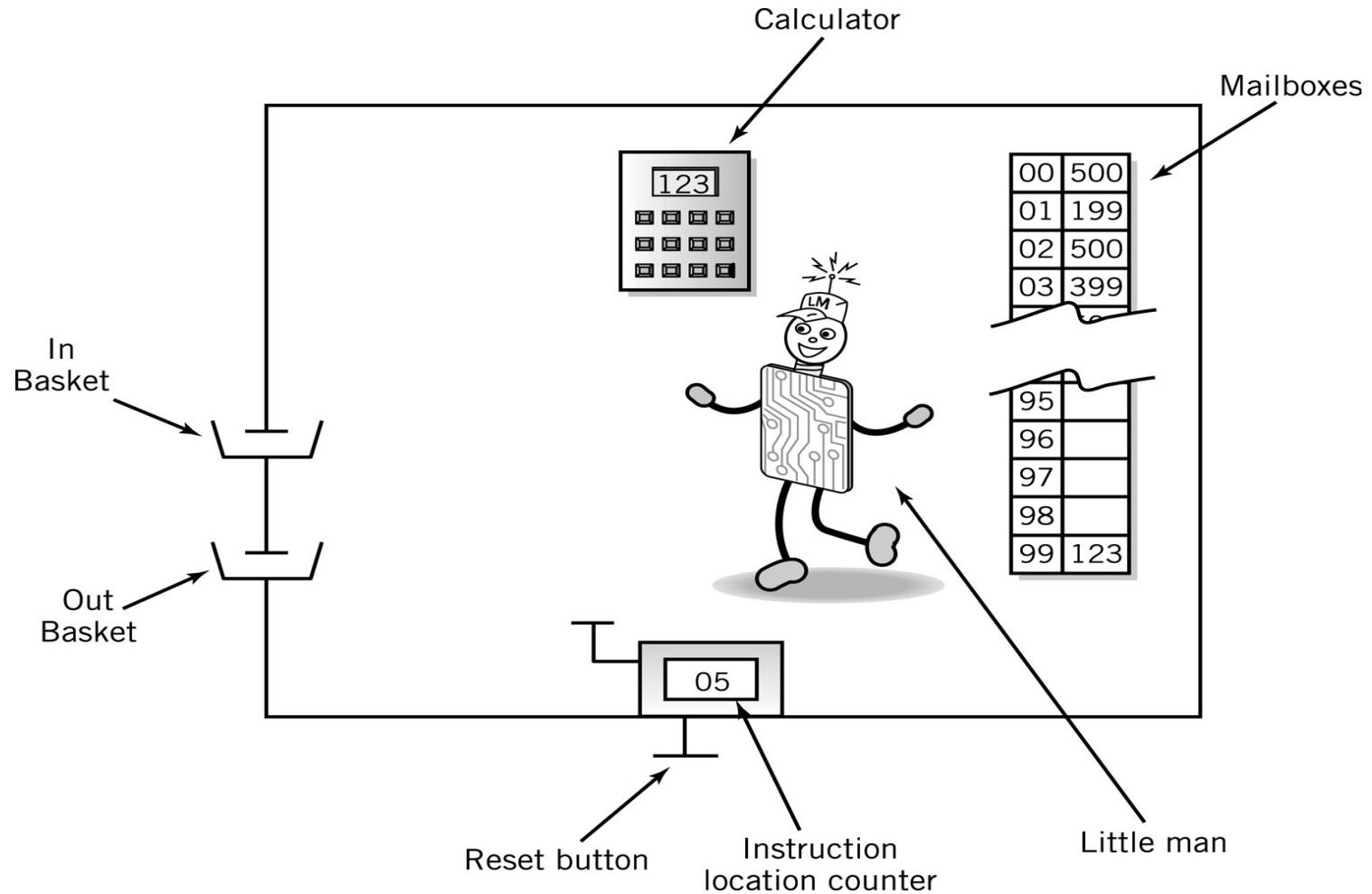
John Wiley and Sons ©2003

Wilson Wong, Bentley College

Linda Senne, Bentley College



Little Man Computer (LMC)





Postaládák: Cím vs. Tartalom

- A postaláda címek egymást követik
- A postaláda tartalma lehet
 - adat vagy
 - utasítás

Cím	Tartalom



Tartalom: Utasítások

- Műveleti kód
 - művelet kódja
 - tetszőleges rövidítés (mnemonic)
- Művelet operandusa
 - módosítandó „objektum”
 - ▣ adat értéke vagy
 - ▣ adat címe, ahol az érték tárolódik

Cím	Tartalom	
	Műveleti kód	Operandus



Varázslat!

- Program betöltése a memóriába
- Adatok belehelyezése a bemeneti kosárba („In Basket”)



Assembly nyelv

- CPU specifikus
- egy az egyhez összerendelés az assembly nyelv utasításai és a bináris (CPU által végrehajtható) utasítások között
- *Mnemonic-ok* (rövid karakter sorozatok) utasításokat reprezentálnak
- Akkor használjuk, ha a programozónak a hardver pontos vezérlésére van szüksége
 - pl.: eszközmeghajtók (driver-ek) készítése



Utasításkészlet

Aritmetikai	1xx	Összeadás
	2xx	Kivonás
Adat mozgató	3xx	Tárolás
	5xx	Betöltés
Input/Output	901	INPUT
(Be-/Kimenet)	902	OUTPUT
Gép irányítása (kávészünet)	000	Leállítás COB



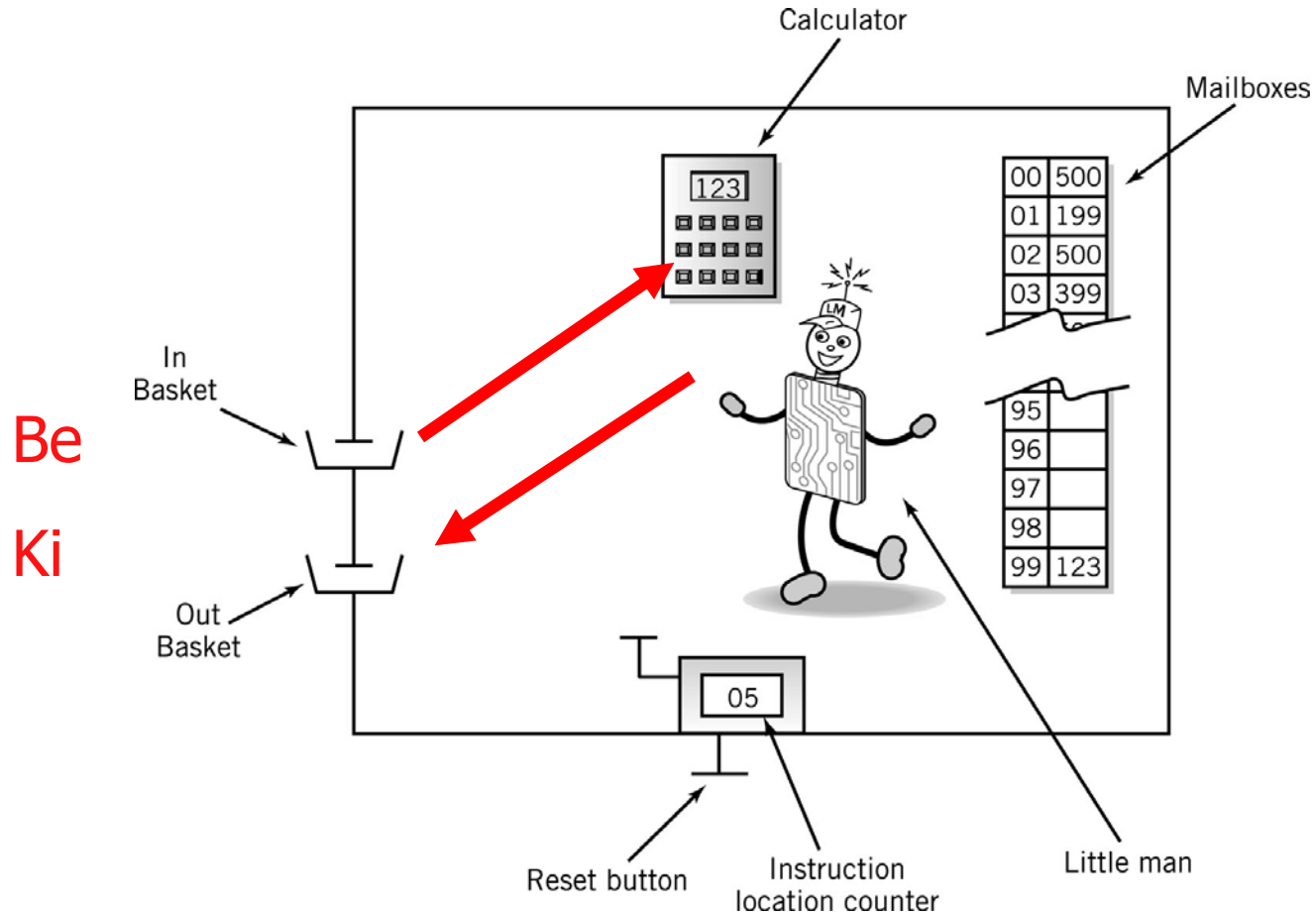
Input/Output

- Adatok mozgatása a számítógép és a be/ki kosarak között

	Tartalom	
	Műveleti kód	Operandus
IN (input)	9	01
OUT (output)	9	02



LMC Input/Output





Belső adatmozgás (LMC-n belül)

- A postaláda és a számológép között

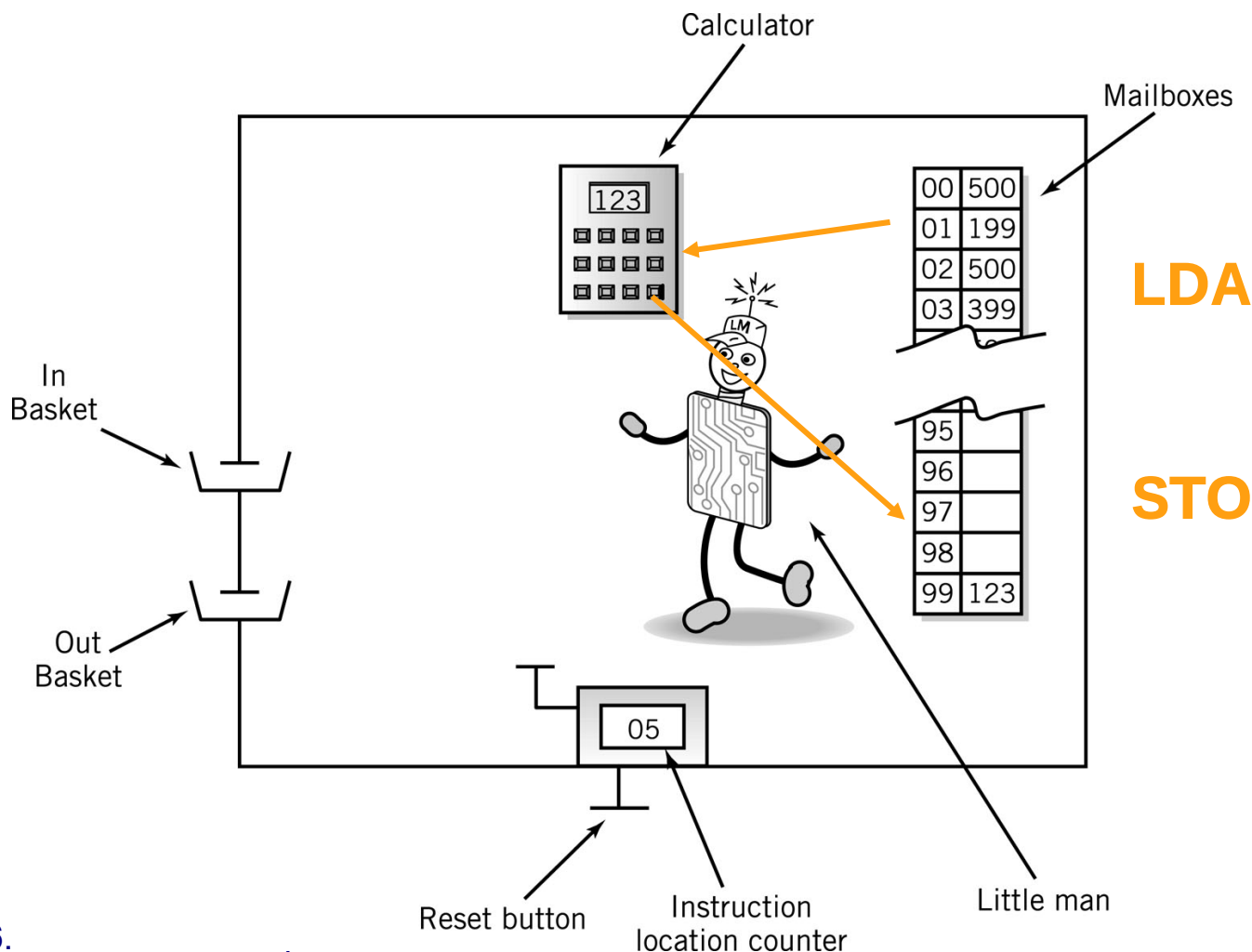
STO
(tárolás)

LDA
(betöltés)

Tartalom	
Műveleti kód	Operandus
3	XX
5	XX



LMC belső adatmozgatás





Adatok tárolása

- Utasításokat és az adatokat tároló postaláda fiókok fizikailag azonosak
- Adatok (általában) nincsenek beleágyazva az utasítások közé
- *DAT* mnemonic azonosítja őket az assembly kódban



Aritmetikai utasítások

- Postaládában tárolt érték beolvasása
- Művelet elvégzése a számológép segítségével

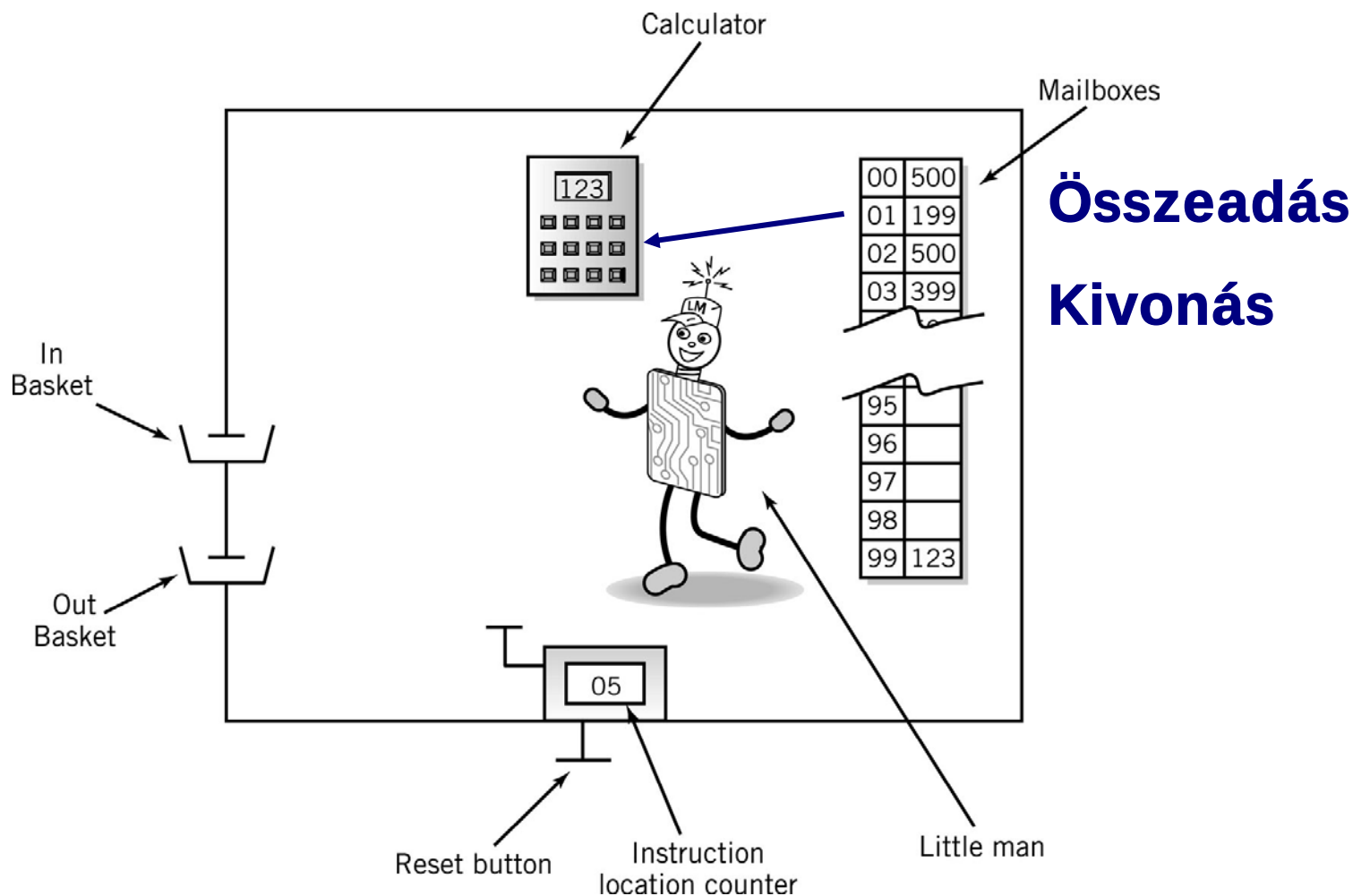
Összeadás

Kivonás

Tartalom	
Műveleti kód	Operandus
1	xx
2	xx



LMC aritmetikai utasítások





Egyszerű program: 2 szám összeadása

- Feltételezzük, hogy az adatokat 90< címekkel címzett postaládákban tároljuk
- Írjuk meg a programot!





Program 2 szám összeadására: Mnemonic-okat használva

Postaláda	Mnemonic	Utasítás leírása
00	IN	;1. Input szám
01	STO 99	;Adat tárolása
02	IN	;2. Input szám
03	ADD 99	;Két szám összeadása
04	OUT	;Eredmény kiírása
05	COB	;Leállás
99	DAT 00	;Adat



Program 2 szám összeadására

Postaláda	Kód	Utasítás leírása
00	901	;1. Input szám
01	399	;Adat tárolása
02	901	;2. Input szám
03	199	;Két szám összeadása
04	902	;Eredmény kiírása
05	000	;Leállítás
99	000	;Adat



Program vezérlés

- Elágazás utasítások (a szekvenciális utasítás-végrehajtás eltérítése)
 - Cím megváltoztatása az utasítás-számlálóban

- Leállítás

BR (Ugrás)

BRZ (Ágazás 0-ál)

BRP (Ágazás +-nál)

COB (Leállítás)

Tartalom	
Műveleti kód	Operandus
6	xx
7	xx
8	xx
0	(ignore)



Utasításkészlet

Aritmetikai	1xx	Összeadás
	2xx	Kivonás
Adatmozgatás	3xx	Tárolás
	5xx	Betöltés
BR	6xx	Ugrás
BRZ	7xx	Ágazás 0-nál
BRP	8xx	Ágazás +-nál
Input/Output	901	INPUT
	902	OUTPUT
Gép vezérlés (kávészünet)	000	Leállás COB



Számoljuk ki két szám különbségének abszolútértékét (kimenet pozitív!)

00	IN	901	
01	STO 10	310	
02	IN	901	
03	STO 11	311	
04	SUB 10	210	
05	BRP 08	808	;teszt
06	LDA 10	510	;ha negatív, fordított sorrendben kell!
07	SUB 11	211	
08	OUT	902	;eredmény kiírása
09	COB	000	;leállítás
10	DAT 00	000	;adattároló
11	DAT 00	000	;adattároló



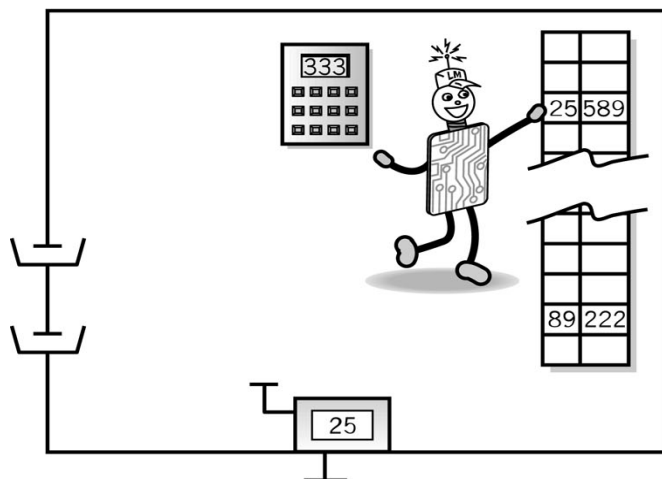
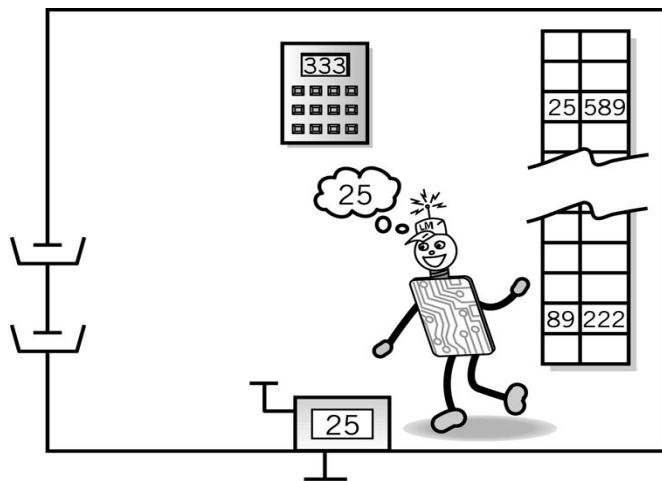
Utasítás-végrehajtási ciklus

Két lépésre osztható:

- *Fetch (~kikeresés)*: Kisember kideríti, hogy milyen utasítást hajtson végre
- *Végrehajtás*: Kisember elvégzi az utasítás által definiált műveletet



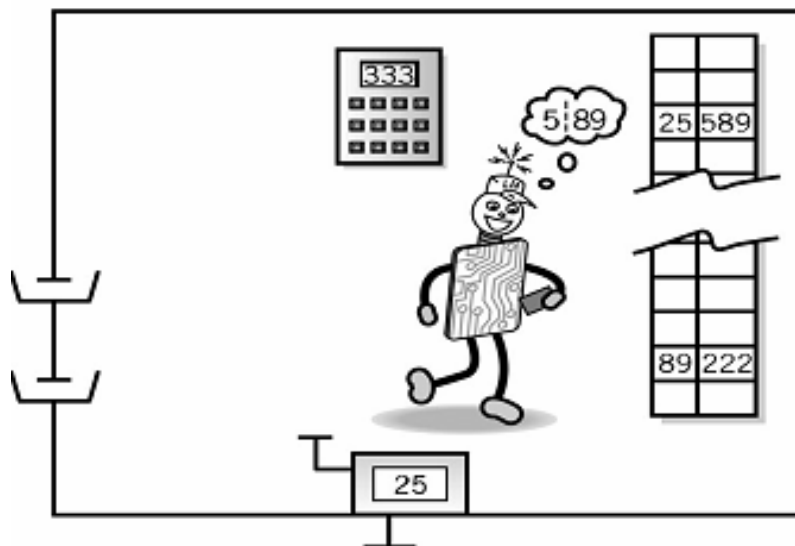
Fetch része az utasítás- végrehajtási ciklusnak



1. Kisember beolvassa a címet az utasítás helyét tároló számlálóból (utasításszámláló)
2. Elmegy a postaládához, ami az utasítás-számlálóban tárolt értékhez tartozik



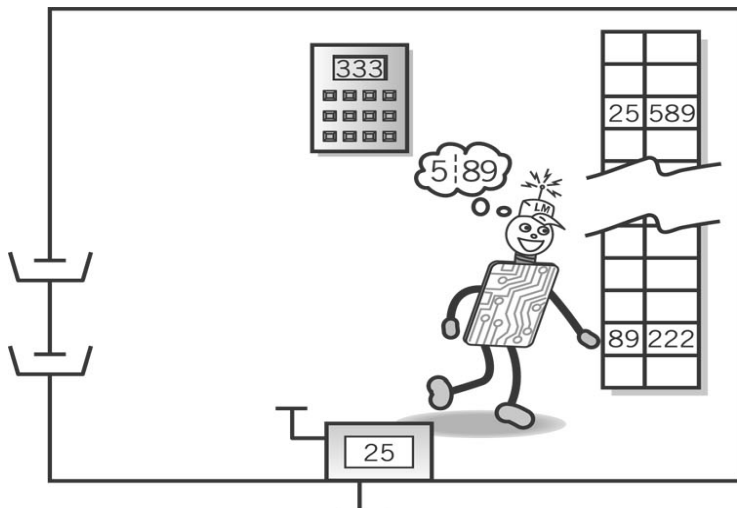
Fetch, (folyt.)



3. A cetliről leolvassa postaládában tárolt a számot (visszarakja a cetlit, hátha szüksége lesz még rá később)

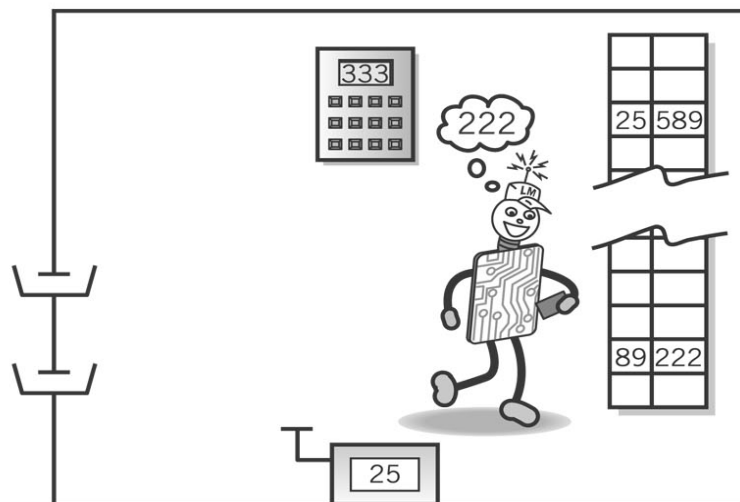


Végrehajtás része az utasítás- végrehajtási ciklusnak



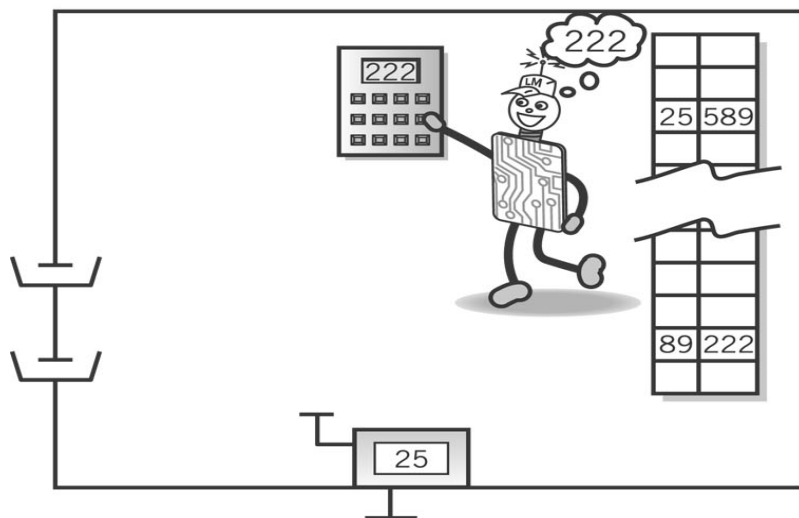
1. Kisember odamegy ahhoz a postaládához, ami az imént fetch-elt utasítás kódjában volt definiálva.

2. Kiolvassa a számot a postaládából (emlékezik a cserére, hátha szüksége lesz rá később).



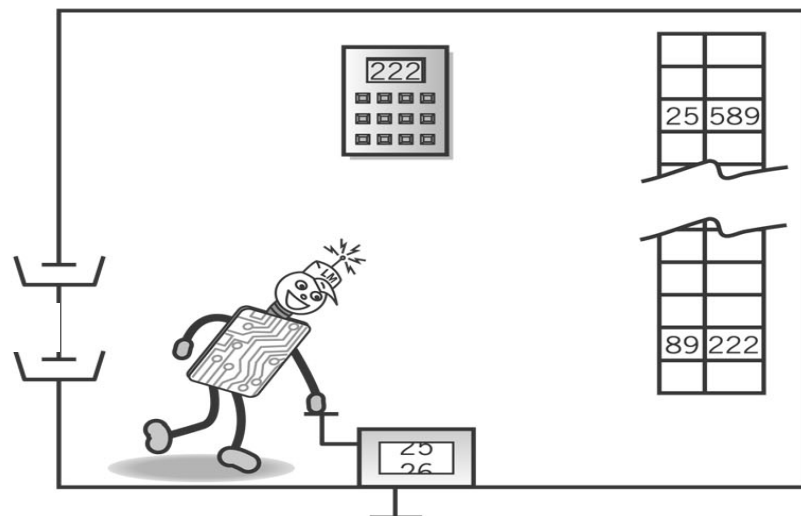


Végrehajtás (folyt.)



3. Odamegy a számológéphez és begépel a számot.

4. Odamegy a az utasítás helyét tároló számlálóhoz (utasításszámlálóhoz) és megnyomja (inkrementálja), ami után készen áll fetch-elni a következő utasítást.





Neumann architektúra (1945)

- Tárolt program elve
 - a programokat és az adatokat azonos memóriában tároljuk
- Memória használata a tartalom figyelembevétel nélkül
- Bináris számábrázolás használata
- Memória lineáris (szekvenciális) címzése



7. Fejezet

A processzor és a memória

**The Architecture of Computer Hardware
and Systems Software:
An Information Technology Approach**

3rd Edition, Irv Englander

John Wiley and Sons ©2003

Wilson Wong, Bentley College

Linda Senne, Bentley College

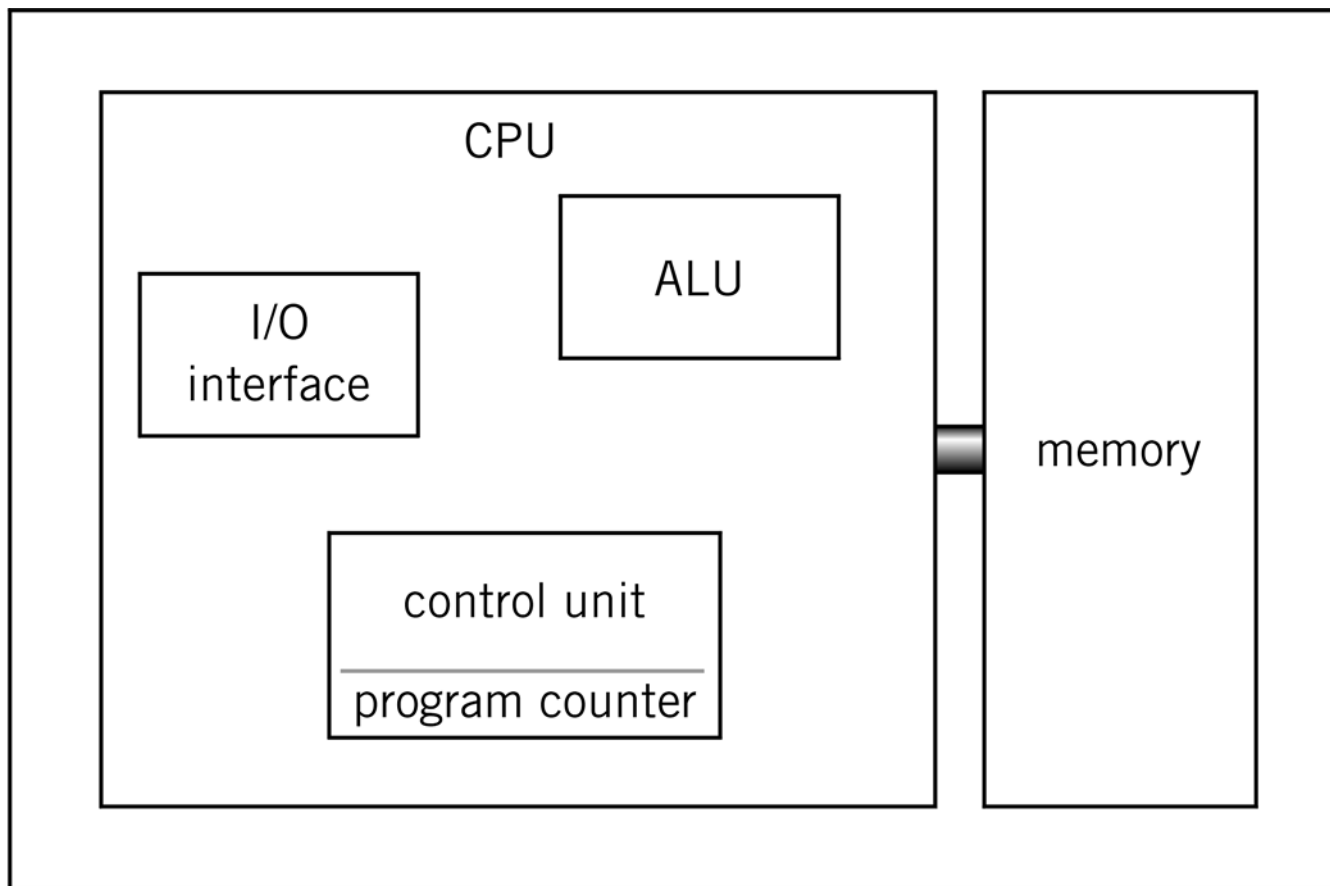


A CPU három fő része

- *ALU (Arithmetic/Logic Unit = Aritmetikai/Logikai Egység)*
 - Számítási (aritmetikai és logikai), ill. összehasonlító műveleteket hajt végre a tárolt adatokon (regiszterek tartalma változik)
- *CU (Control Unit = Vezérlő Egység)*
 - Vezérli az utasítás-végrehajtást (fetch/execute ciklusokat)
 - Funkciók:
 - A CPU regisztereiben az adatokkal végez műveleteket (nem változnak meg az adatok)
 - Beolvassa a program utasításait és parancsokat ad az ALU-nak
 - Részei:
 - *Memory Management Unit (Memória Vezérlő Egység):*
 - Felügyeli a „fetch”-elését, az utasításoknak és adatoknak
 - *I/O Interfész*
 - néha egybeépítik a memória vezérlővel, melyet *Bus Interface Unit*-nak nevezünk
- *Regiszterek*
 - Példa: *Program counter (PC – program számláló)* vagy *instruction pointer* (utasítás mutató) meghatározza a következő utasítást a futtatáshoz

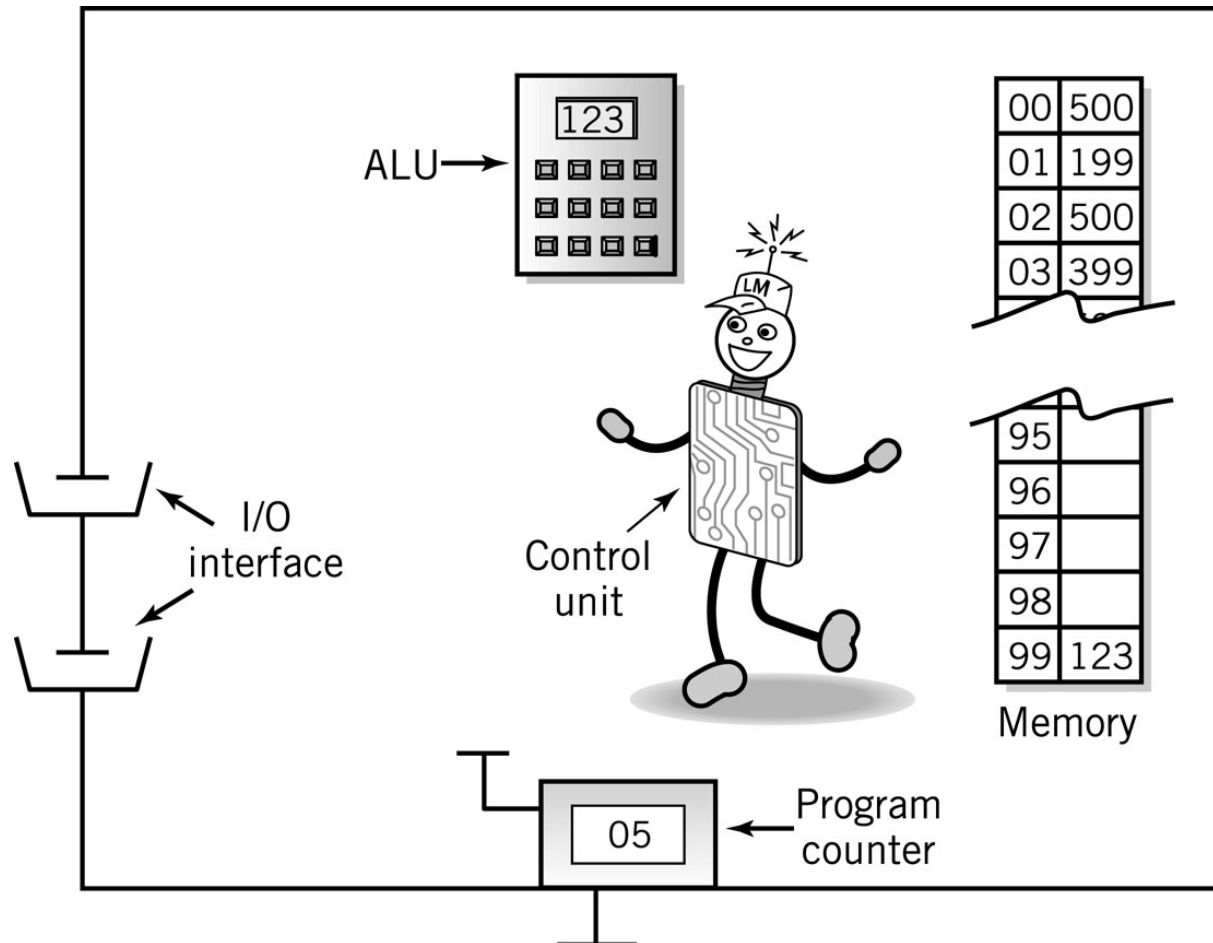


Számítógép sematikus felépítése





Little Man Computer (LMC)





Regiszterek



A regiszterek tulajdonságai

- Kicsi, *permanens (nem múlékony)* tárolóhelyek a CPU-n belül, amelyeket a számolások részeredményeinek tárolására használ
- A Vezérlő Egység (CU) vezérli őket
- *Speciális használatra* vannak tervezve
 - *általában mindegyik adott feladatot lát el*
- Tároló méretük néhány bit, ill. byte
- Tárolhat adatot, címet és utasítást
- Hány regisztere van az LMC-nek?



Regiszterek

- Regiszterek használata
 - „Scratchpad” („vázlatfüzet”) az aktuálisan futó program számára
 - ▣ A gyakran használt, ill. gyorsan elérendő adatokat tárolja
 - A CPU és a futó programra vonatkozó információt tárolja
 - ▣ A következő program utasítás címe
 - ▣ Külső eszközökből érkező jelek
 - általános, ill. speciális célú regisztereket különböztetünk meg
- **Általános célú regiszterek**
 - *Felhasználó számára láthatóak*
 - Megszakítási és adat értékeket tartalmaz
 - Megegyezik az LMC számológépével
 - Minden processzorban van néhány tucat



Speciális célú regiszterek

- *Program Count Register =
Program számláló regiszter (PC)*
 - Instruction pointernek (utasítás mutatónak) is hívjuk
- *Instruction Register =
Utasítás regiszter (IR)*
 - Memóriából származó utasításokat tartalmaz
- *Memory Address Register =
Memória Cím Regiszter (MAR)*
- *Memory Data Register = Memória Adat Regiszter (MDR)*
- *Status Registers = Állapot Regiszterek*
 - A processzor és a futó program állapotát tartalmazza
 - *Flag*-eket (*jelzőbit*) (egy bites logikai változókat) tartalmaz,
 - ▣ nyomon követhessük a számítógép működését
 - ▣ túlcsordulás, ill. carry, elektromos kimaradás, egyéb belső hibák.



Regiszter műveletek

- Más helyekről származó értékek tárolása (regiszterek és memóriák)
- Összeadás és kivonás
- Bit műveletek (Shift, Rotate – Eltolás, Forgatás)
- A regiszter tartalmának ellenőrzése, pl.: nulla vagy pozitív



Memória

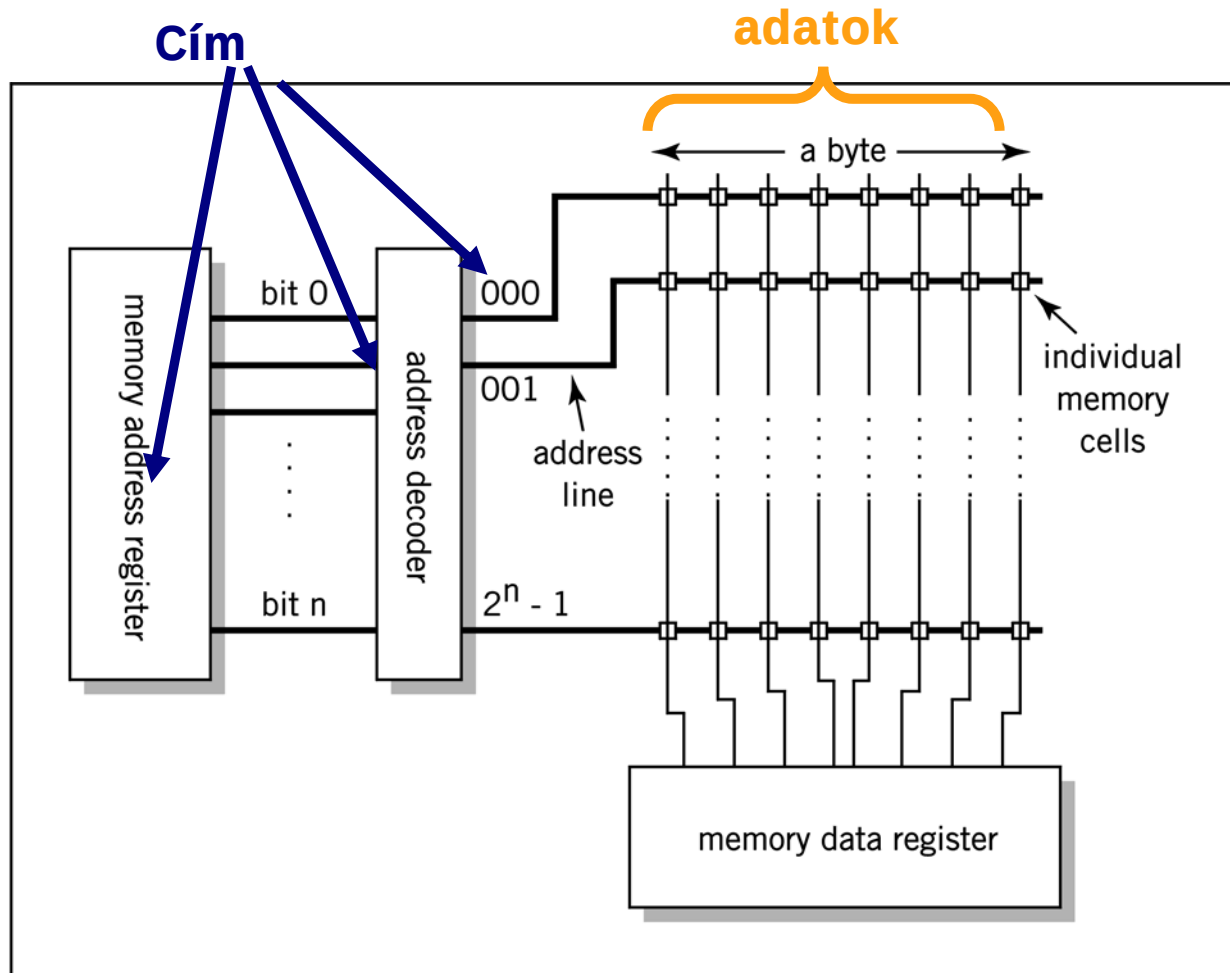


A memória működése

- Minden memóriatartomány egyedi címmel rendelkezik
- Az utasítás címe a Memória Cím Regiszterbe kerül, amely megtalálja a helyét a memóriában
- A processzor határozza meg, hogy tároljon vagy olvasson adatot
- A Memória Adat Regiszter és a memória közötti szállítási helyet igényel
- A Memória Adat Regiszter kétirányú regiszter



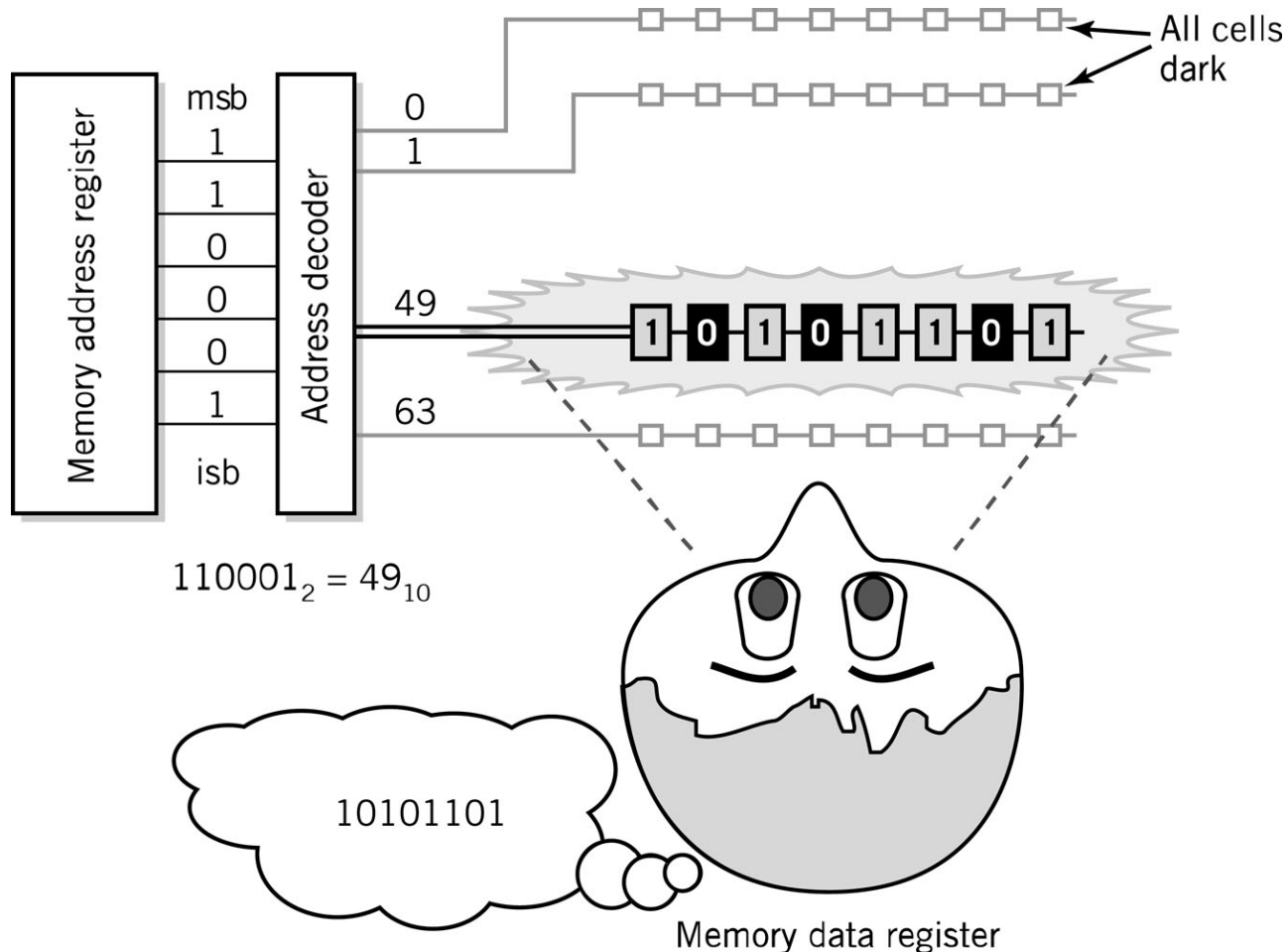
Kapcsolatok a MAR, a MDR és a memória között





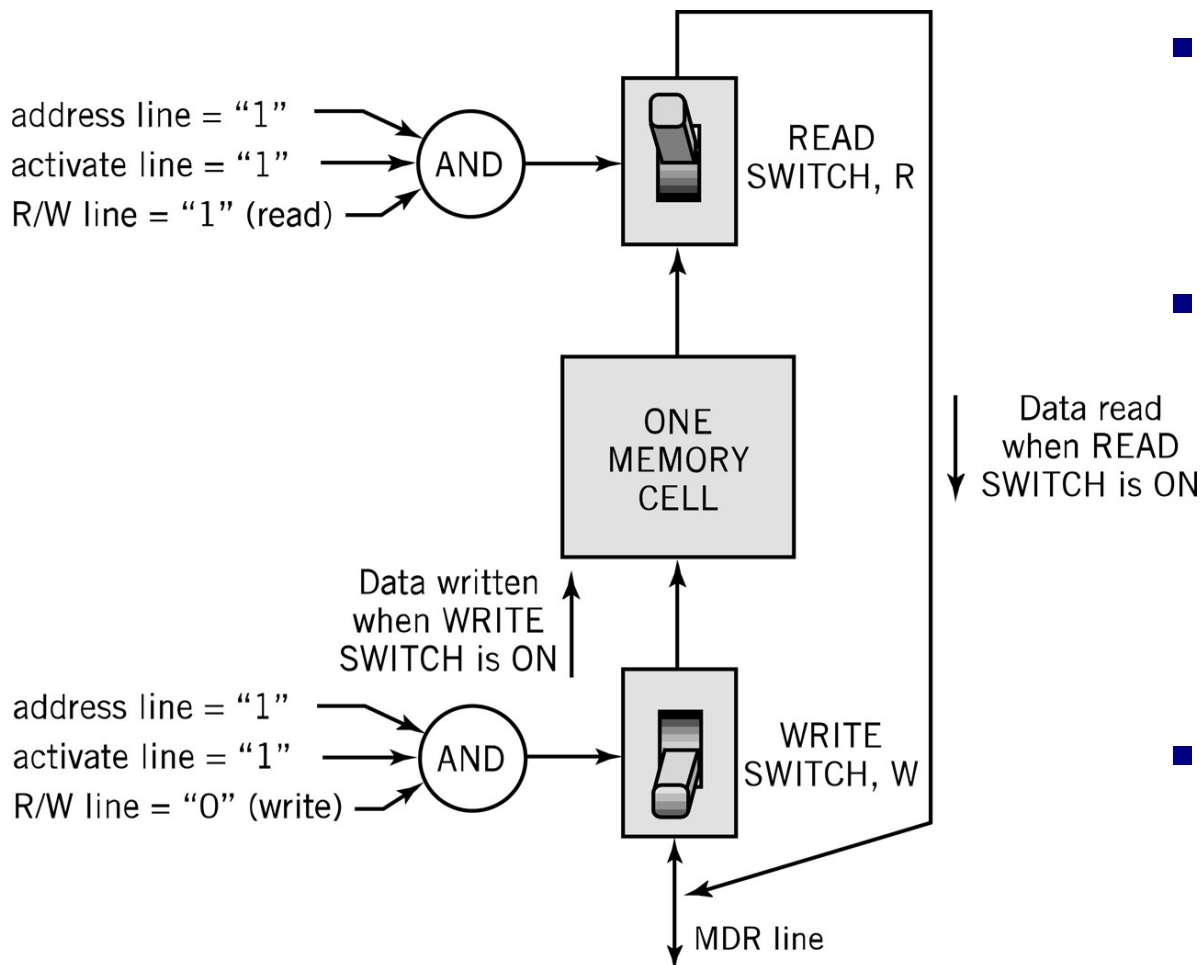


A memória működésének vizuális leírása





Egy memória cella működése



- Két kapcsoló:
 - Olvasás (Read)
 - Írás (write)
- Két vezérlő áramkör (AND), három vezérlő jellel:
 - address line
 - activate line
 - R/W line
- Információt tároló memória cella



Memória-kapacitás

- Két tényező határozza meg
 1. A MAR bit-jeinek száma
 - ▣ LMC = 100 (00 - 99)
 - ▣ 2^K , ahol K = a regiszter szélessége bit-ekben
 2. Az utasításban szereplő cím rész (operandus) mérete
 - ▣ 4 bit 16 helyet tudunk megkülönböztetni
 - ▣ 8 bit enged 256 helyet tudunk megkülönböztetni
 - ▣ 32 bit 4,294,967,296 vagy 4 Giga helyet tudunk megkülönböztetni
- A memória mérete lényegesen befolyásolja a számítógép teljesítményét
 - A kevés memória arra kényszerítheti a processzort, hogy 50%-alatt dolgozzon



RAM: Random Access Memory (Véletlen elérésű memória)

- *RAM*
 - eredetileg mágnesezhető tároló cellák, ma DRAM és SRAM
- *DRAM (Dynamic RAM=Dinamikus memória)*
 - Ma a legáltalánosabban használt, olcsó
 - Volatile-s tároló: feszültség kikapcsolásával elvesz a tartalma
 - Tartalmát hamar elveszti : másodpercenként kb. 1000-szer kell újraírni a tartalmát az értékek tárolásához
- *SRAM (Statikus RAM)*
 - Gyorsabb és drágább, mint a DRAM (más gyártási technológiája)
 - Volatile-s tároló: feszültség kikapcsolásával elvesz a tartalma
 - Keveset használnak belőle a **cache** memóriában, a gyors eléréshez



ROM - Read Only Memory (Csak olvasható memória)

- Stabil, nem volatile-s tároló olyan programok tárolásához, amelyek nem változnak
- Mágneses magmemória
- EEPROM
 - Electrically Erasable Programmable ROM (Elektromosan törölhető és programozható ROM)
 - Lassabb és kevésbé rugalmasan használható, mint a FLASH ROM
- *Flash ROM*
 - Gyorsabb, mint a merevlemezek, de sokkal drágább is
 - Felhasználási területek:
 - ▣ BIOS: BOOT program, diagnosztikai programok
 - ▣ Digitális kamerák



CPU utasítások végrehajtása



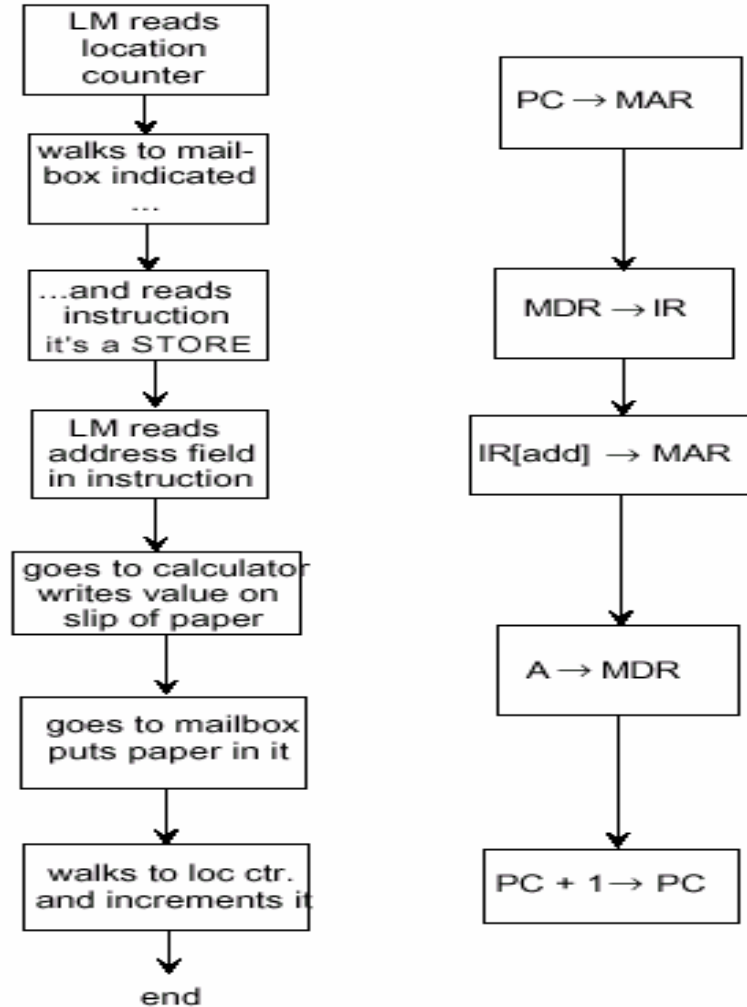
Fetch-Execute ciklus

- A CPU műveletek végrehajtása két ciklusra osztjuk (két memória hozzáférésre)
 - mind a utasítás kódja, mind az az adat, amelyen a utasítást végrehajtjuk a memóriában van tárolva
- *Fetch*
 - Megkeressük, ill. dekódoljuk az utasítást, a memóriából a regiszterekbe töltjük és jelzést küldünk az ALU-nak
- *Execute*
 - Végrehajtjuk azt a műveletet, amit az utasítás kódol
 - Adatokat mozgatjuk, ill. módosítjuk



LMC vs. CPU

Fetch és Execute ciklus





A LOAD utasítás Fetch/Execute ciklusa

- | | |
|----------------------------------|---|
| 1. PC $\rightarrow$ MAR | Az adatokat a PC-ből a MAR-ba mozgatja |
| 2. MDR $\rightarrow$ IR | Az adatokat az IR-be mozgatja |
| ----- | |
| 3. IR(address) $\rightarrow$ MAR | Az utasítás cím része (operandus) betöltődik a MAR-ba |
| 4. MDR $\rightarrow$ A | Az aktuális adatokat a tároló regiszterbe mozgatja |
| 5. PC + 1 $\rightarrow$ PC | A programszámláló növelése |



A STORE utasítás Fetch/Execute ciklusa

- | | |
|----------------------------------|--|
| 1. PC $\rightarrow$ MAR | Az adatokat a PC-ből a MAR-ba mozgatja |
| 2. MDR $\rightarrow$ IR | Az adatokat az IR-be mozgatja |
| ----- | |
| 3. IR(address) $\rightarrow$ MAR | Az utasítás cím részletei betöltődnek a MAR-ba |
| 4. A $\rightarrow$ MDR* | A tároló regiszterből az MDR-be mozgatja az adatokat |
| 5. PC + 1 $\rightarrow$ PC | A programszámláló növelése |

* Vegyük észre, hogy a 4. pont hogyan változott meg a LOAD utasításhoz képest! Mikor történik a memória használata a két esetben?



Az ADD utasítás Fetch/Execute Ciklusa

1. PC $\rightarrow$ MAR

Az adatokat a PC-ből a MAR-ba mozgatja

2. MDR $\rightarrow$ IR

Az adatokat az IR-be mozgatja

3. IR(address) $\rightarrow$ MAR

Az utasítások cím részletei betöltődnek a MAR-ba

4. $A + \text{MDR} \rightarrow A$

Az MDR tartalma a tár tartalmához adódik

5. $\text{PC} + 1 \rightarrow \text{PC}$

A programszámláló növelése



LMC utasításainak Fetch/Execute ciklusai

<u>Kivonás</u>	<u>Bemenet</u>	<u>Kimenet</u>	<u>Szünet</u>
PC $\rightarrow$ MAR	PC $\rightarrow$ MAR	PC $\rightarrow$ MAR	PC $\rightarrow$ MAR
MDR $\rightarrow$ IR	MDR $\rightarrow$ IR	MDR $\rightarrow$ IR	MDR $\rightarrow$ IR
-----	-----	-----	-----
IR[addr] $\rightarrow$ MAR	IOR $\rightarrow$ A	A $\rightarrow$ IOR	
A - MDR $\rightarrow$ A	PC + 1 $\rightarrow$ PC	PC + 1 $\rightarrow$ PC	
PC + 1 $\rightarrow$ PC			

Ugró (elágazás) utasítás

PC $\rightarrow$ MAR

MDR $\rightarrow$ IR

IR[addr] $\rightarrow$ PC

Feltételes (elágazás) ugró utasítás

PC $\rightarrow$ MAR

MDR $\rightarrow$ IR

Ha a feltétel hamis: PC + 1 $\rightarrow$ PC

Ha a feltétel igaz: IR[addr] $\rightarrow$ PC



Bus-ok



Bus

- Fizikai kapcsolat, ami lehetővé teszi az adatátvitelt a számítógépben két pont között
- Elektromos csatlakozások csoportja a két csomópont közötti jelátvitelhez
 - *Line (vonal)*: csatlakozási pont a buszon
- A jeleknek négy fajtáját különböztetjük meg
 1. Adat (alfa-numerikus, numerikus, utasítások)
 2. Cím
 3. Vezérlő jelek
 4. Táp (néha)



Bus-ok a számítógépben

- Processzor és memória közötti bus
- Az I/O perifériák lehetnek azonos bus-on a CPU-val és a memóriával vagy külön buszon
- Ha a (plug-in) I/O elemek a CPU és a memória közötti bus-t használják, akkor a fizikai egységet *backplane-nek* hívják
 - *Rendszer bus-nak és külső bus-nak is hívják*
 - Ez jó példa a „*broadcast*” bus-ra
 - A PC-kben általában ezeket az elemeket egy nyomtatott áramköri lapra az *alaplapra (motherboard)* integrálják
 - ▣ ez tartalmazza a CPU-t és összekapcsolja azt a többi komponenssel



Bus jellemzői

- Protokoll
 - A kommunikáció jól dokumentált leírása
 - Világosan megmagyarázza az összes vonal és a vonalakon lévő minden jel jelentését
- Adatátviteli kapacitás:
 - az adat átvitel mértékegysége bit/másodperc
- Adat-bus szélessége:
 - Az egyszerre átvihető adatok szélessége bitekben



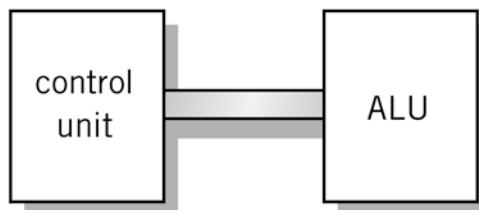
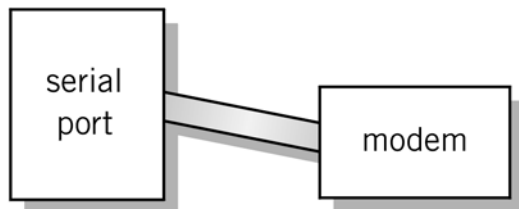
Bus topológiák és üzenetváltás

- Pont-pont kapcsolat
- Többpontos kapcsolat
 - „Broadcast bus”: minden elem megkapja a bus-ra kitett adatokat

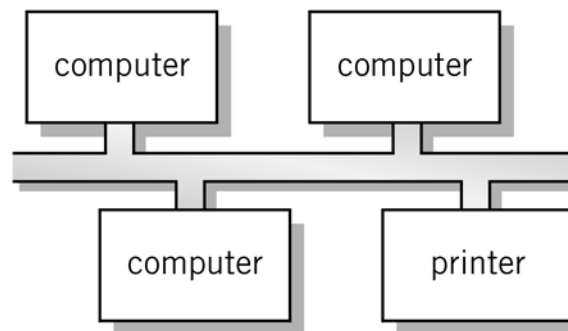


Pont-pont kapcsolat vs. több pontos

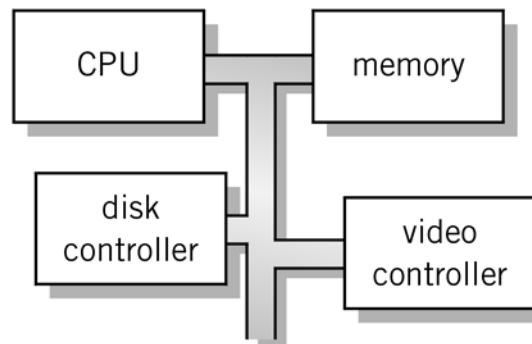
Plug-in eszköz



examples of point-to-point buses



**Broadcast bus
példa:
Ethernet**



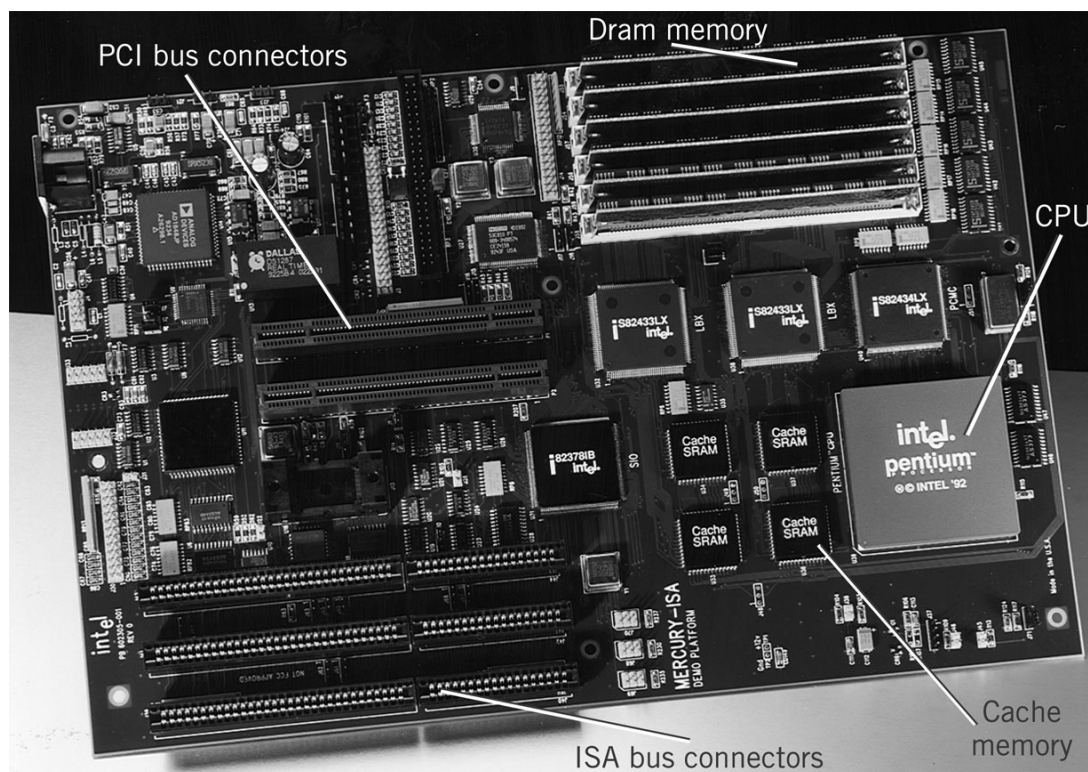
examples of multipoint buses

**Az összetett
eszközök
mindegyike között
van kapcsolat**



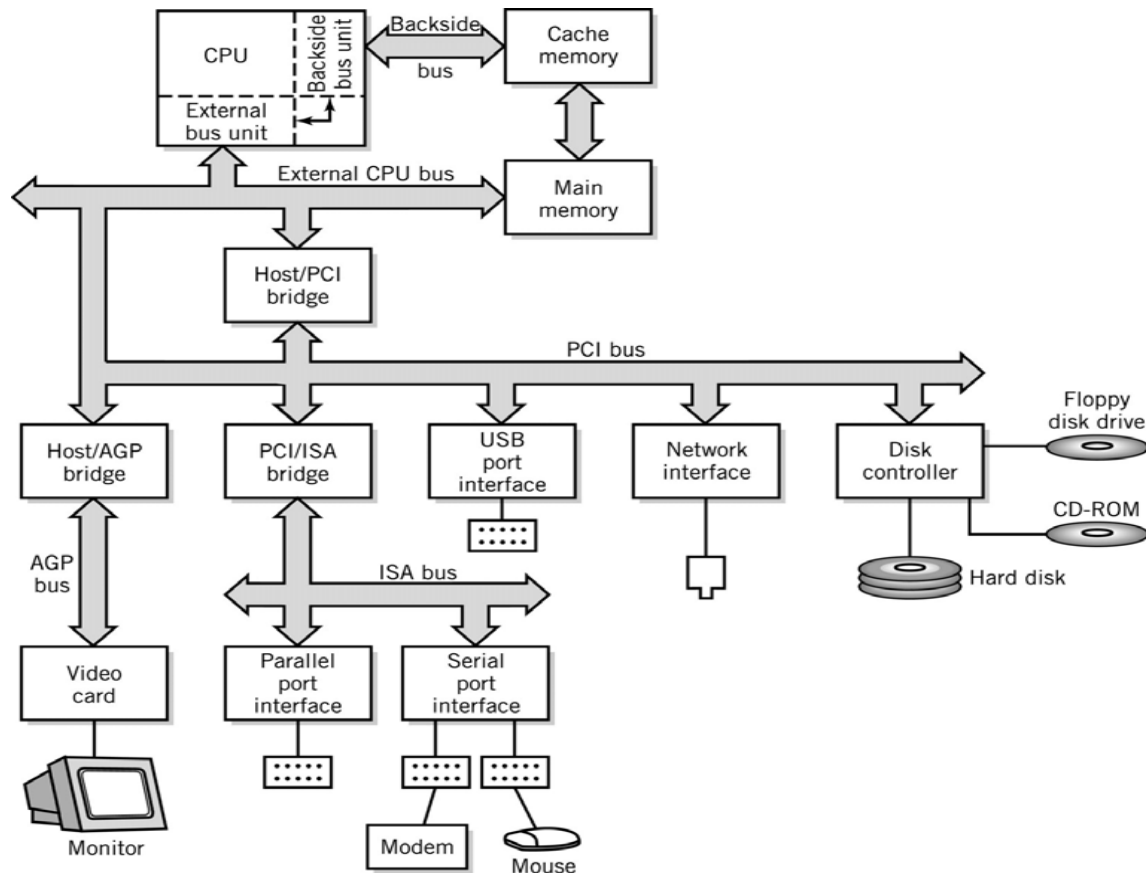
Az alaplap

- Nyomtatott áramkör, melyen a processzor és egyéb fő elemek találhatók





Összeköttetések egy tipikus PC-ben

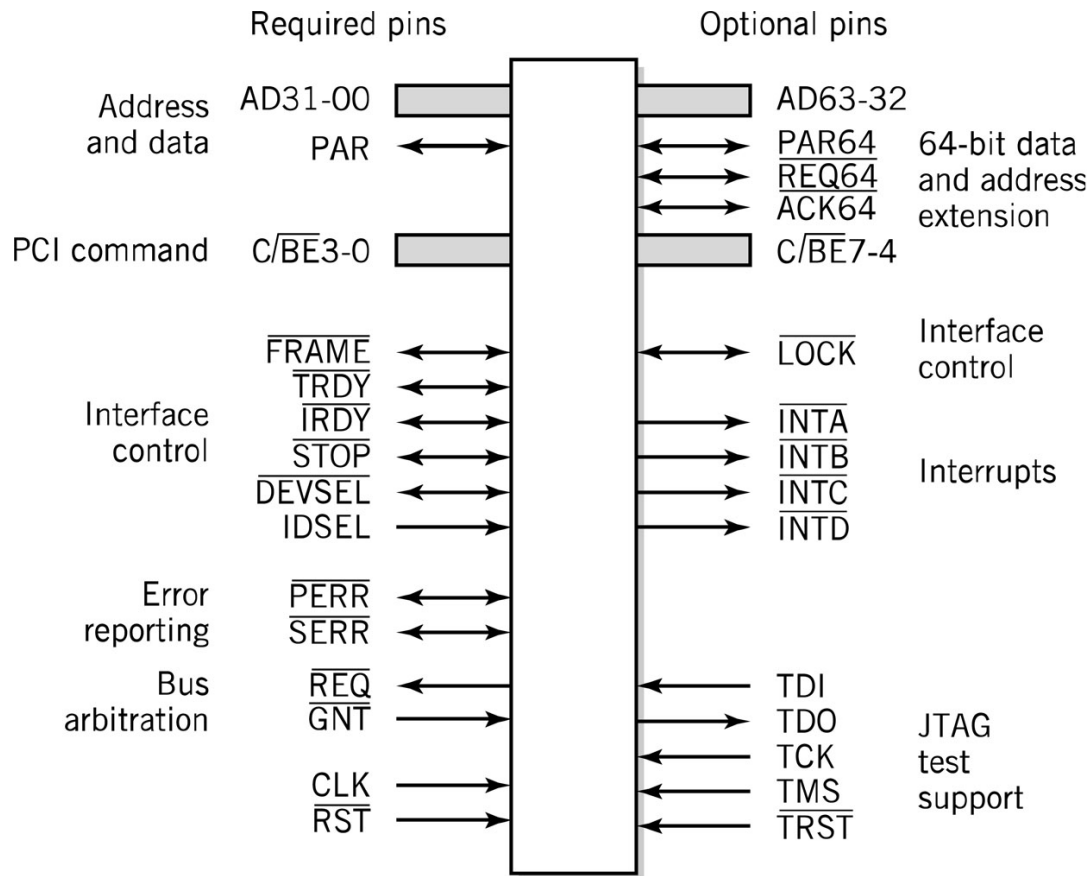


Bus interface bridge-ek:

különböző típusú bus-okat kapcsolnak össze, átviszik az adatokat az egyik bus-ról a másikra



PCI bus-on használt jelek



Source: Copyright © PCI Pin List/PCI Special Interest Group, 1999.



A CPU utasítás készlete



Utasítások

- Utasítás
 - A számítógépet vezérlő parancsok
 - Végrehajtásukkor elektromos jelek hatására a számítógép egyes áramkörei különböző műveleteket hajtanak végre
- Utasítás készlet
 - A tervező által definiált funkciók, amelyet a processzor végre tud hajtani
 - A különböző architektúrájú számítógépek között különbséget tehetünk a következő szempontok alapján:
 - ▣ Végrehajtható utasítások száma
 - ▣ Az egy utasítással végrehajtható műveletek bonyolultsága szerint
 - ▣ Támogatott (feldolgozható) adattípusok
 - ▣ Utasítások formátuma (kötött vagy változó hosszúság)
 - ▣ Regiszterek használata
 - ▣ Címzés (cím mérete, címzési módok)



Utasítás részei

- Műveleti kód (OP code):
 - a feladat, amit a számítógépnek végre kell hajtani
- Forrás operandus
- Eredmény operandus
 - Adat helyének meghatározása (regiszter, memória)
 - ▣ Explicit: tartalmazza az utasítás
 - ▣ Implicit: alapértelmezés szerinti helyen

Címek

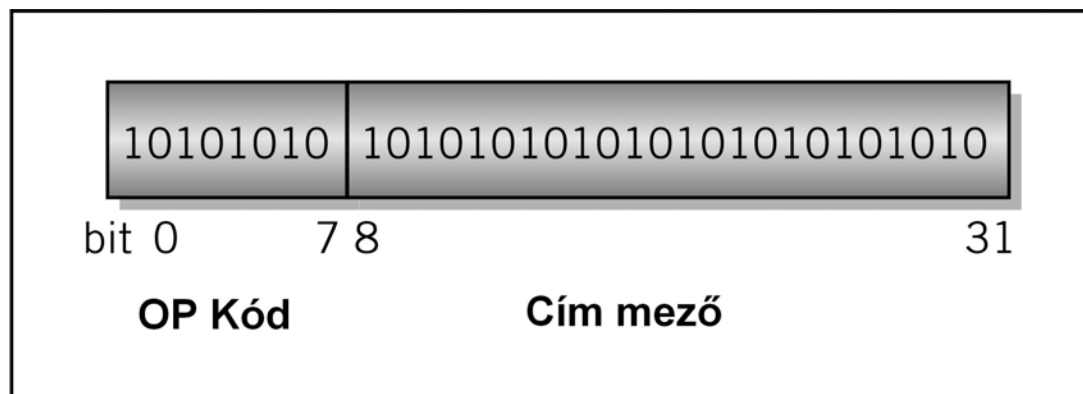
OP code	Forrás operandus	Eredmény operandus
---------	------------------	--------------------



Utasítások formátuma

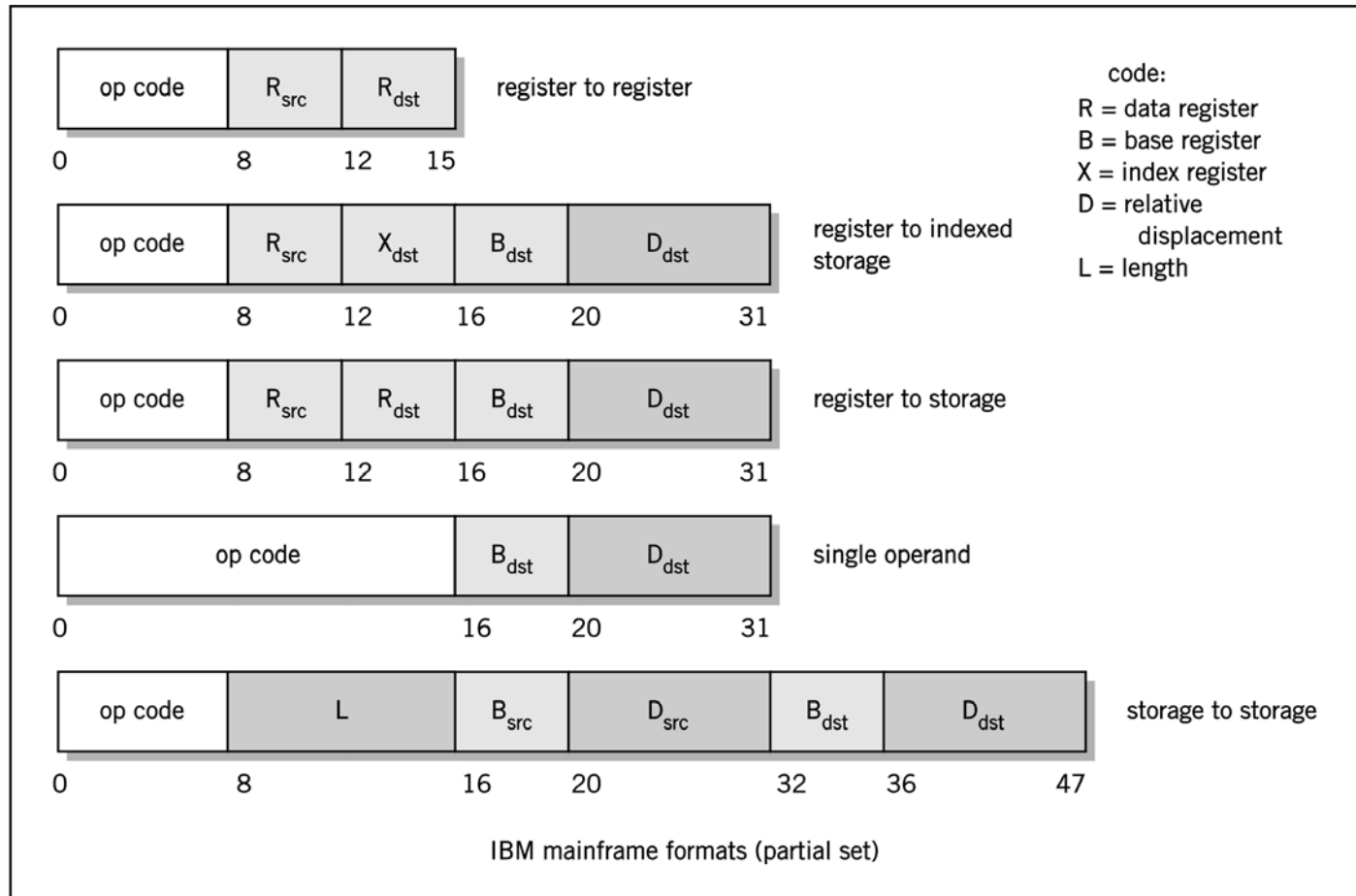
- *Számítógép-specifikus* leírás, amely meghatározza:
 - műveleti kód hosszát
 - Operandusok számát
 - Operandusok hosszát

Egy egyszerű
32-bites
utasítás
formátum



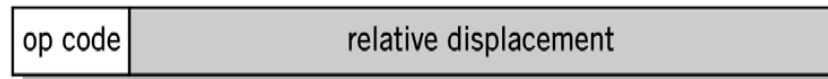


Utasítás formátumok: CISC



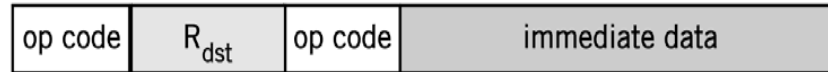


Utasítás formátumok: RISC



CALL instruction

31 29 0



LOAD high 22 bits immediate

31 29 25 22 0



BRANCH

31 29 28 25 22 0



INTEGER instructions
(also, with 1 in bit 14, and
bits 0–13 immediate address)

31 29 25 19 14 13 5 0



FLOATING POINT instructions

31 29 25 19 14 5 0

SPARC RISC formats (complete set)



Utasítások típusai

- Adat átviteli műveletek (load: betöltés, store: tárolás)
 - Általánosan használt, változatos megvalósítások
 - A memória és a regiszterek közötti kapcsolat
 - Mekkora a *word* adattípus? 16? 32? 64 bit?
- Aritmetikai utasítások
 - operátorok: $+$ $-$ $/$ $*$ $^$
 - Egész és lebegőpontos adatokon is végezhető
- Boolean logikai műveletek és relációs operátorok
 - Relációs operátorok: $>$ $<$ $=$
 - Logikai operátorok: **AND**, **OR**, **XOR**, **NOR**, és **NOT**
- Egyetlen operandust használó utasítások
 - Negáció, dekrementálás, inkrementálás

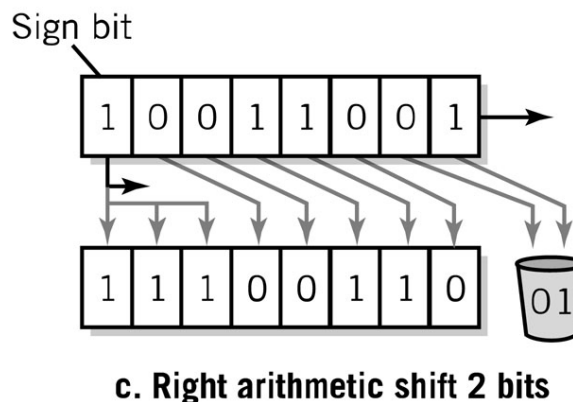
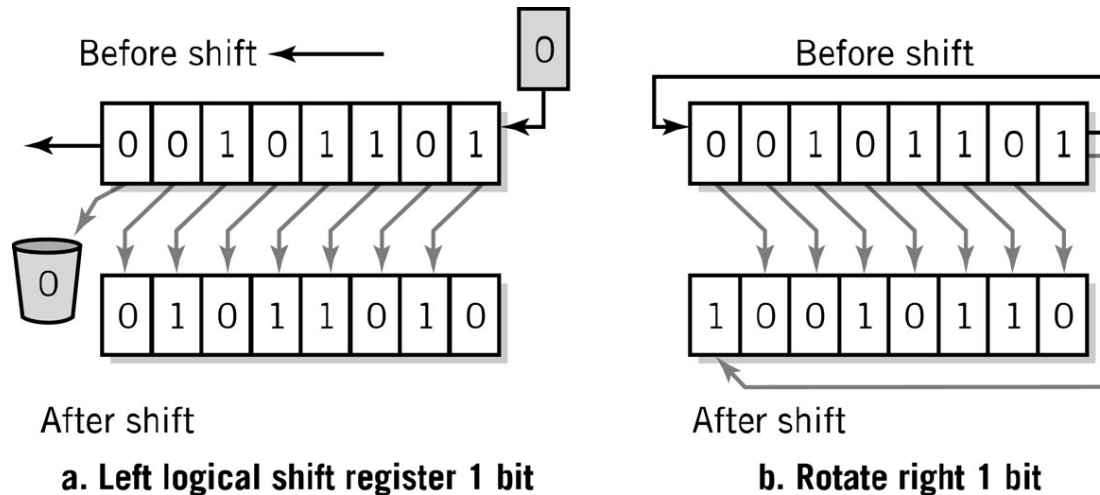


Egyéb utasítás típusok

- Bit-manipuláló utasítások
 - Flag-ek (jelzőbitek) a feltételek tesztelésére
- Bit-enkénti eltolás, ill. forgatás
- Program vezérlő utasítások
- Verem tár kezelő utasítások
- Összetett adatelemeken végzett utasítások
- I/O elemeket és számítógépet vezérlő utasítások



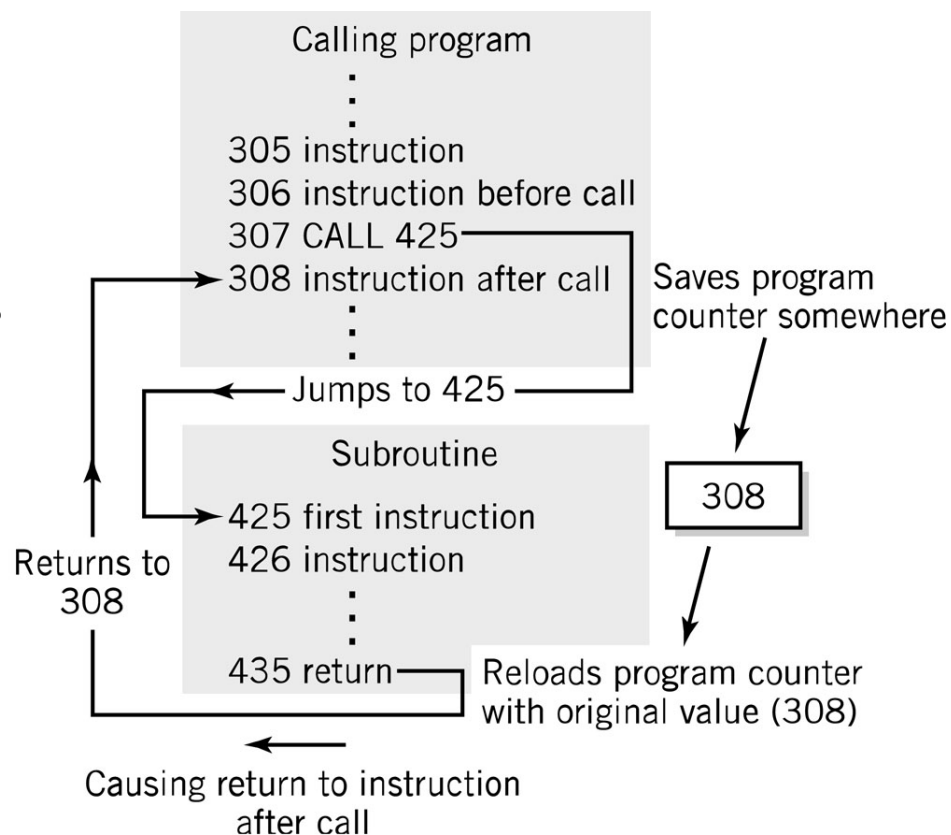
Regiszterben tárolt értékek eltolása és forgatása





Program vezérlő utasítások

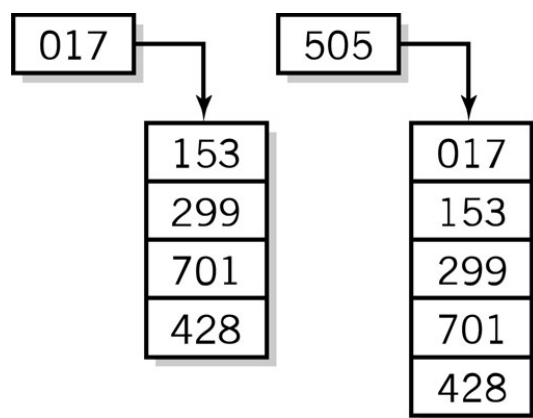
- Program vezérlése
 - Ugrás és elágazás utasítások
 - Szubrutin hívása és visszatérés
 - ▣ CALL (CÍM)
 - ▣ RETURN





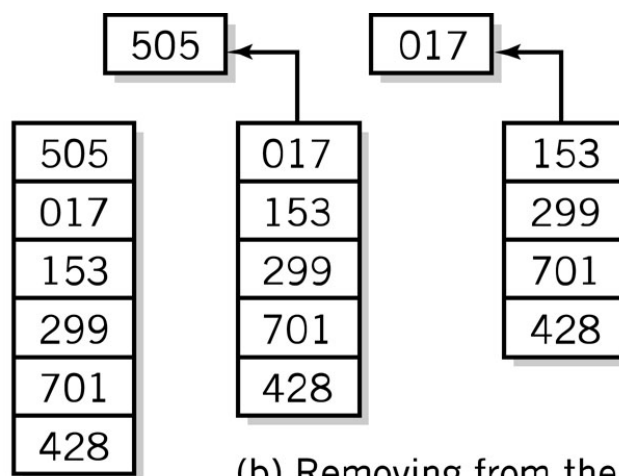
Veremtár kezelő utasítások

- Veremtár kezelő utasítások
 - LIFO (utolsó be, első ki) működés, információ tárolása
 - Elemek kivétele fordított sorrendben történik, mint az elemek behelyezése



(a) Adding to the stack

Beszúrás

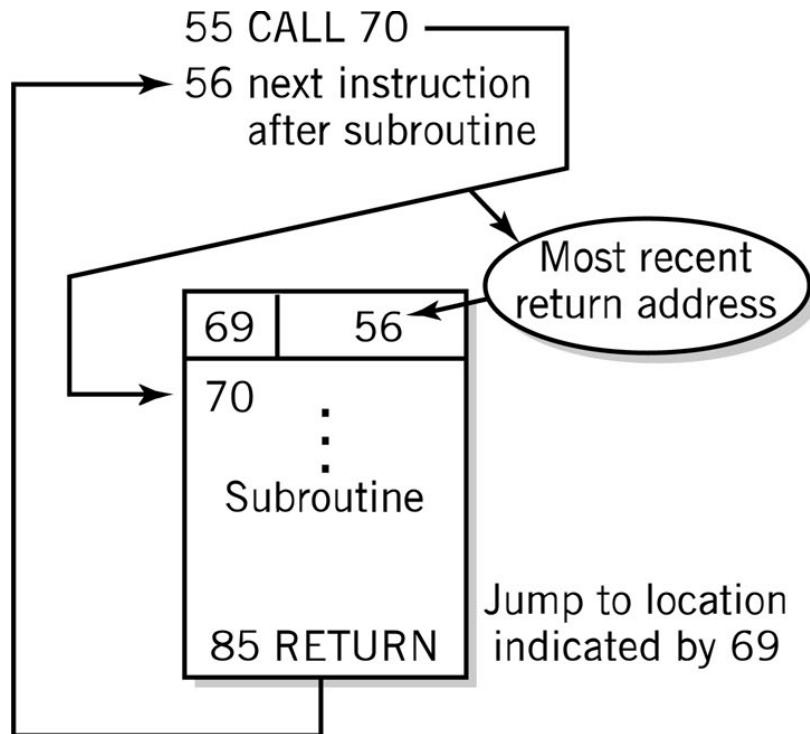


(b) Removing from the stack

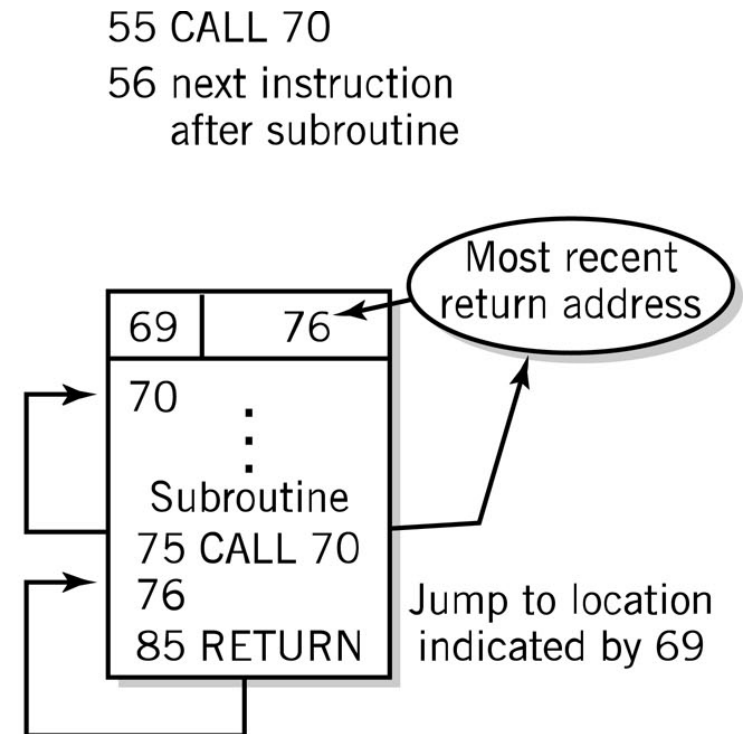
Kivétel



Szubrutin visszatérési címének tárolása állandó helyen : *Oops!*



a. Subroutine called from loc.55

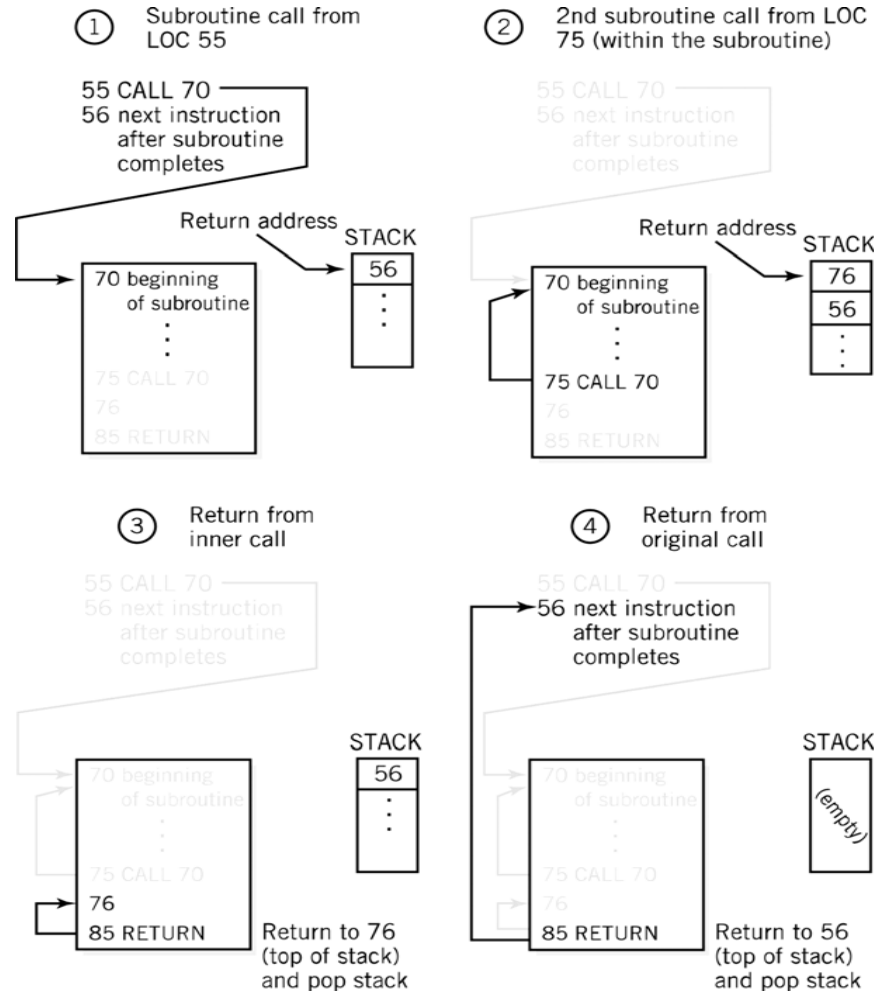


b. Subroutine re-called from 75, within the subroutine

Hibás működés lehetséges!!



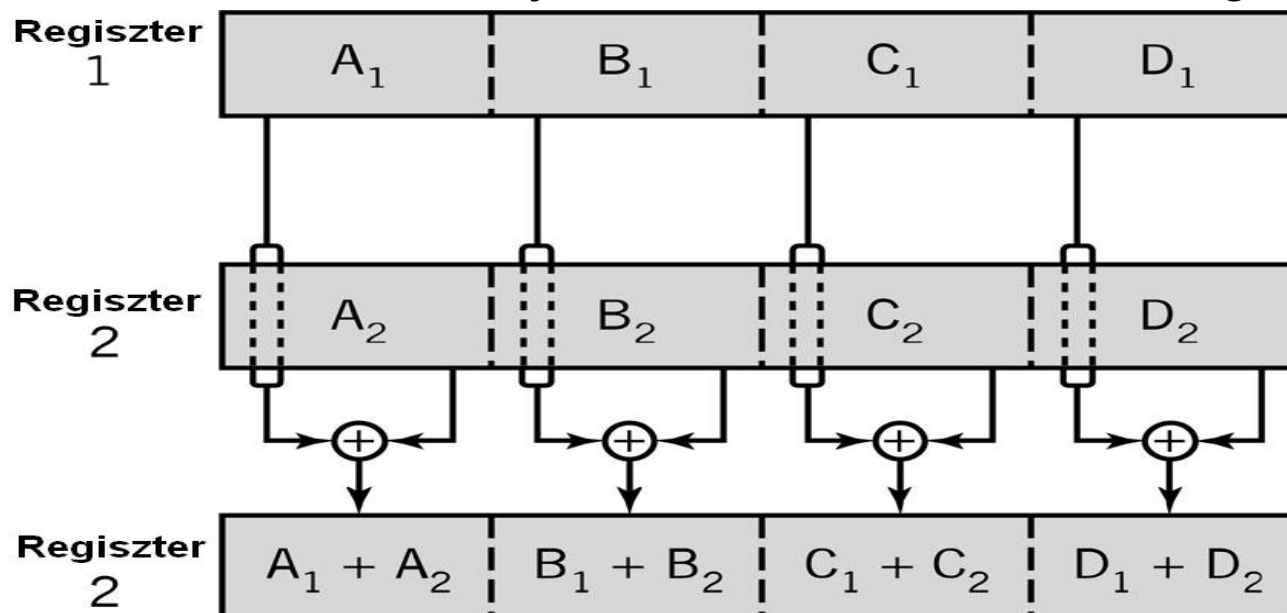
Veremtár a szubrutin visszatérési címének tárolására





Összetett adatkezelő utasítások

- Párhuzamosan hajt végre egyszerű műveleteket adatok csoportján
 - SIMD: Single Instruction, Multiple Data
 - ▣ egy utasítás, több adat
 - Intel MMX™: 57 multimédiás utasítás
 - Általánosan használják vektorok és tömbök feldolgozásakor





8. Fejezet

Processzor (CPU) és memória:
tervezés, implementáció, modern megoldások

The Architecture of Computer Hardware and Systems Software: An Information Technology Approach

3rd Edition, Irv Englander

John Wiley and Sons ©2003

**Wilson Wong, Bentley College
Linda Senne, Bentley College**



CPU architektúrák

- CISC – Complex Instruction Set Computer
- RISC – Reduced Instruction Set Computer
- CISC és RISC összehasonlítása
- VLIW – Very Long Instruction Word
- EPIC – Explicitly Parallel Instruction Computer



CISC architektúra

- Példák
 - Intel x86, IBM Z-széria, idősebb CPU architektúrák
- Tulajdonságai
 - Néhány általános célú regiszter
 - Több különböző címezési mód
 - Nagyszámú speciális, összetett utasítások
 - Változó méretű utasítások



CISC architektúra korlátai

- Az összetett utasításokat ritkán használják a programozók és fordítók
- Memóriát használó utasítások (load és store) végrehajtása lassú pedig ezek teszik ki utasítások jelentős részét
- Eljárás, ill. függvény hívások problémát okoznak
 - Hívási paraméterek átadása
 - Regiszterek tartalmát el kell menteni és vissza kell állítani a hívások után

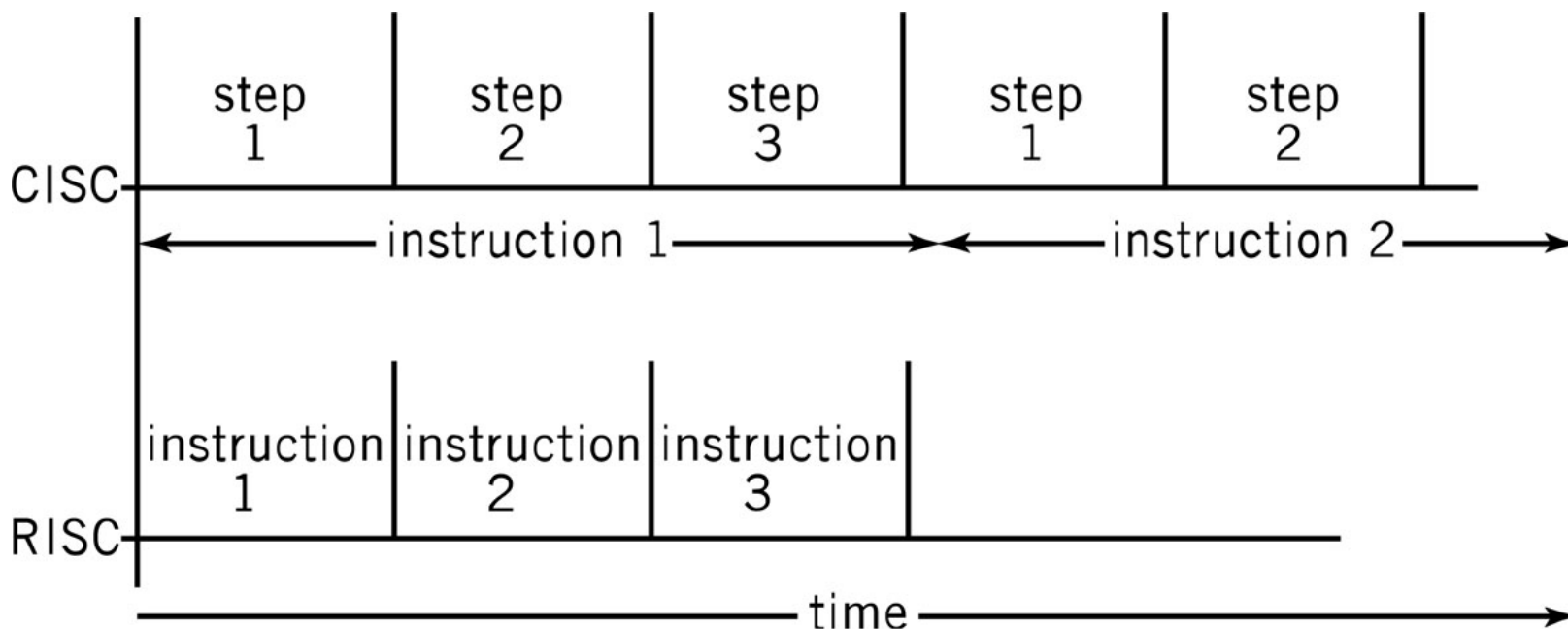


A RISC tulajdonságai

- Példák
 - Power PC, Sun Sparc, Motorola 68000
- Korlátozott és egyszerű utasításkészlet
- Fix hosszúságú, meghatározott formátumú utasítás szavak
 - Pipeline feldolgozás lehetősége, párhuzamos fetch és execution (azonos feldolgozási idő)
- Korlátozott számú címezési mód
 - Egyszerűbbé teszi a megvalósító hardvert (CPU)
- Regiszter-orientált utasításkészlet
 - Csökkenti a memória hozzáférések számát
- Nagyszámú regiszter
 - Csökkenti a memória hozzáférések számát
 - Hatékony a szubrutin (alprogram) hívások végrehajtása

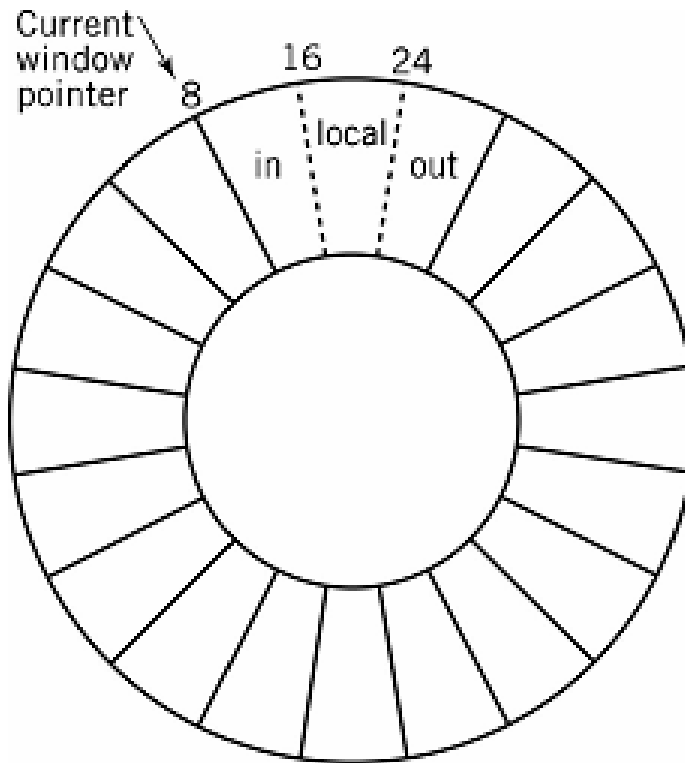


CISC vs. RISC feldolgozás

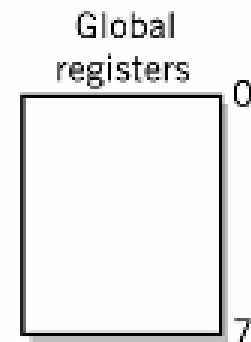




Körkörös regiszter puffer



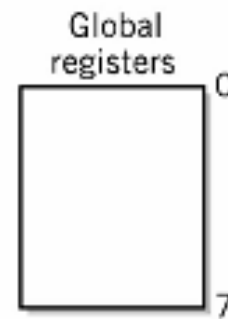
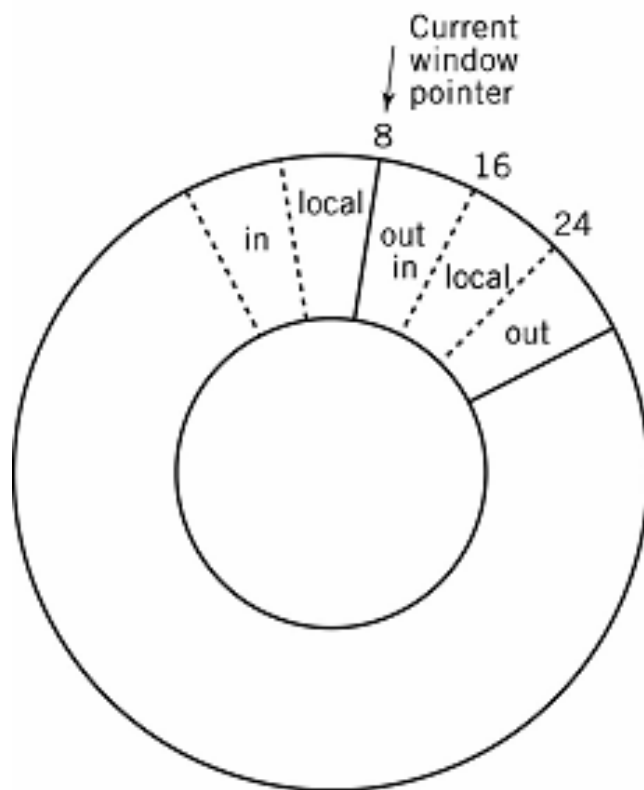
a. Current window





Körkörös regiszter puffer

- Szubrutin hívás után



b. After window shift



CISC és RISC teljesítményének összehasonlítása

- RISC → Egyszerűbb utasítások
→ több utasítás
→ több memória hozzáférés
- RISC → nagyobb a bus forgalom és nagyobb a cache „hibák” valószínűsége
- Több regiszter növelhetné a CISC teljesítményét de nincs számukra hely
- Modern CISC és RISC architektúrák egyre hasonlóbbak lesznek



Modern megoldások az utasítás végrehajtás meggyorsítására

- VLIW – Very Long Instruction Word
- EPIC – Explicitly Parallel Instruction Computer
- A gyorsítás módja:
 - utasítások párhuzamos végrehajtása
- Probléma:
 - utasítások végrehajtás függ egymástól:
 - ▣ adat függés
 - ▣ vezérlés függés



VLIW architektúra

Very Long Instruction Word (Nagyon Hosszú Utasítás Szó)

- Transmeta Crusoe CPU
- 128-bit-es utasítás csomag = molekula
 - 4 db 32-bit-es atom (atom = utasítás)
 - 4 utasítás párhuzamos elvégzése
- 64 általános használatú regiszter
- Kód átalakító réteg (code morphing layer/code morphing software)
 - A más CPU-k assembly utasításait tartalmazó kódot molekulákra fordítja
 - x86 (Intel Pentium) assembly utasítás-sorozatok futási időben történő „átfordítása” VLIW utasítás molekulákká
 - a végrehajtott utasítások NEM a Crusoe CPU utasításai



Transmeta Crusoe processzor felépítése

Transmeta™ Crusoe™ Processor

Block Diagram

DDR Memory
Interface Controller

SDR Memory
Interface Controller

Transmeta LongRun™
Power/Thermal Management

Serial ROM
Interface Controller

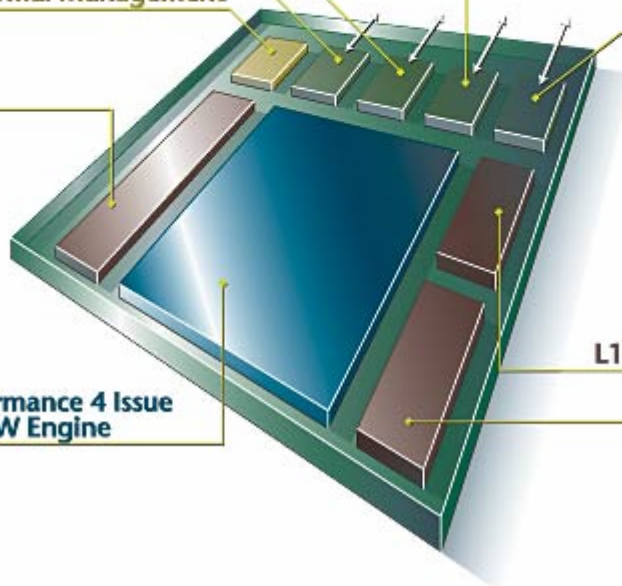
32-bit, 33MHz
PCI Bus
Interface Controller

L2 Cache

High Performance 4 Issue
128-bit VLIW Engine

L1 Instruction Cache

L1 Data Cache

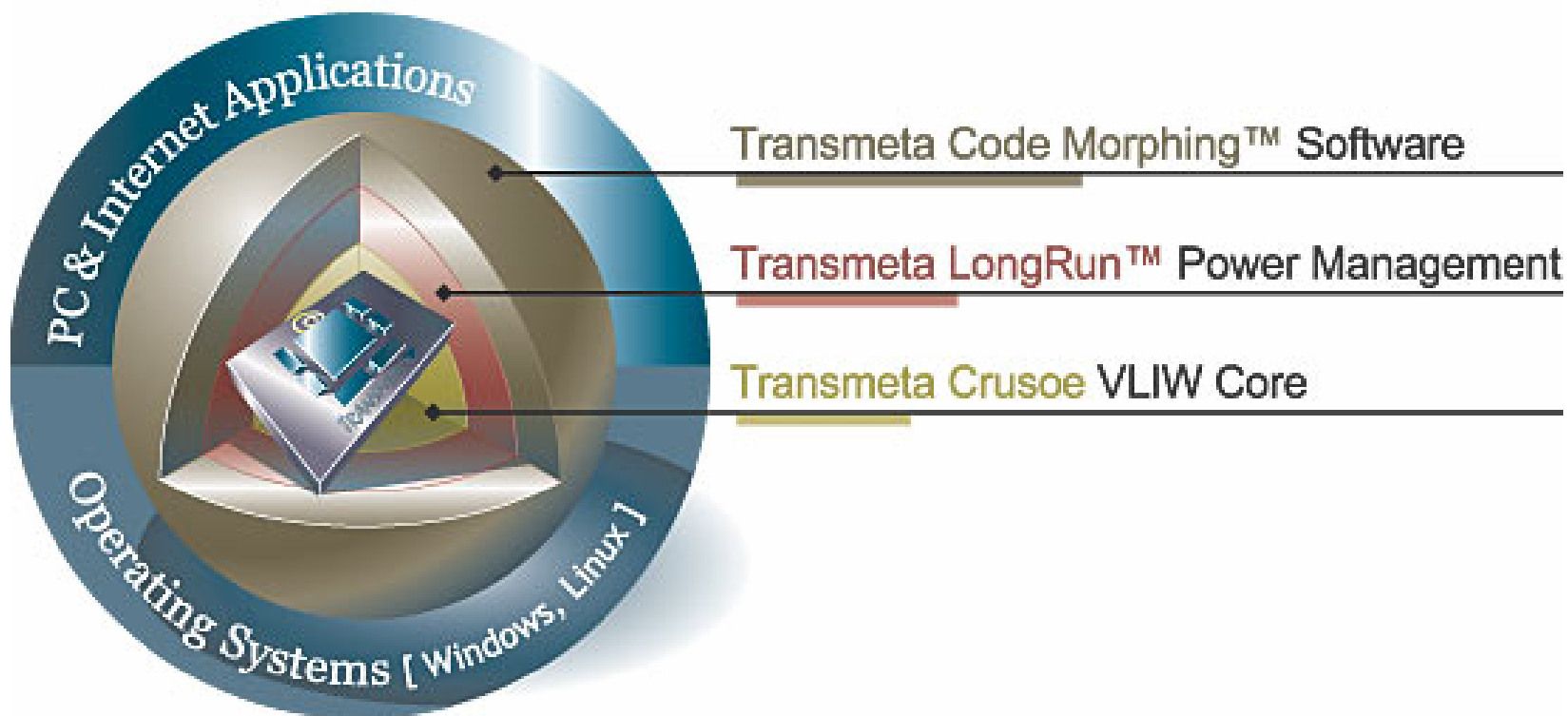




VLIW architektúra

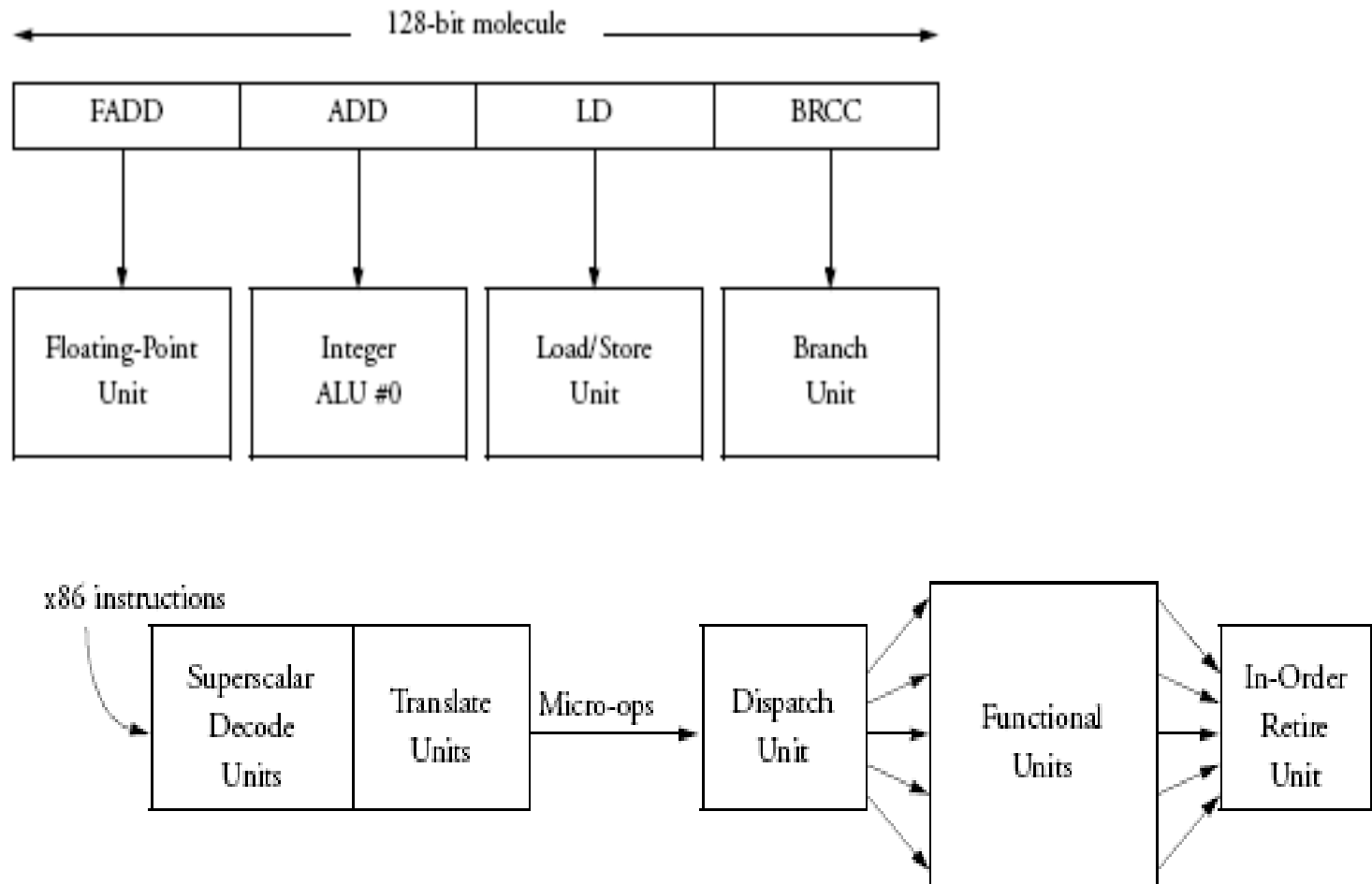
Transmeta™ Crusoe™ Processor

Software Hierarchy





VLIW utasítás végrehajtás





EPIC architektúra

- Intel Itanium CPU (IA-64)
- Új utasítás készlet az EPIC architektúrájú CPU-khoz
- 128-bit utasítás csomag
 - 3 db 41-bit-es utasítás (=123 bit)
 - maradék 5 bit határozza meg az utasítások típusát a csomagban
- 128 db 64-bit-es általános használatú regiszter
- 128 db 82-bit-es lebegőpontos regiszter
- Alkalmassá tették a hagyományos Intel X86-os utasítások végrehajtására is
- Ajánlások programozók és a fordítóprogramok számára közeli utasítások függőségének kiküszöbölésére
 - biztosítja az utasítások párhuzamos végrehajthatóságát



VLIW vs. EPIC

- VLIW
 - bármilyen utasítás sorozat végrehajtása
 - „optimalizálás” (függőségek vizsgálata) a processzor (ill. a processzor környezet) feladata
- EPIC
 - csak „optimalizált” utasítás sorozat végrehajtása
 - „optimalizálás” (függőségek vizsgálata) a programozó (ill. a program fejlesztő környezet) feladata



CPU fejlett szolgáltatásai



Memóriacím-transzformáció: Paging (lapozás)

- Futási időben történő memóriacím-transzformáció (címfordítás)
- Operációs rendszer feladata
- Hardver támogatja a megvalósítását
- A futó programtól független

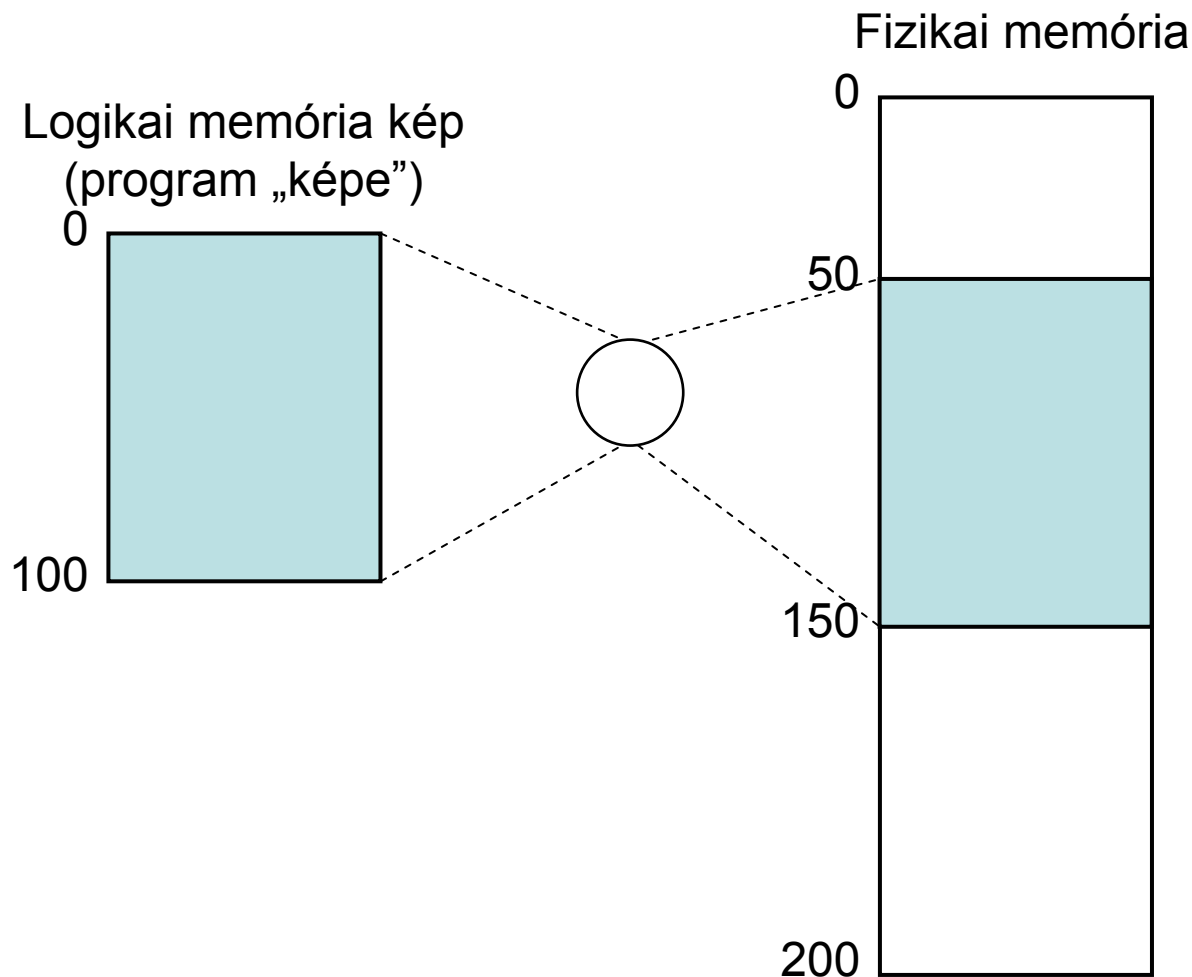


Logikai és fizikai címek összehasonlítása

- A logikai címek az adatok, utasítások, ill. ugróutasítás-címek **relatív** (egymáshoz viszonyított) helyét adják meg, amely **független** azok valós elhelyezkedésétől (fizikai címtől)
- A logikai címeket **átfordítjuk (konvertáljuk)** fizikai címekké
- A fizikai címeknek nem kell feltétlenül sorrendhelyesnek lenni

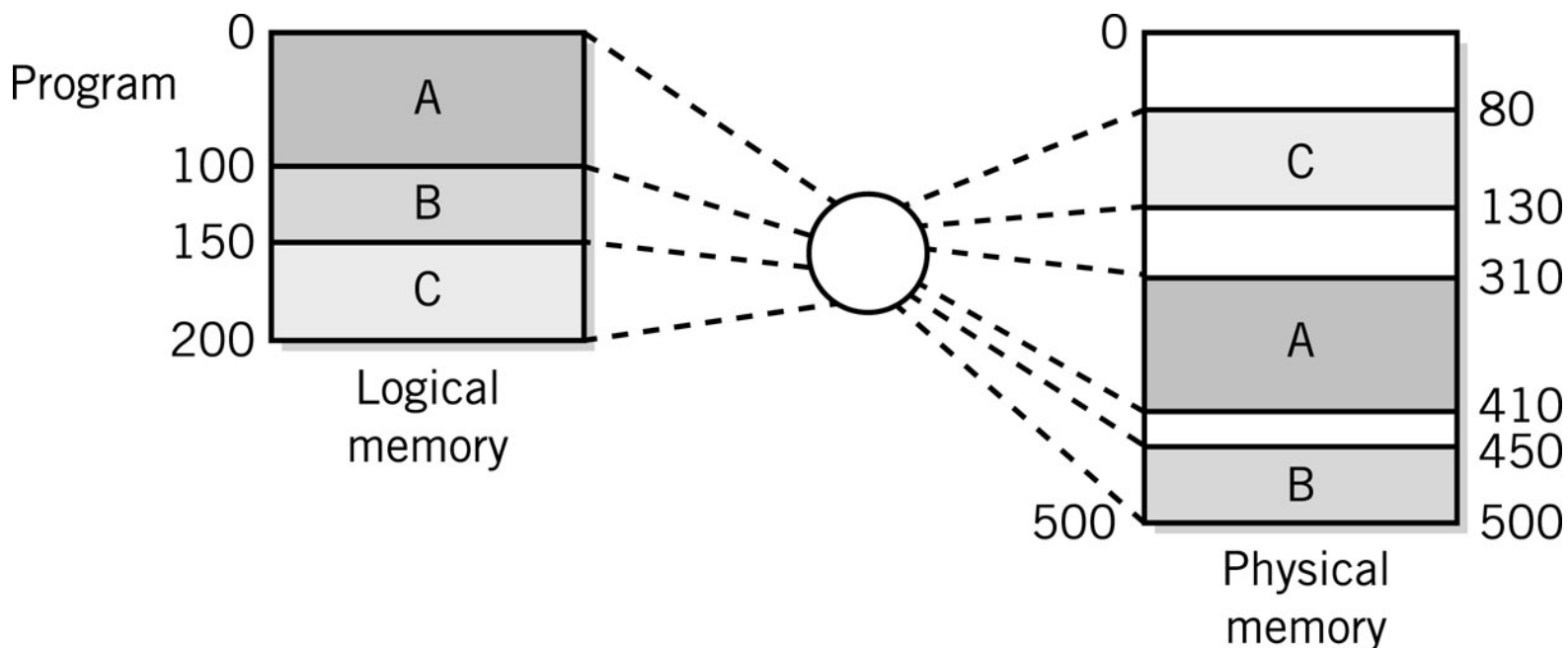


Címfordítás logikája





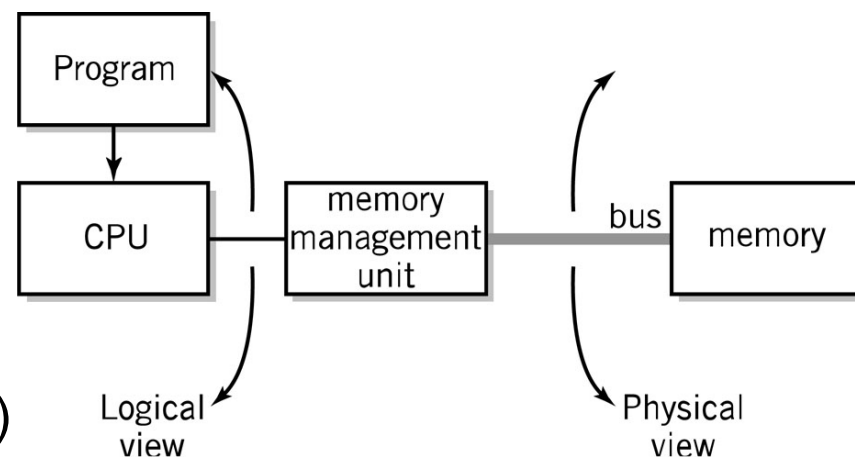
Logikai és fizikai címek összerendelése (mapping)





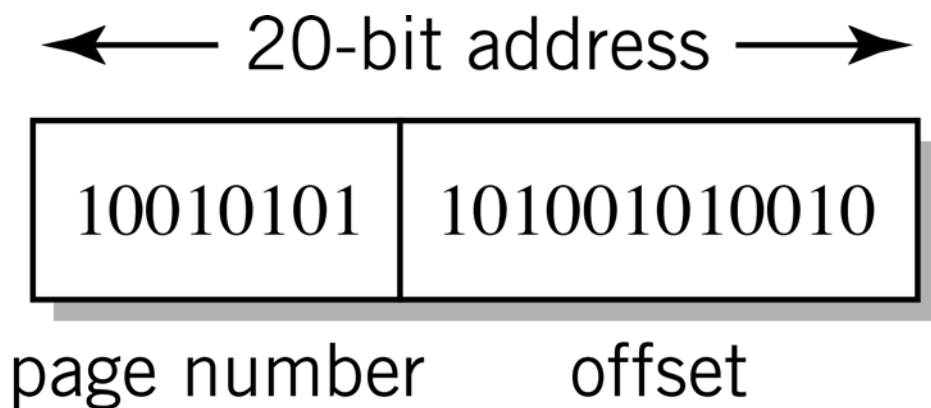
Logikai és fizika memóriakép

- Memória címfordítás HW támogatása
 - MMU (Memory Management Unit)
 - általában része a CPU-nak
- Logikai címek
 - programban tárolt címből a CPU az aktuális címezési logika szerint állítja elő
 - MMU (+operációs rendszer) átfordítja fizikai címmé
- Fizikai címek
 - CPU külső buszán jelennek meg



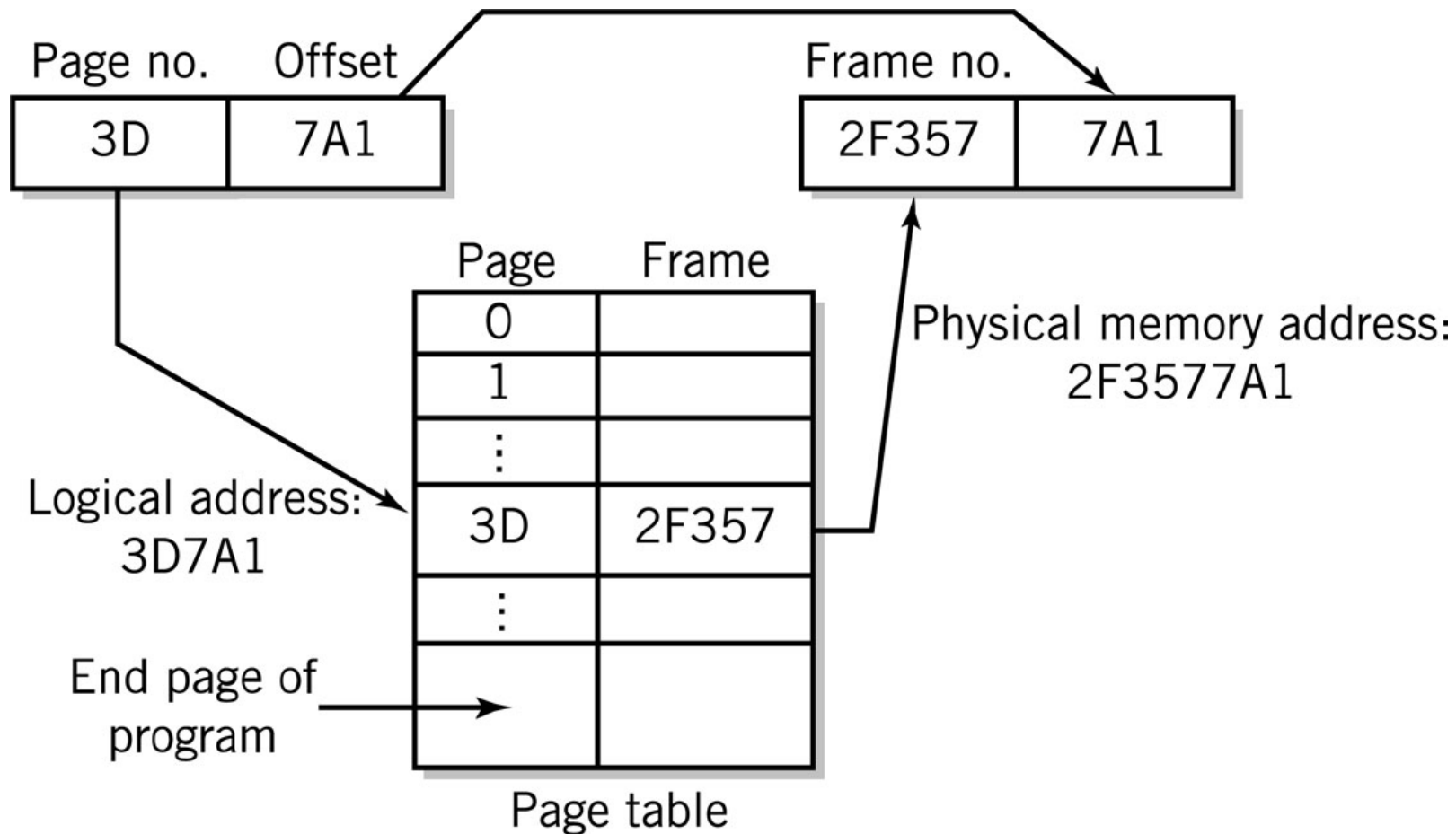


Lapcím felépítése



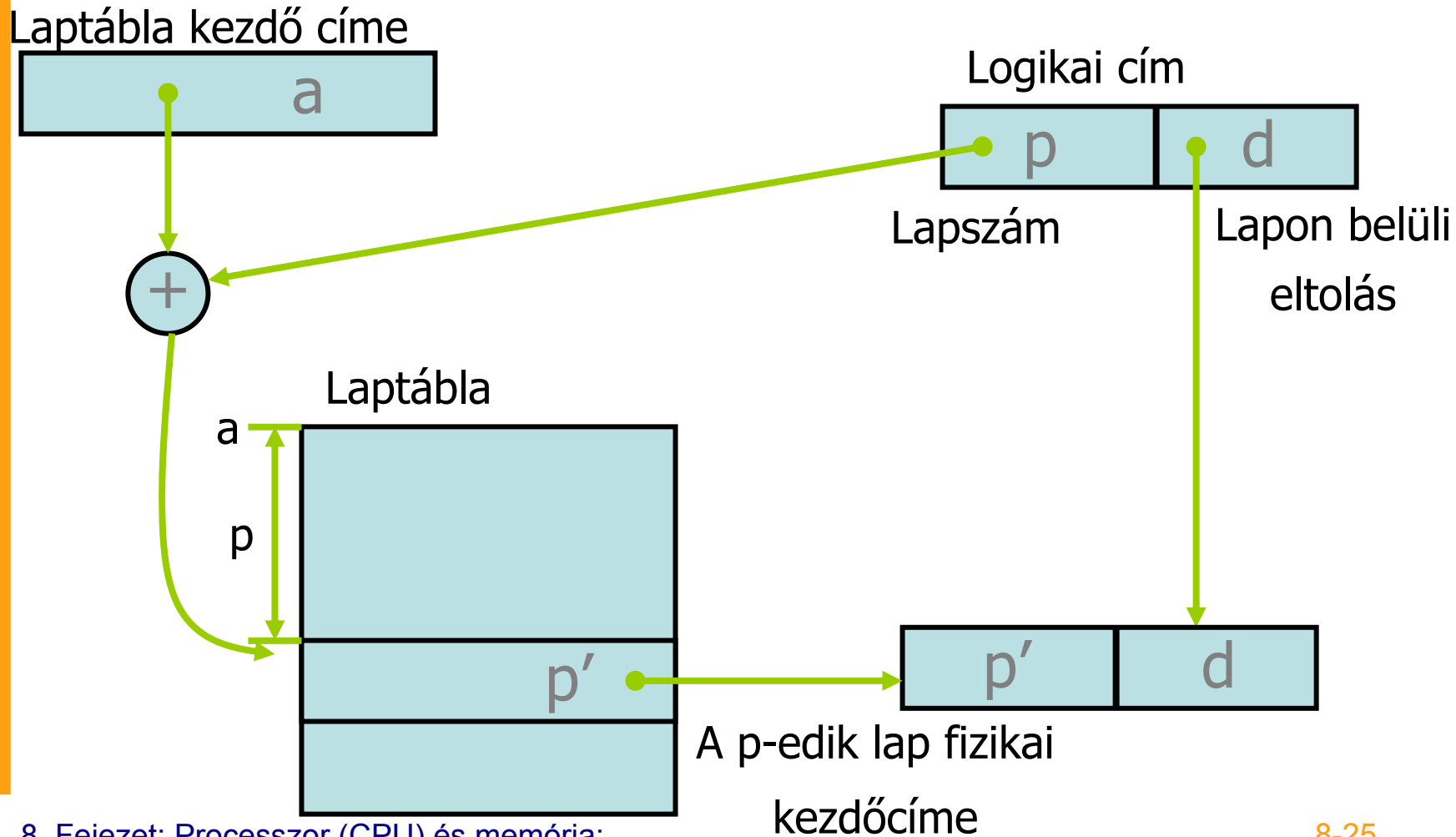


Címfordítás menete





Címfordítás lapozás esetén





Címfordítás szegmentálás esetén

Szegmenstábla kezdő címe



Logikai cím

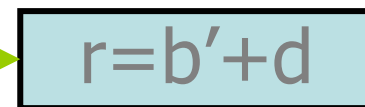
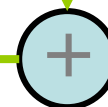
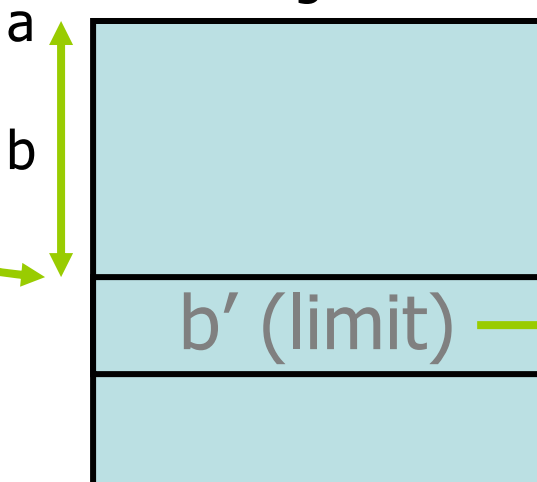


Szegmens szám

Szegmensen belüli eltolás



Szegmenstábla



A b-edik szegmens fizikai kezdőcíme



Címtranszformáció adta további lehetőségek

- Memória elérés:
 - címtranszformáció
- MMU (+ OR):
 - címfordító táblázat
- Virtuális memóriakezelés (VM):
 - memória kiváltására használjunk háttértárat
 - címtranszformációt egészítsük ki a VM támogató lépésekkel
 - Bonyolult működés, számos többlet feladat: operációs rendszer!



Fejlett módszerek a memória- elérés gyorsítására

**Wide Memory Path Access
(Széles sávon történő memória elérés)**

**Memory Interleaving
(Memória felosztás)**

Cache memória

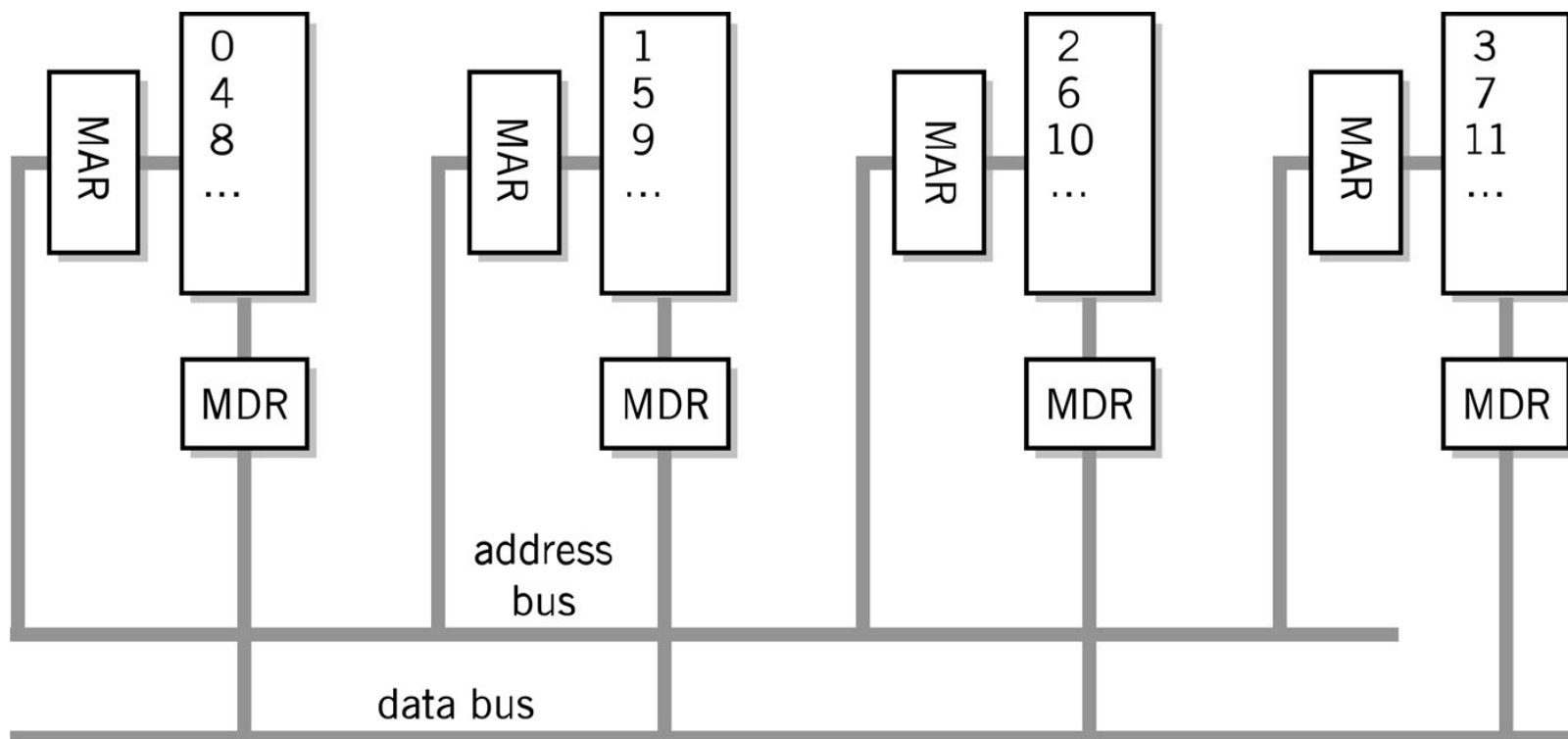


A memória elérés gyorsítása

- A memória lassú a CPU sebességéhez képest!
 - 2 Ghz-es CPU = 1 ciklus a másodperc $\frac{1}{2}$ milliárdod része alatt ($0,5 \times 10^{-9}$ másodperc)
 - 70ns DRAM = 1 elérés a másodperc 70 milliomod része alatt (70×10^{-9} másodperc)
- Eljárások a memória elérések gyorsítására
 - **Wide Memory Path Access (~Széles sávú memória elérés)**
 - Több byte-ot olvas ki a memóriából egy olvasási ciklusban
 - Széles (2,4,8) byte-os adat busz és MDR
 - **Memory Interleaving (~Memória felosztás)**
 - Memóriát részekre osztjuk, mindegyiknek lesz saját cím- és adatregisztere (MAR és MDR)
 - **Cache memória**



Memory Interleaving (Memória felosztás)



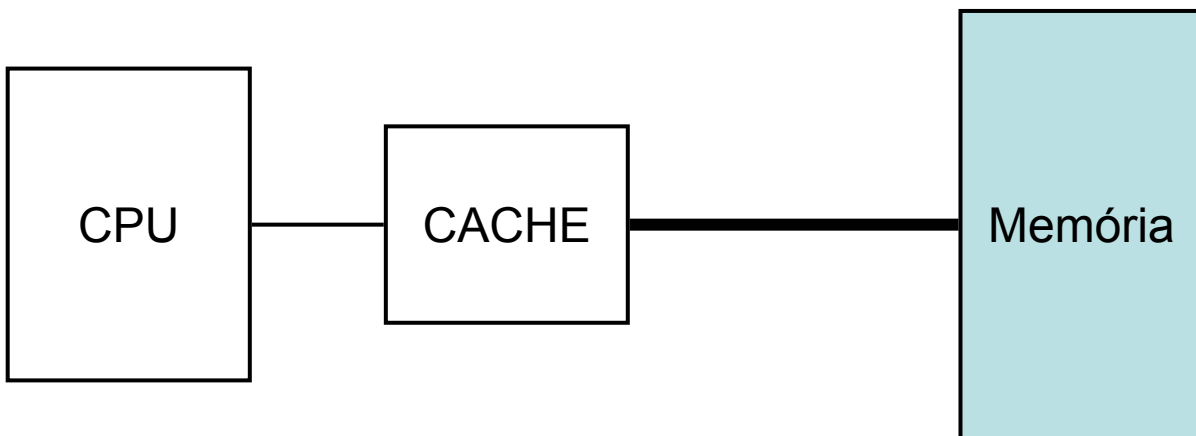


Cache



Cache

- Gyorsítótár, gyors elérésű memória (SRAM) a memória (ill. más lassú tárolók) elérésének meggyorsítására





Miért éri meg a cache használata?

- A leggyorsabb merevlemezek elérési ideje is körülbelül 10 milliszekundum (10×10^{-3} másodperc) körül van
- 2 GHz-es CPU-nak 10 milliszekundumot kellene várnia egy merevlemezről beolvasott adat elérésekor
($10 \times 10^{-3} / 0.5 \times 10^{-9} = 2 \times 10^7$)
- **20 millió órajelciklust veszítünk el!**



Cache memória felépítése

- Blokkokba szervezett felépítés
- Blokk mérete: 8 vagy 16 byte
 - a memória (tároló) egy részének másolata
 - Pl. 64 KB cache → 8192 db 8 byte-os blokk
- Minden bloknak van egy címkéje:
 - egy memória (tároló) cím!
- Cache controller (vezérlő)
 - Hardver elem, ami képes a címkék kezelésére
- Cache line (vonal)
 - a memória (tároló) és cache memória közötti adatcserét lebonyolító elem

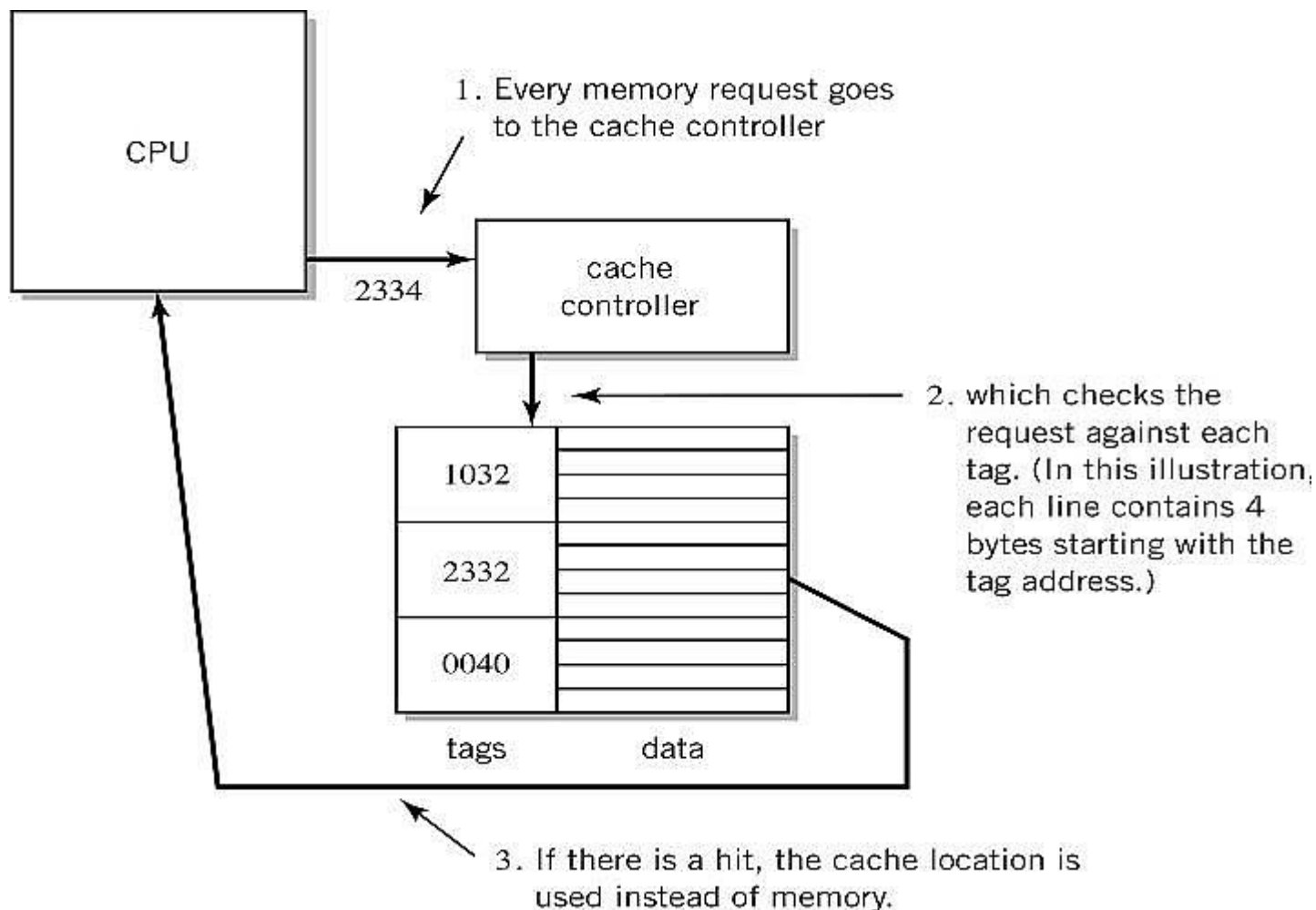


Cache működése

- Memória olvasása
 - Memória elérésekor a cache controller ellenőrzi, hogy a kért tartalom a cache-ben van-e
- Találati arány (hit ratio)
 - a cache találatok aránya az összes memória kérésből
- Memória írása
 - Feladat a cache és a memória összehangolása
- Alternatív megoldások
 - **Write through (~Átírás)**
 - ▣ azonnali memória frissítés
 - ▣ lassabb, de biztonságos
 - **Write back (~Visszaírás)**
 - ▣ memória frissítés csak felülíráskor
 - ▣ gyorsabb, de kockázatos

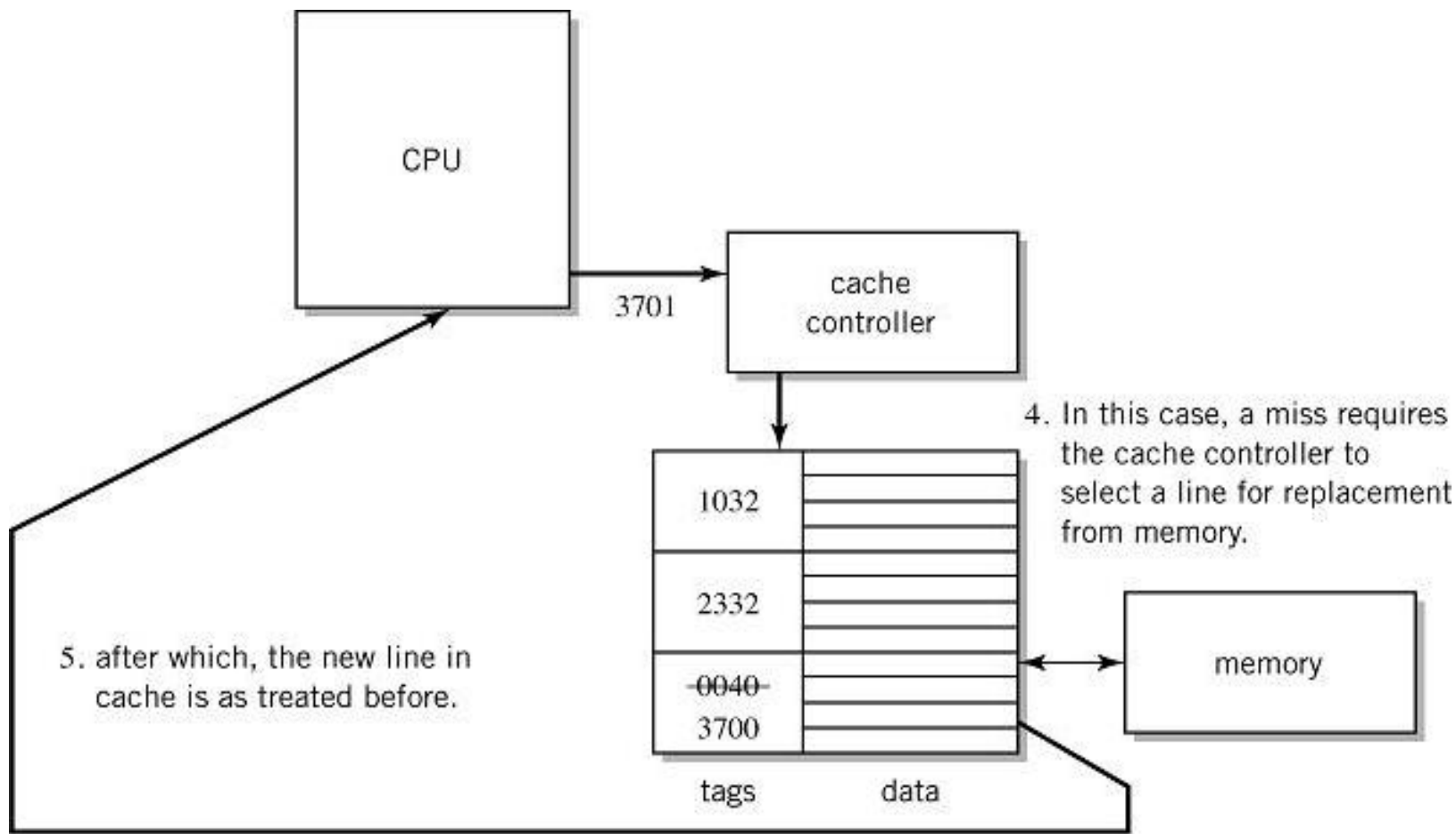


Cache használata lépésről-lépésre





Cache használata lépésről-lépésre





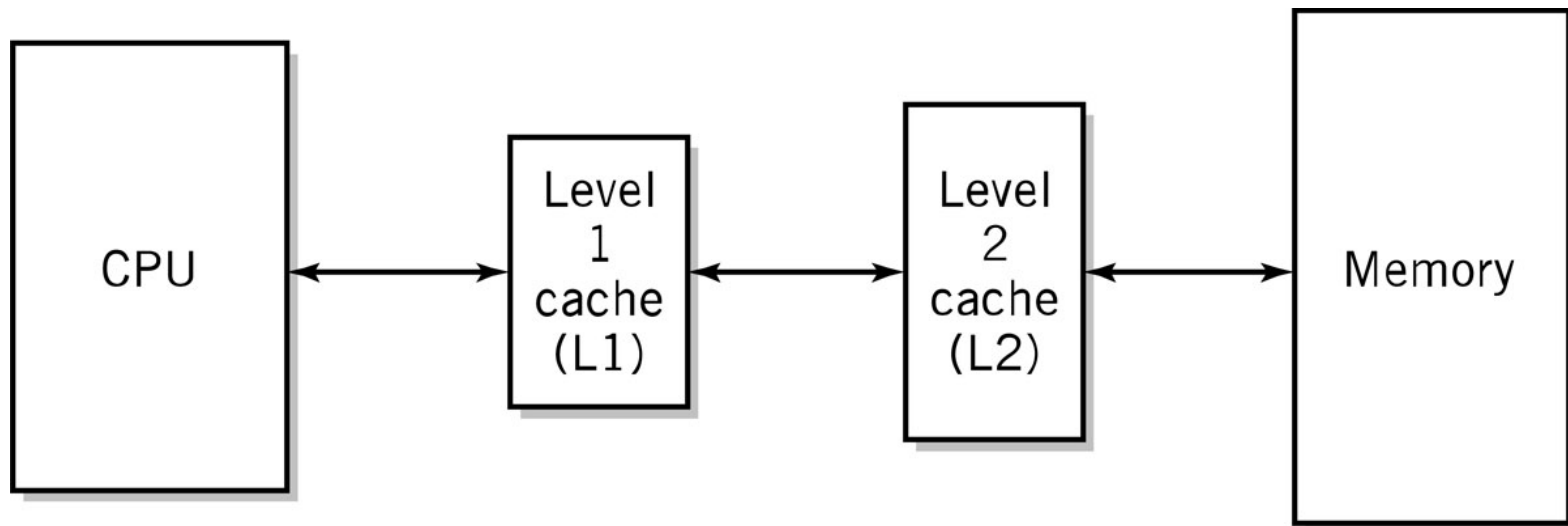
Teljesítménybeli előnyök

- Általános a 90% körüli találati arány
- 50%-ot is meghaladó műveleti sebesség növekedés
- A programok végrehajtásának „lokális természete” miatt működik hatékonyan a cache
 - ***Rövid időintervallumokat nézve a CPU által végrehajtott memória hivatkozások általában a memória kis régiójára vonatkoznak!!!***
 - Jól megírt program jellemzően rövid ciklusokat, eljárásokat vagy funkciókat tartalmaz
 - A feldolgozott adatok gyakran tömbökben vannak tárolva egymás után
 - Egy adott funkció megvalósításához szükséges változókat együtt tároljuk



Kétszintű cache-ek

- Miért kell a két cache méretének különbözőnek lenni?





Cache és virtuális memória összehasonlítása

- Cache **felgyorsítja** a memória elérést
- A virtuális memória használata megnöveli a programok által **„tapasztalt”** tár nagyságát
 - Alkalmazásának lehetősége az adott gép konfigurációjától és a memória kapacitásától függ
 - A memóriát alacsony bit-enkénti költséggel tudja megnövelni



CPU műveletek végrehajtásának gyorsítása



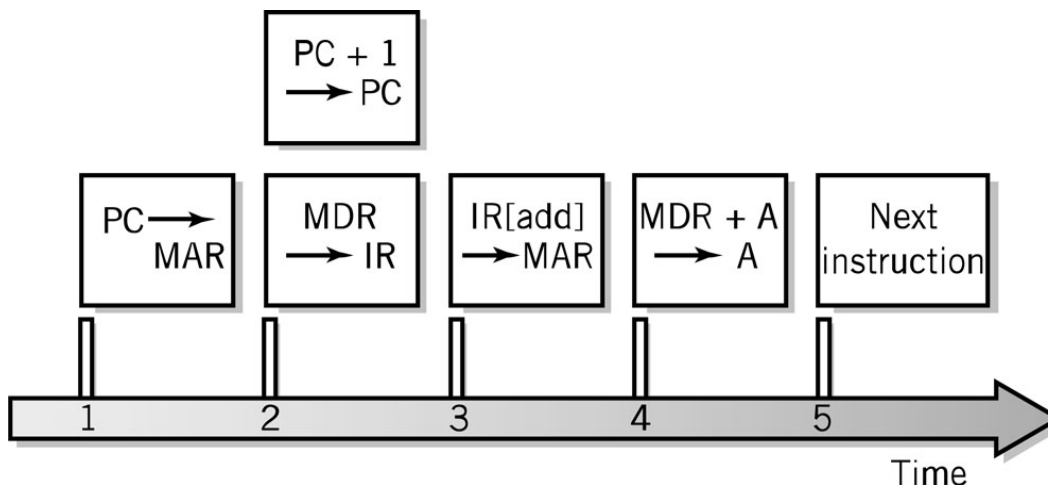
Utasítás végrehajtás modern CPU architektúrákban

- Utasítások ütemezett végrehajtása
- Fetch/Execute egységek különválasztása
- Pipeline (futószalag jellegű) feldolgozás
- Feldolgozás skalár processzorokban
- Feldolgozás superskalár processzorokban



Időzítéssel kapcsolatos kérdések

- Számítógép órajelét használjuk az utasítások lépéseinek ütemezésére
 - MHz – egymillió (10^6) lépés másodpercenként
 - GHz – egymilliárd (10^9) lépés másodpercenként
- Az utasítások végrehajtása általában több lépésből áll



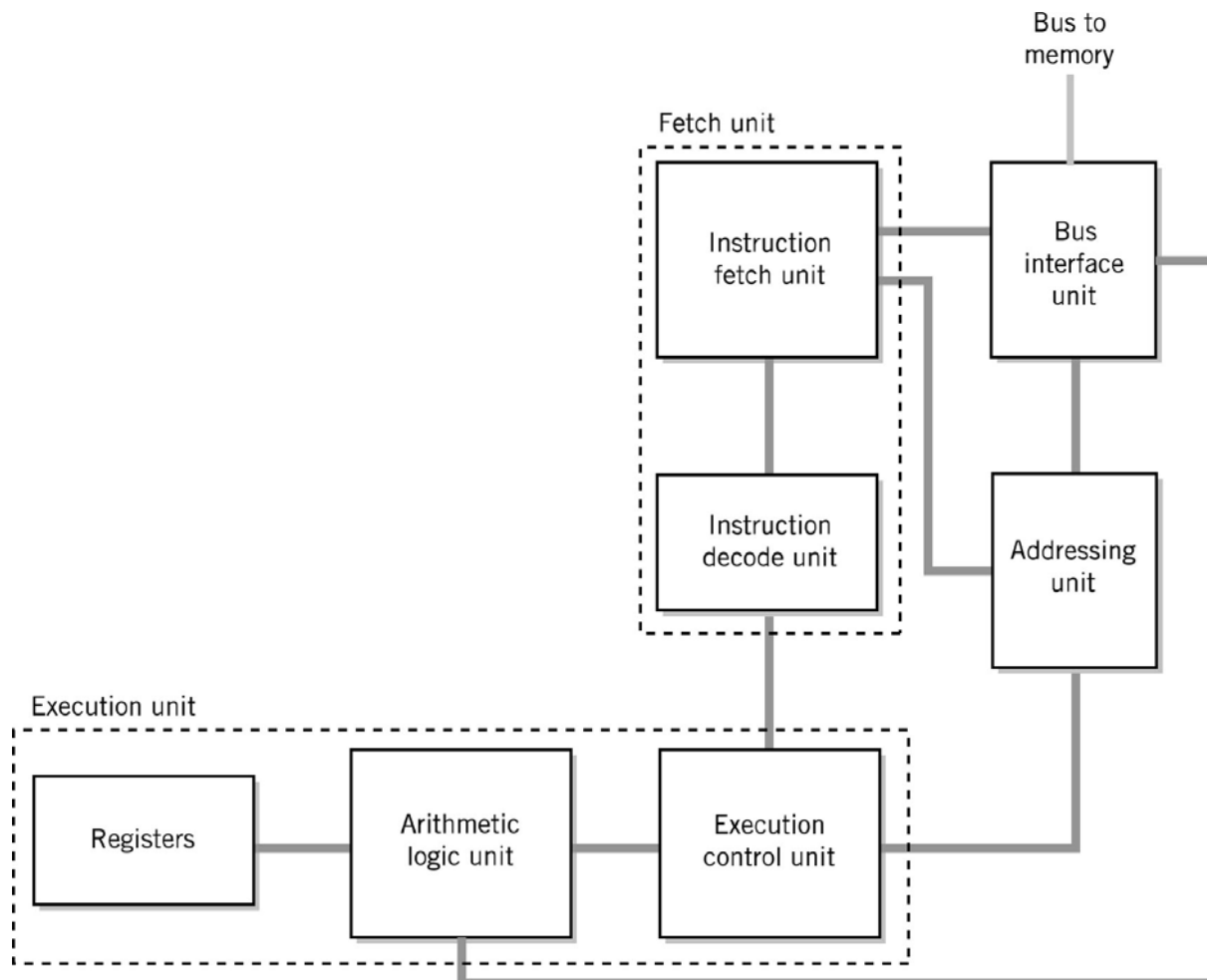


Fetch-Execute egységek különválasztása

- Fetch fázist végrehajtó egység
 - Utasítást memóriából beolvasó egység
 - Utasítás dekódoló egység
 - ▣ Műveleti kód (opcode) meghatározása (utasítás része)
 - ▣ Utasítás típusának és az operandusok azonosítása
 - Ha több utasítás egymással párhuzamosan van fetch-elve, a dekódolásig, ill. a végrehajtásig egy pufferben tároljuk
 - IP – Instruction Pointer (utasítás mutató) regiszter
- Execute fázist végrehajtó egység
 - Az utasítás dekódoló egységtől kap utasításokat
 - A megfelelő végrehajtó egységek hajtják végre az utasításokban kódolt műveleteket



CPU lehetséges belső felépítése





Pipeline utasítás feldolgozás

- Az utasításokat közel azonos hosszú lépésekre bontjuk (pl. fetch és execute fázis)
- A lépéseket egymástól függetlenül működő egységek hajtják végre a CPU-n belül
- „Futószalag” technika, hogy átfedés jöjjön létre a lépések (pl. fetch-execute fázisok) végrehajtása között az egymást követő utasítások feldolgozásakor



Pipeline feldolgozás jellemzői

- Az egyes utasítások végrehajtási ideje nem változik
- Az utasítások párhuzamos végrehajtása miatt gyorsul a feldolgozás
- **Egy feldolgozó egység: egy adott pillanatban (órajelciklusban) csak egy utasítást lehet befejezni**
- Feldolgozás skalár processzorokban
 - A végrehajtott utasítások átlagos száma közel egyenlő az eltelt órajel-ciklusok számával



Pipeline feldolgozás - problémák

- Az utasítások különböző számú lépésből állhatnak
 - Nem tudjuk kihasználni a pipeline lehetőségeit
- Ha az utasításokat szekvenciálisan kell végrehajtani nem gyorsít
 - Adat függések esetén ez a helyzet
- Elágazás (branch) utasítások esetén a pipeline nem jó utasításokat hajthat végre



Elágazás utasítások kezelése

- Mindkét lehetőségre külön a pipeline sort kezel a CPU
- Jóslás alapú megoldások
- Megkötések az utasítások sorrendjére, úgy, hogy az elágazás utasítás utáni utasítás ne függjön a elágazás utasítás feltételétől

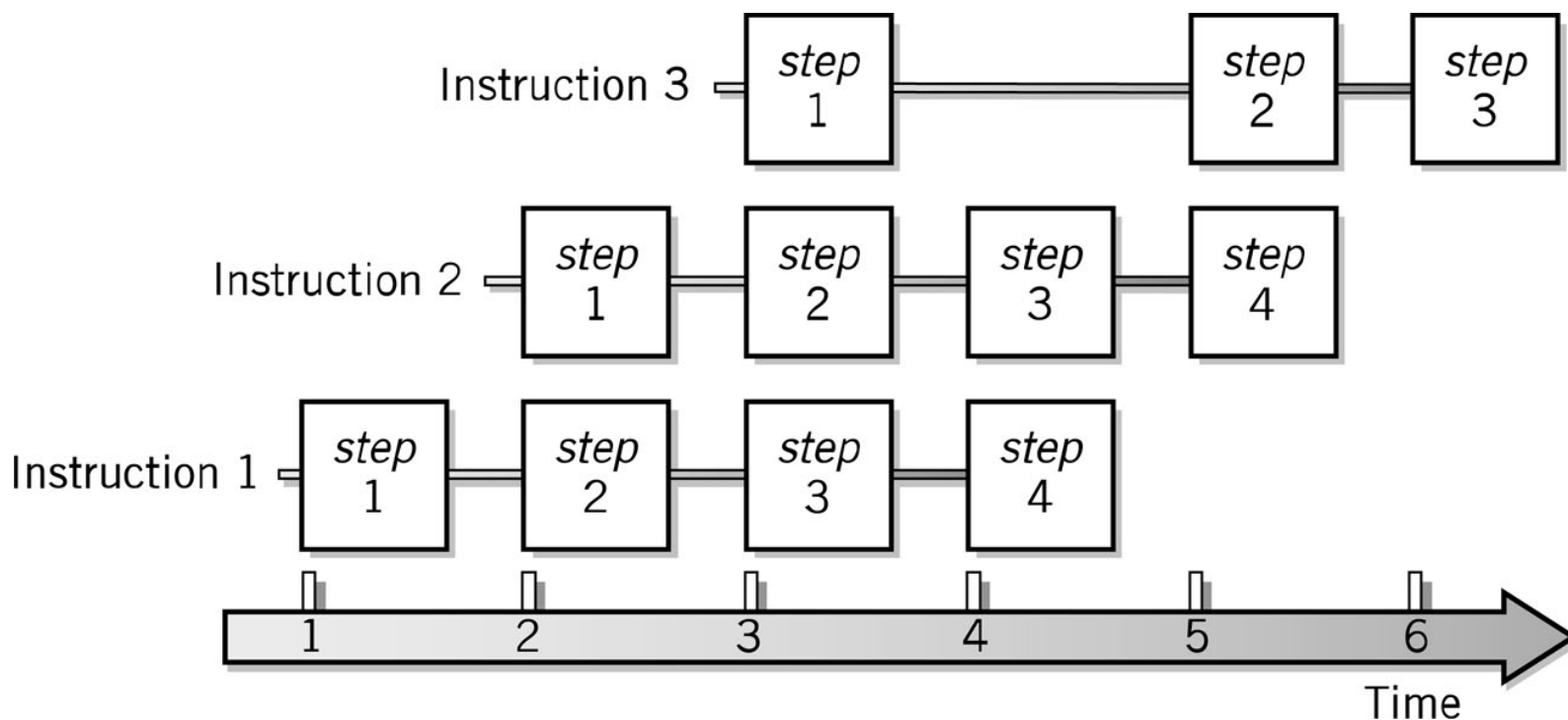


Adat függések kiküszöbölése

- Utasítások átrendezése
 - cél: egymás után következő utasítások eredményei ne függjenek egymástól
 - gyakran alkalmazott megoldás szuperskalár processzorokban a párhuzamos pipeline sorok feltöltésére



Pipeline példa



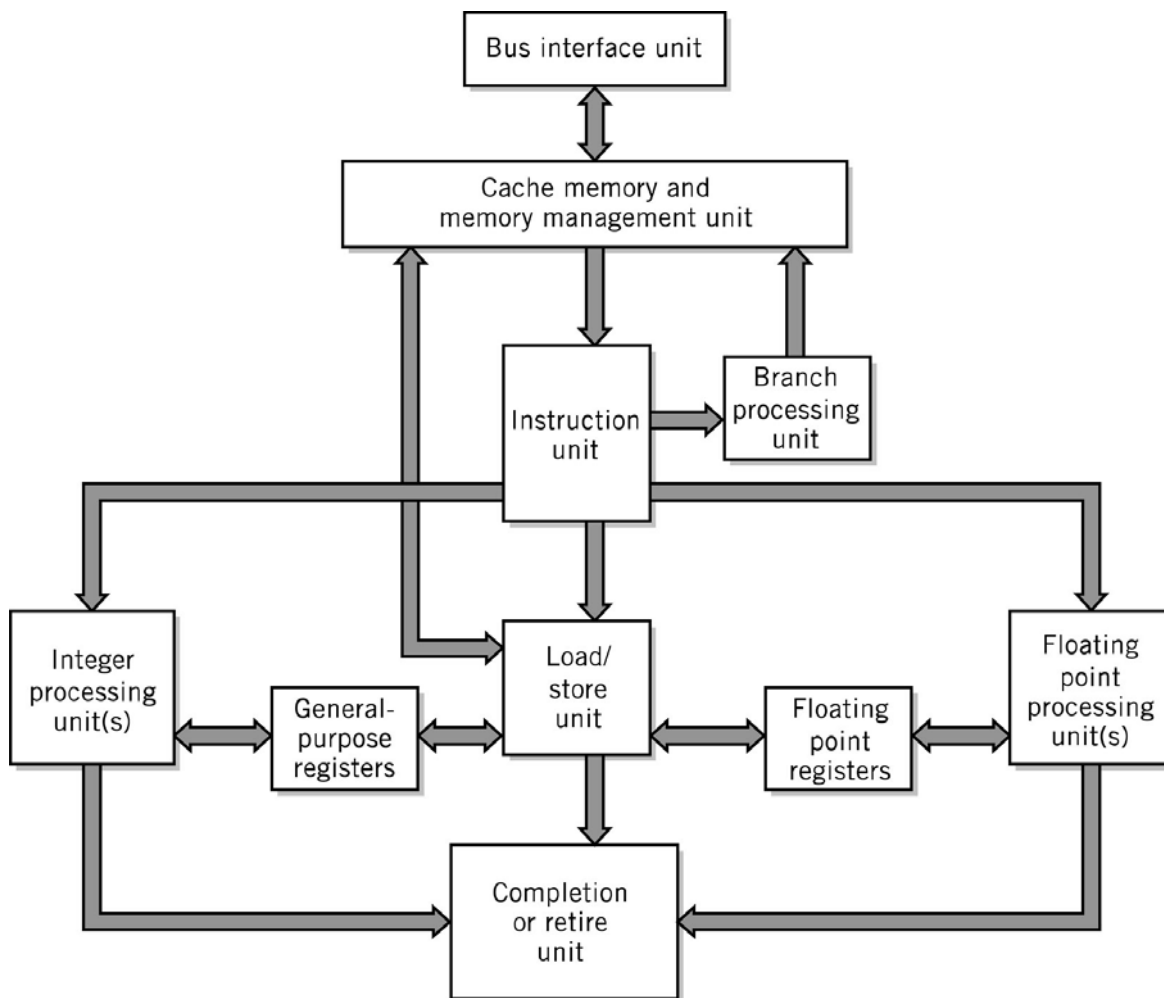


Feldolgozás szuperskalár processzorokban

- Átlagosan több, mint egy utasítást hajt végre egy órajel alatt
- A lehető legjobban elkülöníti a fetch és az execute utasításfázisokat
- Átmeneti tárolók (puffer regiszterek) a fetch és a dekódoló a fázisok
- Több piplene sor kezelése különböző utasításcsoportokhoz
 - Utasítás-csoportokbeli utasítások hossza azonos!!
 - Párhuzamos végrehajtó egységek
 - ▣ Load/Store utasítás végrehajtó egység
 - ▣ Branch (elágazás) kezelő egység
 - ▣ Integer aritmetikai egység
 - ▣ Lebegőpontos aritmetikai egység

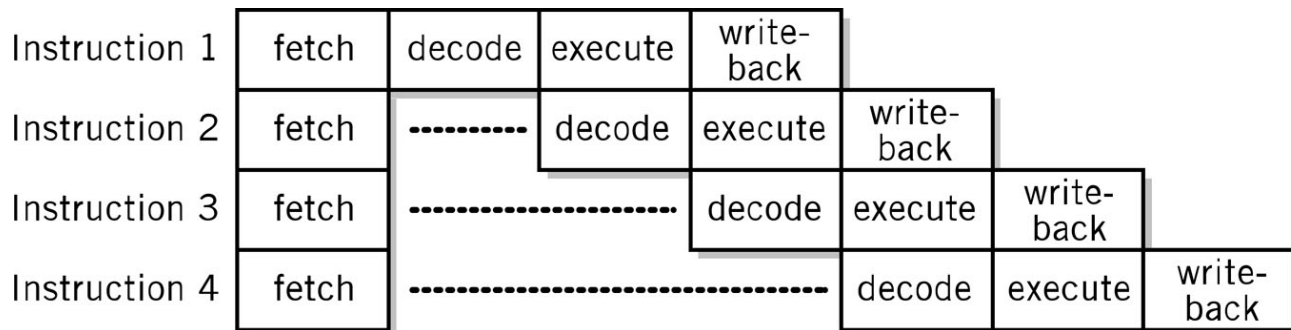


Szuperskalár CPU blokkdiagramja

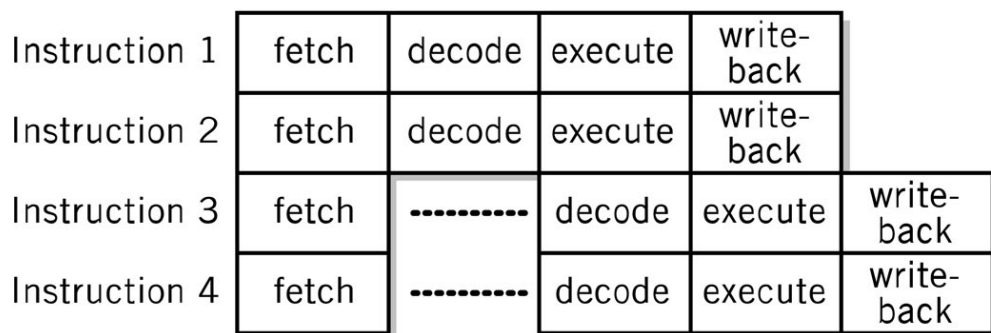




Skalár és Szuperskalár feldolgozás



a. Scalar



b. Superscalar





Megoldások szuperskalár processzorokban

- Függőségek (hazárdok) kezelése fontos
- Soron kívüli (out-of-order) feldolgozás
 - „Elvégezhető” utasítások keresése a következő (akár 10-20) utasítás között
 - Jellemző adatfüggések „kikerülésére”
- Elágazás utasítások feldolgozása
 - Párhuzamos több pipeline sor kezelése vagy az elágazások jóslása
 - „Branch History Table” használata a jóslás javítására
- Regiszterek párhuzamos használatából adódó ütközések
 - Logikai regiszterek használata
 - Változtatható szerepű regiszterek



CPU implementációja



CPU műveletek hardveres megvalósítása

- Hardver implementáció
 - a műveleteket logikai kapuk hajtják végre
 - adatmozgató, logikai, bitforgató műveletek
- Előnyei
 - Nagy sebesség
 - RISC processzorok jellemzően ilyenek
 - ▣ utasítások egyszerűek, gyorsak



Mikroprogramozott CPU implementáció

- A CPU un. mikro-programokat hajt végre
 - A mikro-programok rövid programok, amelyeket a CPU-ba integrált ROM tárol
 - A használható utasítások egyszerűek: pl. regiszterek közötti adatcsere, logikai egység vezérlése.
 - A CPU utasításokat ilyen mikro-program utasításokból lehet összeállítani
- Előnyei
 - Rugalmasabban kezelhető utasításkészlet
 - Könnyebb az összetett utasítások végrehajtása
 - Tudjuk emulálni más CPU-k utasításkészletét
- Hátránya
 - Az utasítások végrehajtása általában több órajelet igényel



Copyright 2003 John Wiley & Sons

All rights reserved. Reproduction or translation of this work beyond that permitted in Section 117 of the 1976 United States Copyright Act without express permission of the copyright owner is unlawful. Request for further information should be addressed to the permissions Department, John Wiley & Sons, Inc. The purchaser may make back-up copies for his/her own use only and not for distribution or resale. The Publisher assumes no responsibility for errors, omissions, or damages caused by the use of these programs or from the use of the information contained herein.”



9. Fejezet: Input/Output

The Architecture of Computer Hardware and Systems Software: An Information Technology Approach

3rd Edition, Irv Englander

John Wiley and Sons ©2003

Wilson Wong, Bentley College

Linda Senne, Bentley College



Alapmodell

- Feldolgozási sebesség vagy program futtatás
 - Elsőként meghatározott az I/O műveletek által, hogy egyenesben tartsa a processzort





Szempontok I/O műveletek megvalósításakor

Sebesség

- CPU gyorsabban végzi el a műveleteket, mint a leggyorsabb I/O egység
- Az I/O egységek feldolgozási sebessége különböző
- Az adatok beérkezése nem egyenletes (burst-ös)
- Néhány eszköz igényli a blokkos adatátvitelt

Egységek összehangolása

- Különböző eszközök párhuzamosan hajtanak végre I/O műveleteket
- Váratlan input (beavatkozás) lehetősége
- Különböző formátuma az input adatoknak
- Minden eszköznek szüksége van státusz-információkra



Szemponatok I/O eszköz interfészek kialakítására

- Különböző formátumok
 - Párhuzamos interface
 - Soros interface
- Adatok pufferelése
- Burst vs. Stream
- Különböző követelmények a vezérlésre
 - elektromechanikai elemek vezérlése

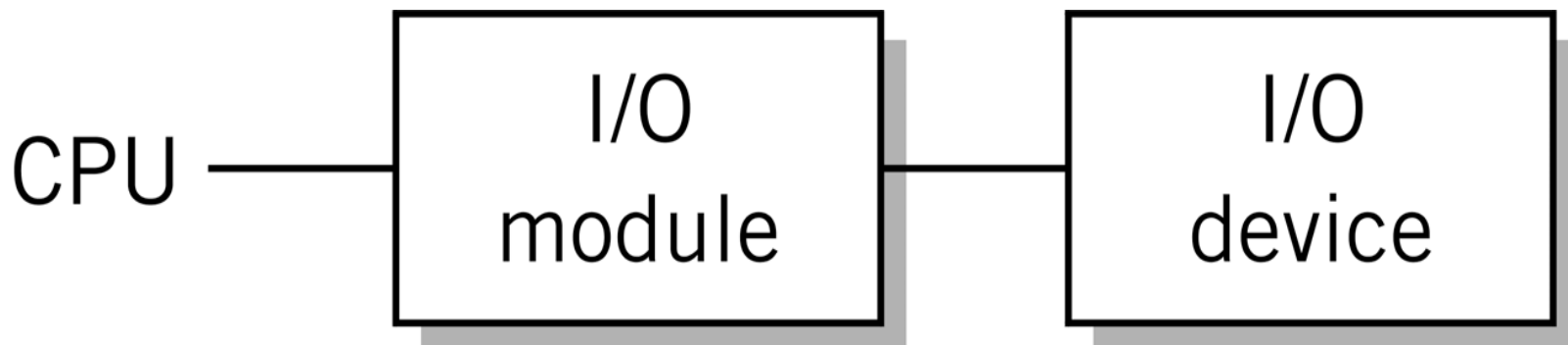


Példák I/O egységekre

Device	Input/Output	Data rate (in Kbytes/sec)	Type
Keyboard	Input	0.01	char
Mouse	Input	0.02	char
Voice input	Input	0.02	block burst
Scanner	Input	200	block burst
Voice output	Output	0.5	block burst
Inkjet printer	Output	1.5	block burst
Laser printer	Output	100-1,000	block burst
Graphics display	Output	30,000	block burst or steady
Local area network	Input or output	200-20,000	block burst or steady
Optical disk	Storage	500-15,000	block burst or steady
Magnetic tape	Storage	1,000-15,000	block burst or steady
Magnetic disk	Storage	2,000-60,000	block burst or steady
Non-compressed video source	Input	10,000	block steady
Non-compressed audio source	Input or output	100	block steady



Alap I/O konfiguráció





I/O modulok funkciói

- Felismeri az általa vezérelt egység(ek)től érkező hozzá címzett üzeneteket, és elfogadja a CPU- tól a parancsokat
- A memóriából származó adatokat átmenetileg tárolja a pufferben, amíg azt az I/O eszköz fel nem dolgozza
- Biztosítja a DMA-hoz szükséges regisztereket és vezérlő funkciókat
- Fizikailag vezérli az eszközöket
- A pufferből adatokat küld az eszköznek, ill. adatokat fogad a CPU-tól és eltárolja a pufferbe
- Megszakítások formájában jelzést küld a CPU számára az I/O folyamatok állapotáról



Input/Output modulok

- Programozott I/O
 - CPU által irányított I/O
- Megszakítás vezérelt I/O
 - Külső input vezérli a folyamatot
- DMA – Direct Memory Access controller:
(Közvetlen memória-elérés vezérlő)
 - Módszer az adatátvitelre az eszköz és a (fő)memória között a CPU közreműködése nélkül

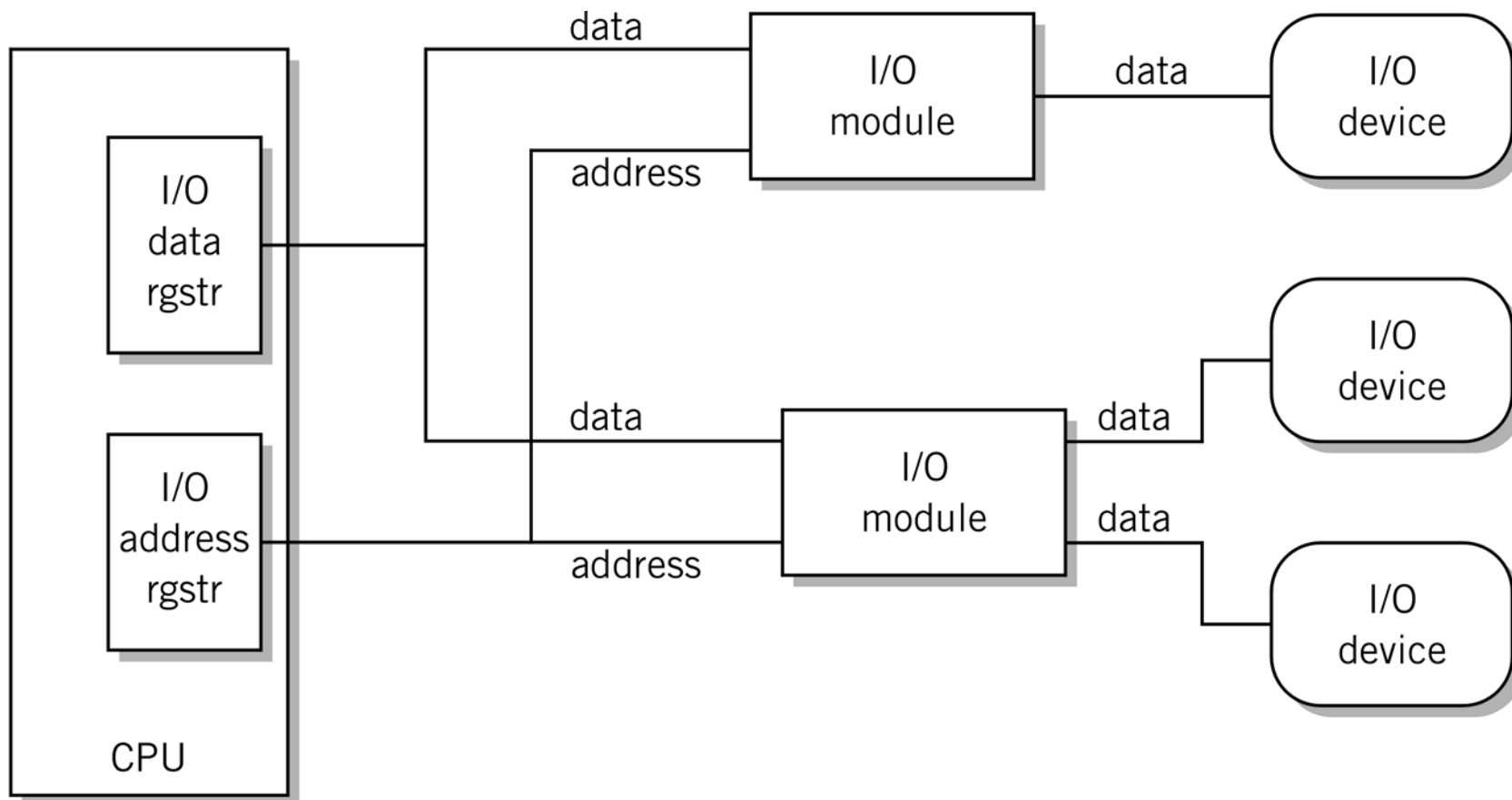


Programozott I/O kezelés

- I/O adat és cím regiszterek a CPU-ban
- Egy szónyi adat továbbítása
- Az összes I/O eszköznek van egyedi címe
 - LMC I/O kapacitás 100 eszközt tud kezelni
- Egy külön utasítás az fetch/execute ciklussal
- Elsődleges használat:
 - billentyűzet
 - kommunikáció I/O modulokkal (lásd DMA)

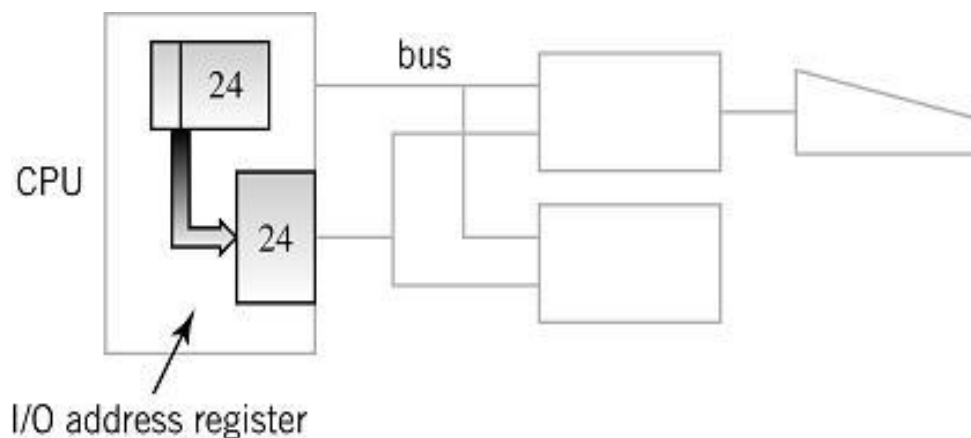


Programozott I/O kezelés

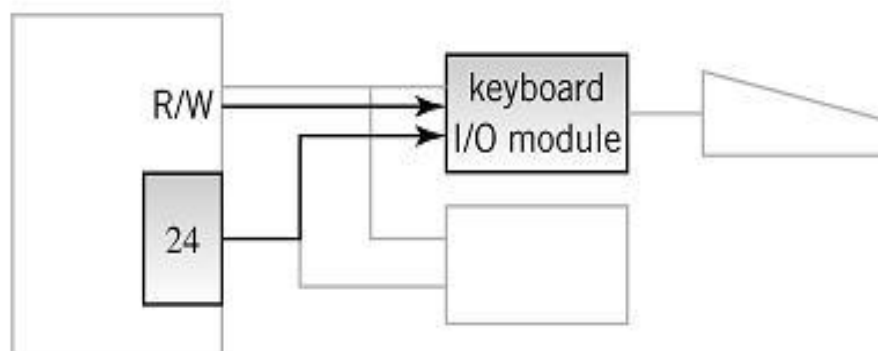




Programozott I/O példa



1. CPU executes INPUT 24 instruction. Address 24 is copied to the I/O address register.

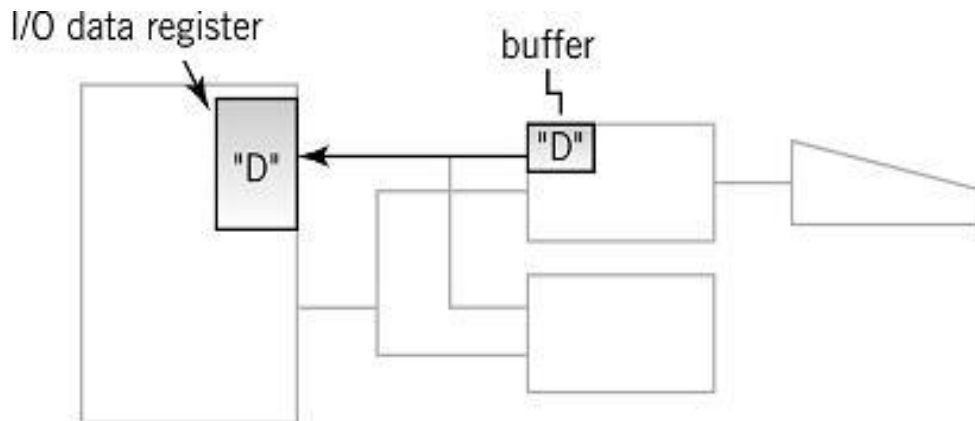


2. Address 24 is recognized by the keyboard I/O module. A read/write control line indicates that the instruction is an INPUT.

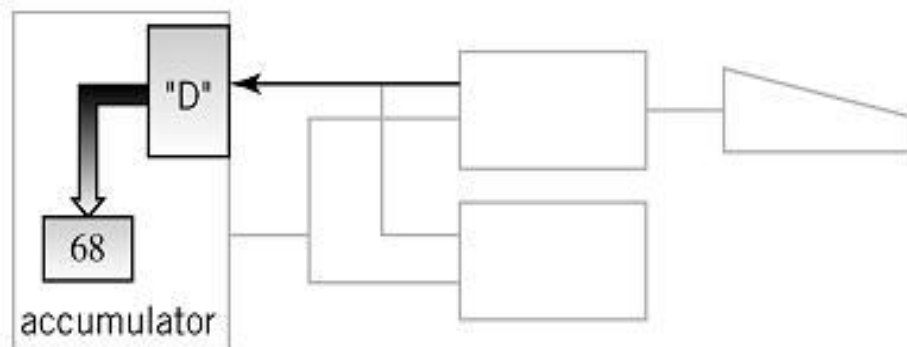
(Figure continues on next slide)



Programozott I/O példa



3. A buffer in the I/O module holds a keystroke, in this case ASCII 68, the letter "D". The data is transferred to the I/O data register.



4. From there it is copied to the appropriate accumulator or general-purpose register, completing the operation.



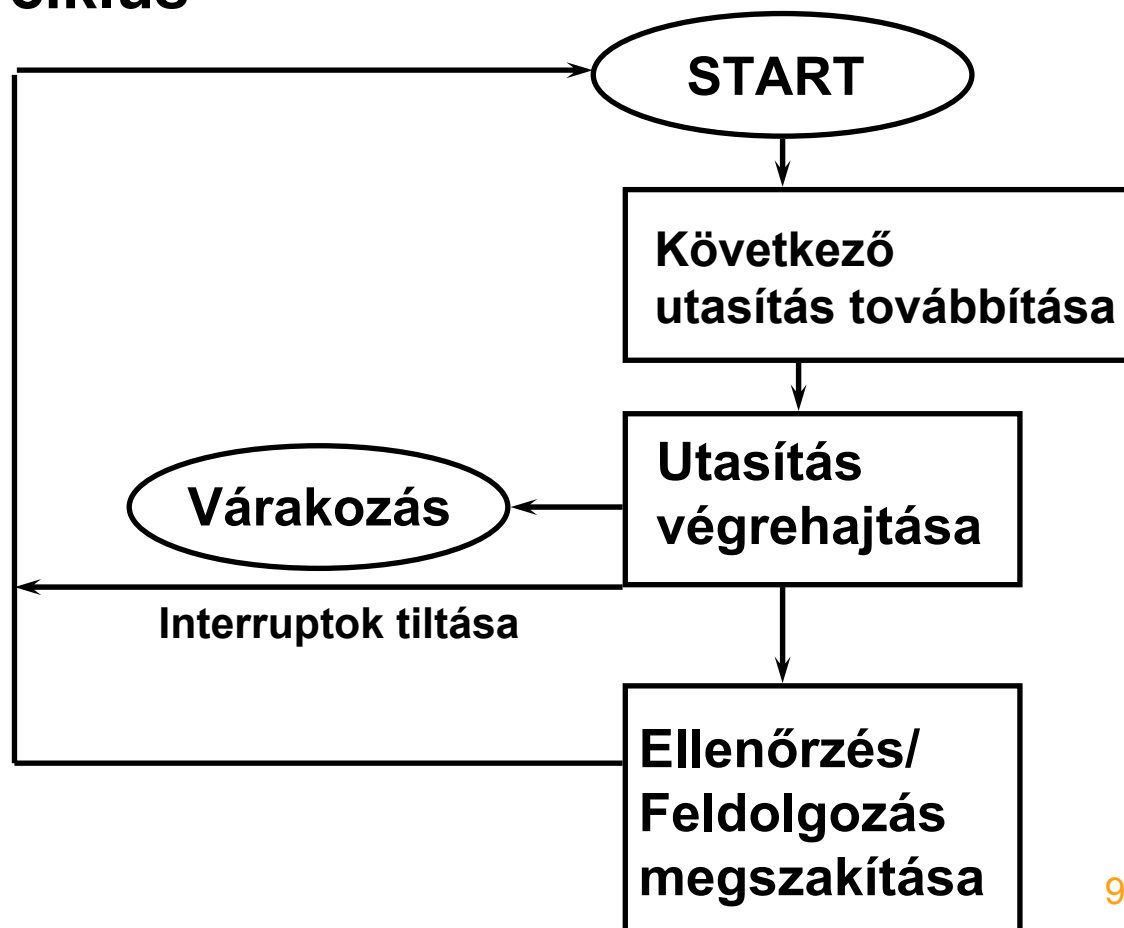
Megszakítások

- Jelzés, amely hatására a CPU megszakítja az aktuálisan végrehajtás alatt álló utasítássorozatot
 - Nem kell a CPU-nak az eseményekre várni
 - Irányítja a külső inputokat
- Használata (példák)
 - Váratlan input események kezelése
 - Abnormális (hiba) szituációk
 - Illegális (nem végrehajtható) utasítások
 - Multiprogramozott működés (párhuzamos feldolgozás) esetén CPU időelosztás



Megszakítás kezelése a CPU-ban

- Fetch/execute ciklus
- Megszakítás ciklus





Megszakítás terminológia

- Megszakítás-csatornák (interrupt lines) (hardware)
- Megszakítás-kérelem (request)
- Megszakítás-kezelők (handlers)
 - Program, ami a megszakítás eseményeket kezeli
 - Megszakításrutinnak is nevezik
- Process Control Block (PCB) (Folyamat vezérlési blokk)
 - A memória egy részében tároljuk, amit stack-nek nevezünk
 - A CPU minden regiszterének tartalma itt tárolódik, mielőtt a vezérlés a megszakítás kezelő rutinhoz kerül

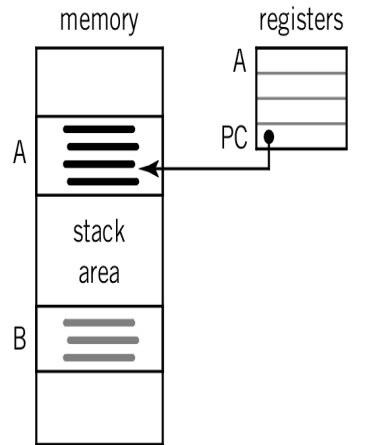


Megszakítás

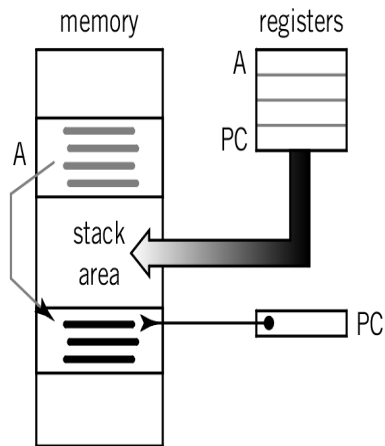
- Megszakítás kiszolgálása
 - Feldolgozás alatt álló programot felfüggesztjük
 - Menti a folyamat (program) folytatásához szükséges információkat, beleértve az utoljára végrehajtott utasítást, és a PCB-t (regiszterek)
 - Ugrás a megszakítás-kezelőre



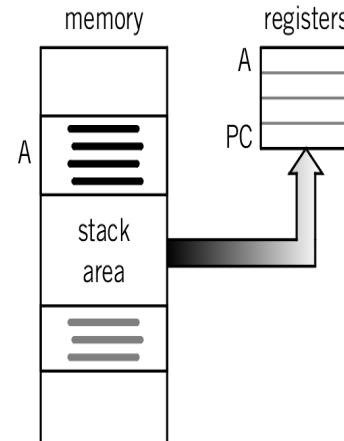
Egy megszakítás szolgáltatás



1. Before interrupt arrives, program A is executing. The program counter points to the current instruction.



2. When the interrupt is received by the CPU, the current instruction is completed, all the registers are saved in the stack area (or in a special area known as a process control block). The PC is loaded with the starting location of program B, the interrupt handler program. This causes a jump to program B, which becomes the executing program.



3. When the interrupt routine is complete, the registers are restored, including the program counter, and the original program resumes exactly where it left off.



Megszakítások használata

- Észreveszi egy külső esemény bekövetkezését
 - Valós idejű vagy idő érzékeny
- Jel befejezés
 - Nyomató kész vagy tele a buffer
- Kiosztja a CPU idejét
 - Időbeosztás
- Abnormális eseményeket idéz elő
- Illegális műveletek, hardverhiba
- Szoftver megszakítás

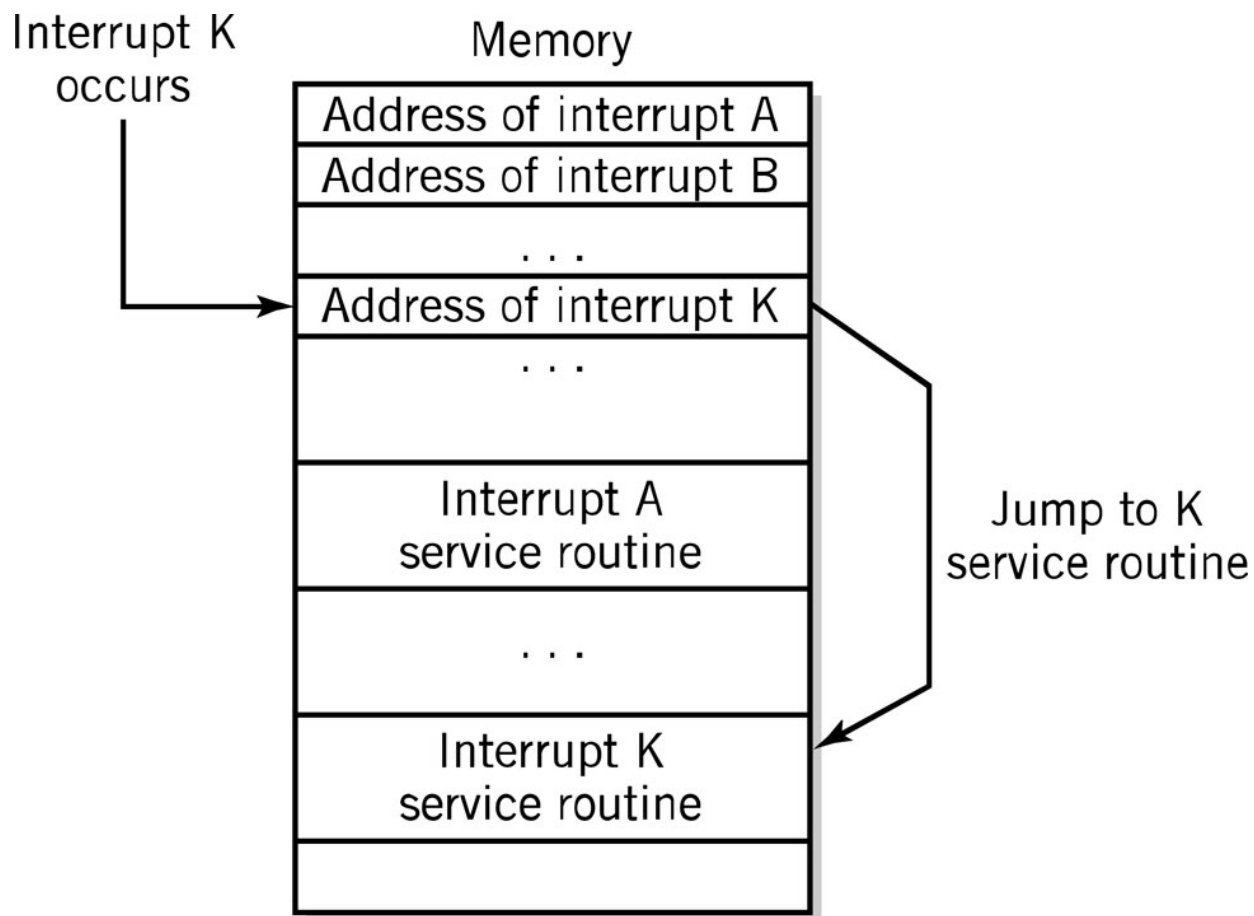


Összetett megszakítás

- Azonosítja az egységeket
 - Lekérdezés (utánanézés az input torlódásnak)
 - Vektoriális megszakítások (tartalmazza a megszakított eszközök címét)
- Megszakítás prioritás
 - Adatvesztés vs. Művelet-végrehajtás
- Maszkolható (letiltott) megszakítások

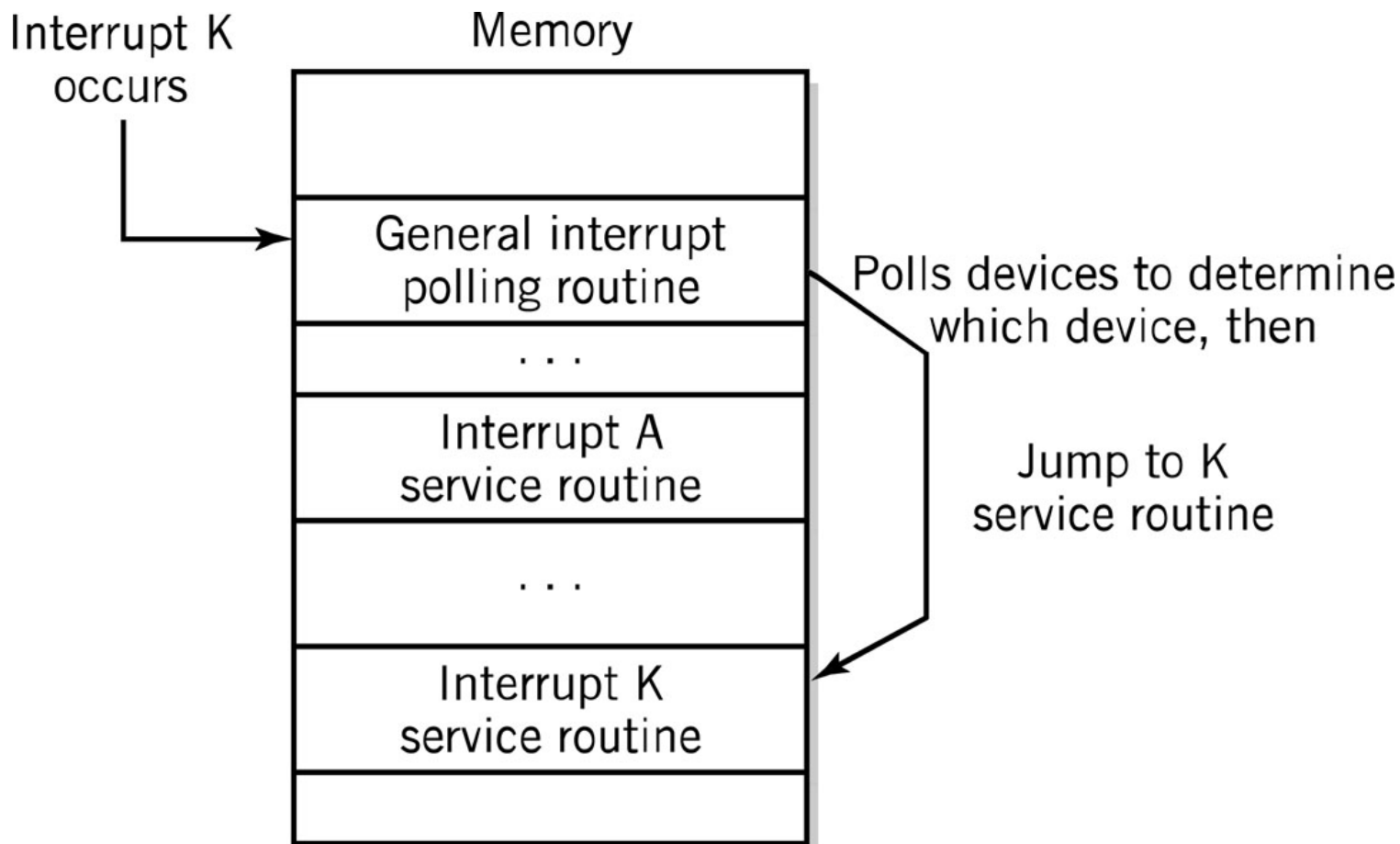


Vektoriális megszakítások



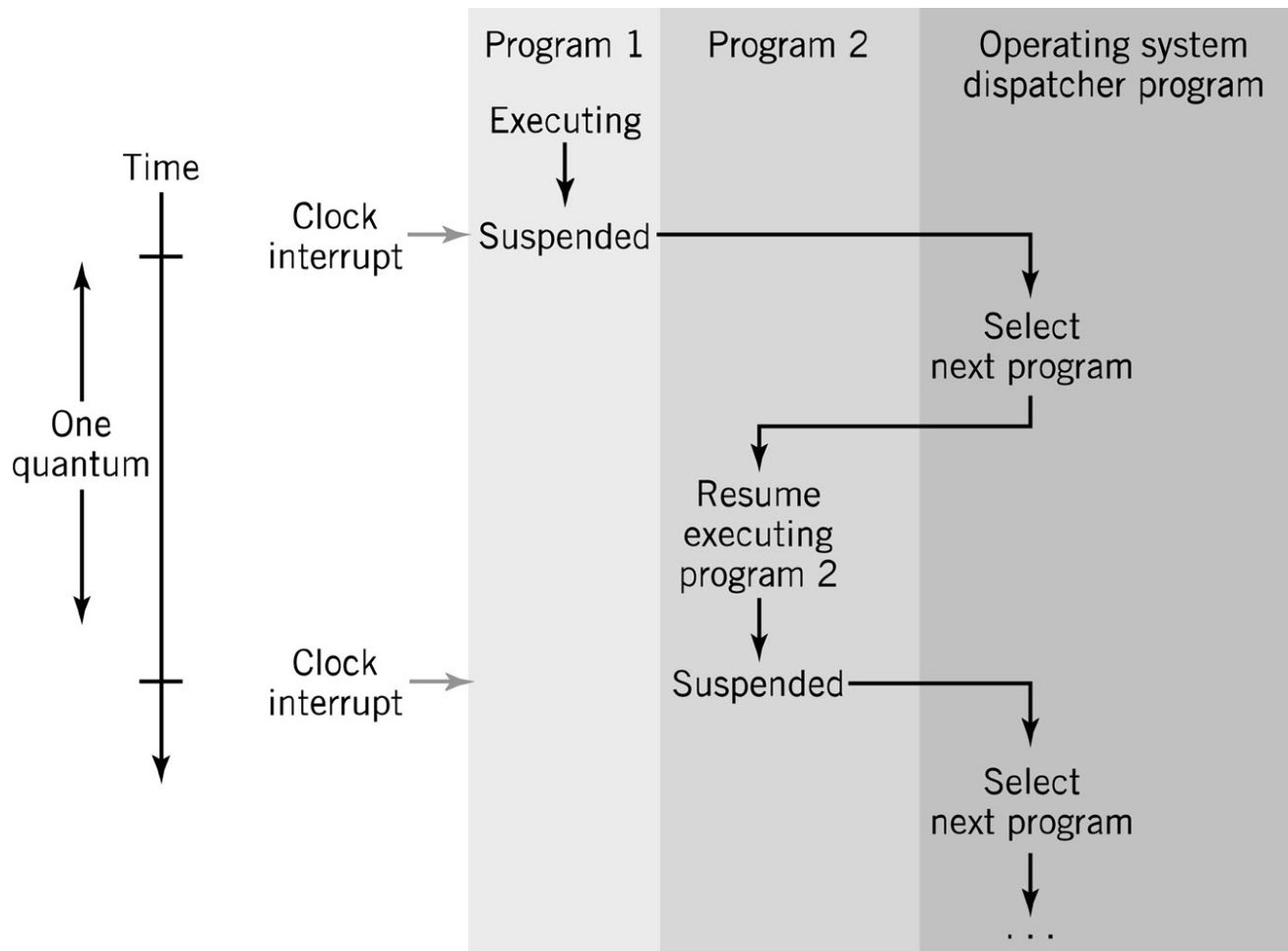


Lekérdező megszakítások





Összetett megszakítási példák





Közvetlen memória hozzáférés

- Nagy adatblokkok továbbítása
- Közvetlen átvitel az egységek között
- CPU nem vesz aktívan részt az átviteli körforgásban, folyamatban
- Kikötéseket igényel a DMA számára
 - Az I/O interfésznek és memóriának kapcsolódnia kell egymáshoz
 - Az I/O modulnak képesnek kell lennie a memóriából olvasni és abba írni
 - Az ütközéseket a CPU és a I/O modulok között el kell kerülni

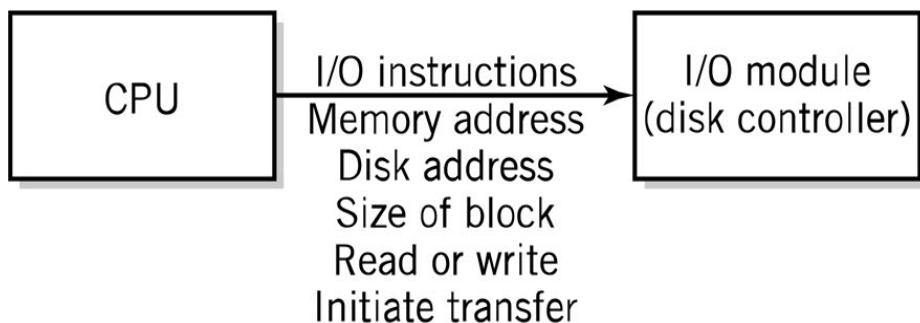


DMA utasítás beállítás

- A programalkalmazásnak szüksége van az operációsrendszer I/O szolgáltatásaira
 - Megkülönböztetett utasítások
- A DMA elindításához a programozott I/O-nak a következő utasításokat kell küldeni
 1. Adat elhelyezése az I/O egységen
 2. A memória elhelyezésének kezdete
 3. A blokk nagysága
 4. olvasás/írás
- Megszakítja a CPU-t a befejezéskor



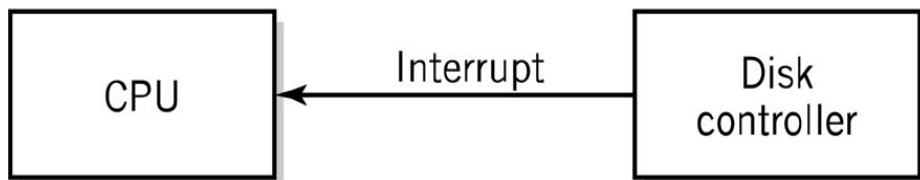
DMA indítás és irányítás



1. Programmed I/O used to prepare I/O module for transfer by providing required information and initiating transfer.



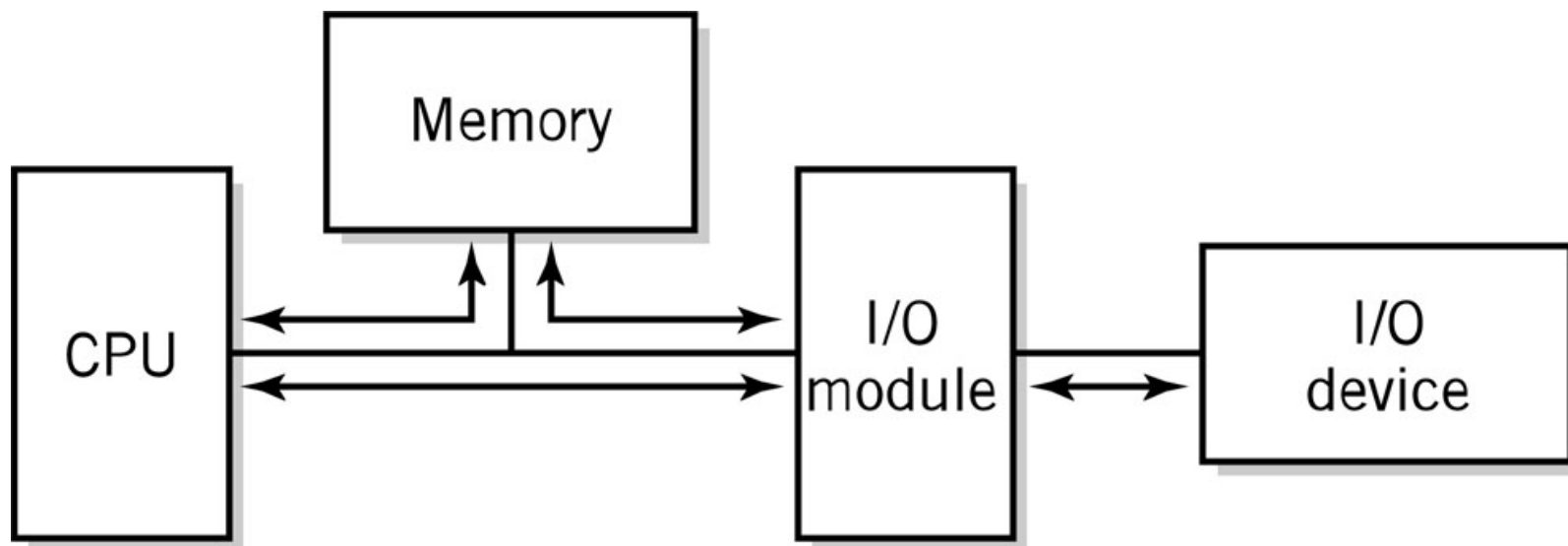
2. DMA transfer. In this case data is transferred from disk to memory.



3. Upon completion, disk controller sends **completion** interrupt to CPU.



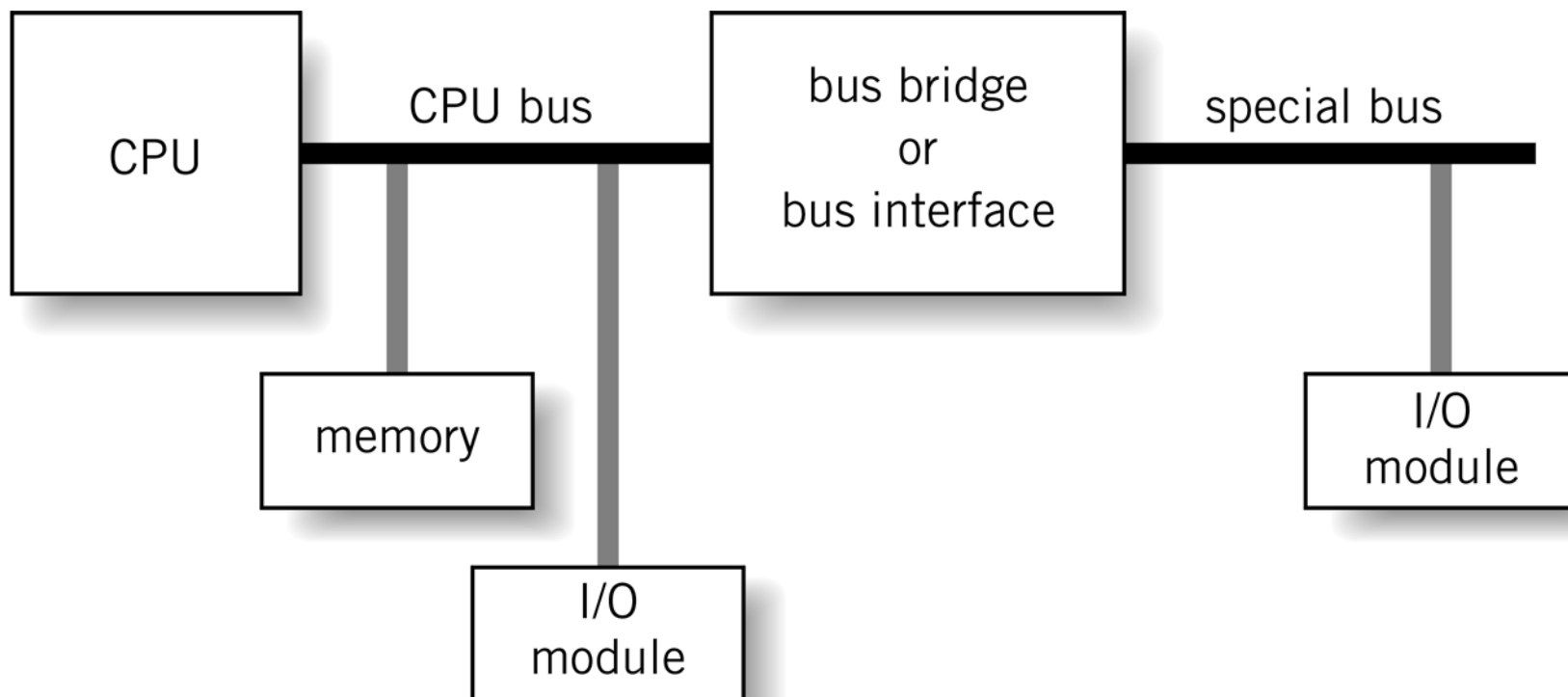
Alap CPU-memória-I/O ösvény*



Source: From *PCI Local Bus Specification Production Version 2*, Copyright © 1993, by PCI Special Interest Group, pg. 9. Reprinted by permission.



Bus beállítás





Bus karakterisztikák

- Adatszélességet bit-ekben
- Eredmény, adatátviteli arány bit/másodpercben
- Pontról pontra vs. Többpontos
- Párhuzamos vs. Soros
- Használat
- Távolság
- Protokoll

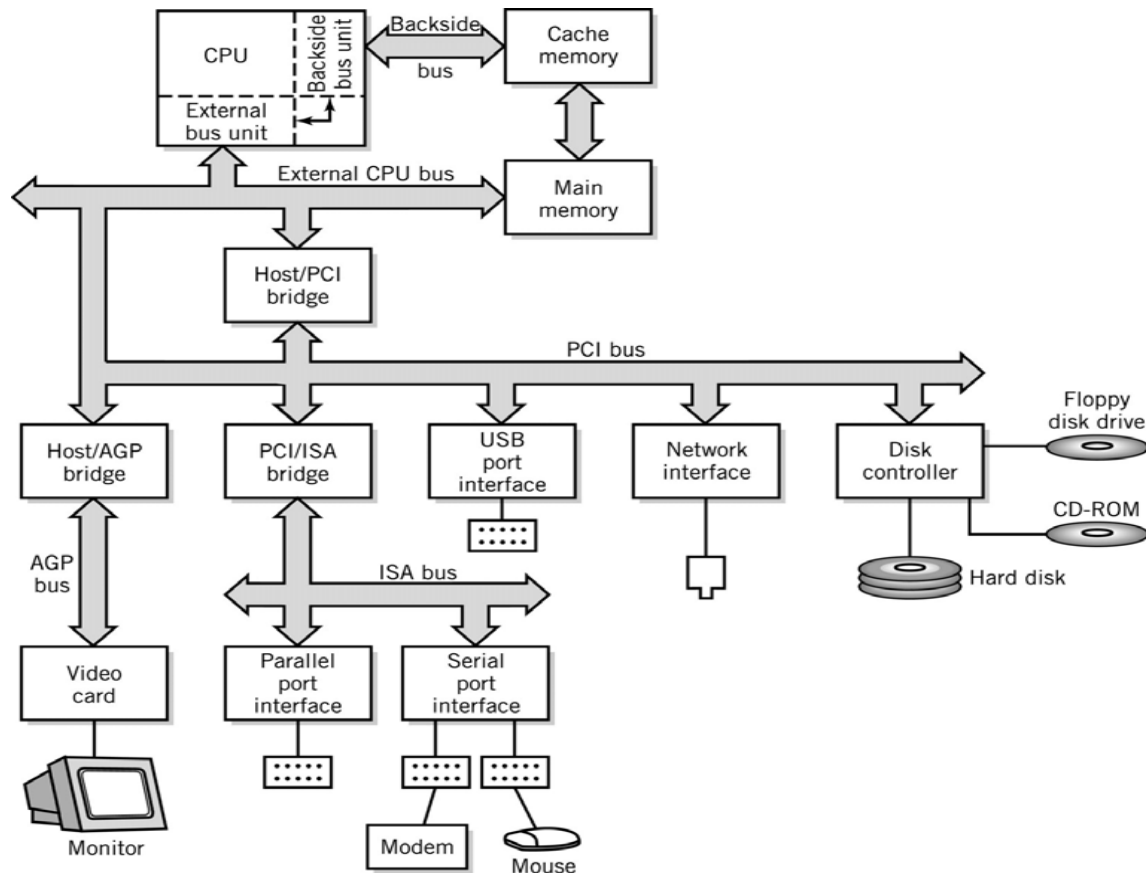


Bus hierarchia

- Processzor bus: chipen
- Cache bus (hátsó oldali busz)
- Memória bus (elülső oldali busz)
 - összekapcsolja a memória alrendszereket és a processzort
- Helyi I/O bus
 - A magas sebességű bus-ok a sebességorientáltan működő perifériákat köti össze
 - Példák : PCI, VESA helyi bus
- Szabvány I/O bus
 - A lassabb perifériákat köti össze (ISA) a helyi I/O bus-okkal



Összeköttetések egy tipikus PC-ben



Bus interface bridge-ek:

különböző típusú bus-okat kapcsolnak össze, átviszik az adatokat az egyik bus-ról a másikra

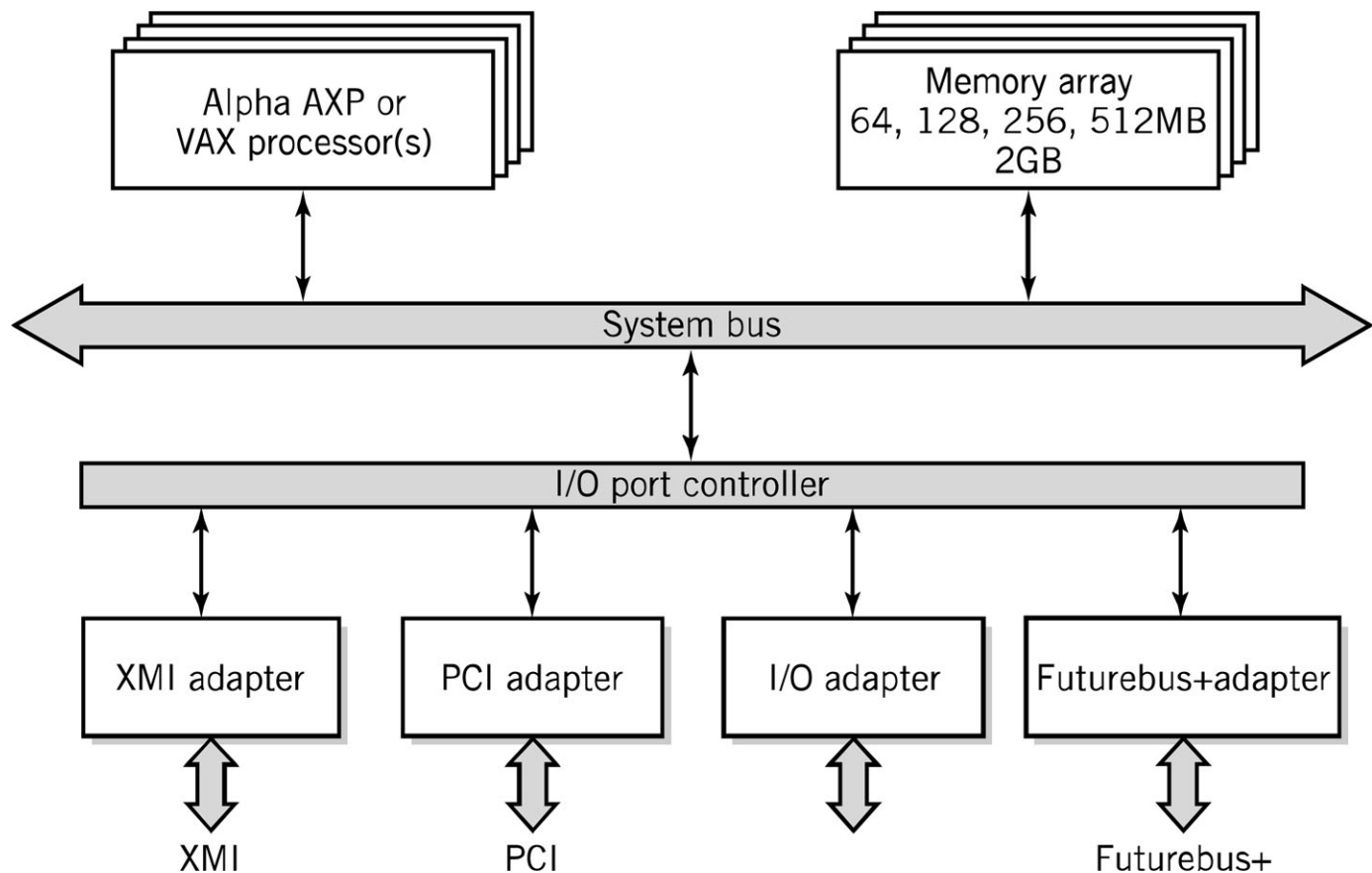


Wintel bus-rendszerek

- ISA: Industry Standard Architecture
- MCA: Micro Channel Architecture
- EISA: Extended Industry Standard Architecture
- Helyi Busz
 - PCI: Peripheral Component Interconnect (Apple, Sun, Compaq Alpha Server)
 - VLB: VESA (Video Electronics Standards Association) helyi busz
- AGP: Accelerated Graphics Port
 - Közvetlen csatorna a grafikusvezérlő és a memória között
 - Létezik PCI is



Compaq 7000 és 10000 rendszer architektúra



Source: Reproduced with permission from the *Digital Technical Journal*, vol. 4, no. 4 of Digital Equipment Corp.



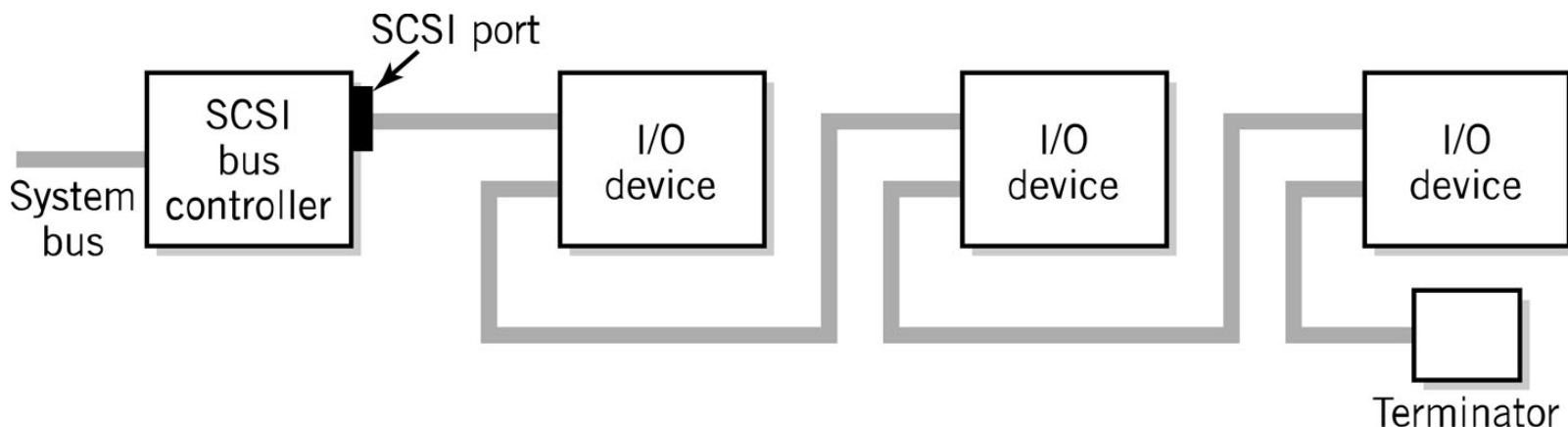
Külső interface bus-ok és portok

- Párhuzamos portok
- Soros port
 - RS-232C és RS-422 buszok
- SCSI
 - Small Computer System Interface
- USB, USB 2.0
 - Universal Serial Bus
- IEEE 1394
 - Firewire
 - i.link



SCSI Busz

- ANSI szabvány, de összetett variációkkal
- Inkább I/O busz, mint egy szokásos interfész
 - Megengedi az összetett egységeket szimpla SCSI portról

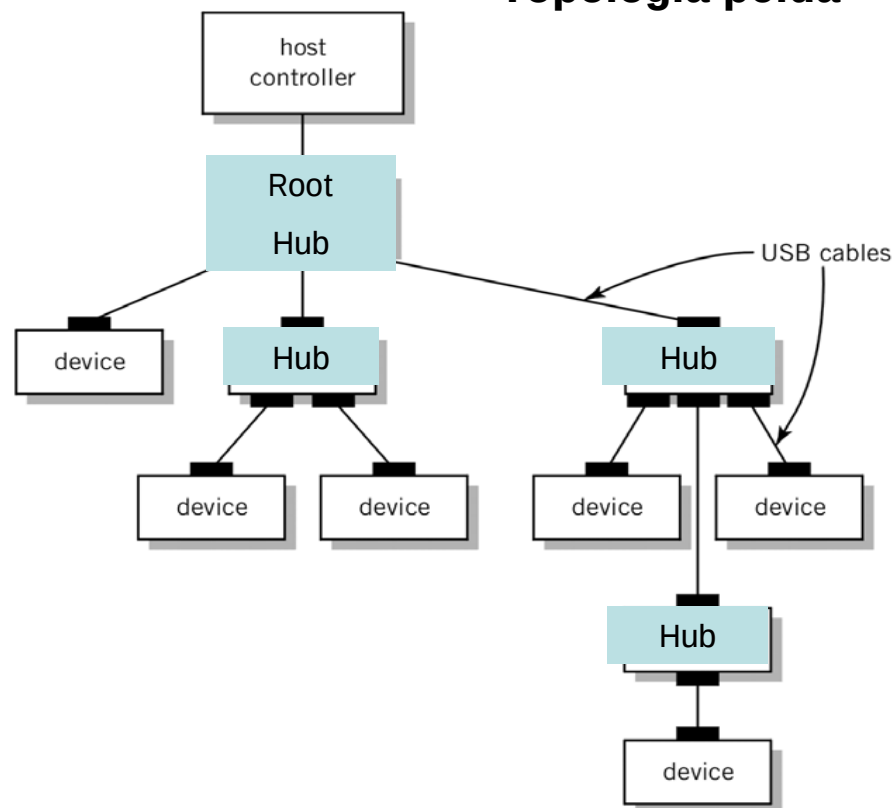




USB

- Multipoint („többpontos”) buszok
 - A **hubok** lehetővé teszik az összetett kapcsolati-pontokat az I/O egységeknek
 - 127 eszközt, egységet támogat

Topológia példa





USB és FireWire (IEEE 1394)

- Mindegyik soros, multipoint bus-os specifikáció
- Hozzáadás/törlés egységek w/o lekapcsolása
- Csomag protokoll „isochronous” adat átvitelhez
 - Isochronous: pontos időben történő szállítás
 - Specifikált eredményeket garantál



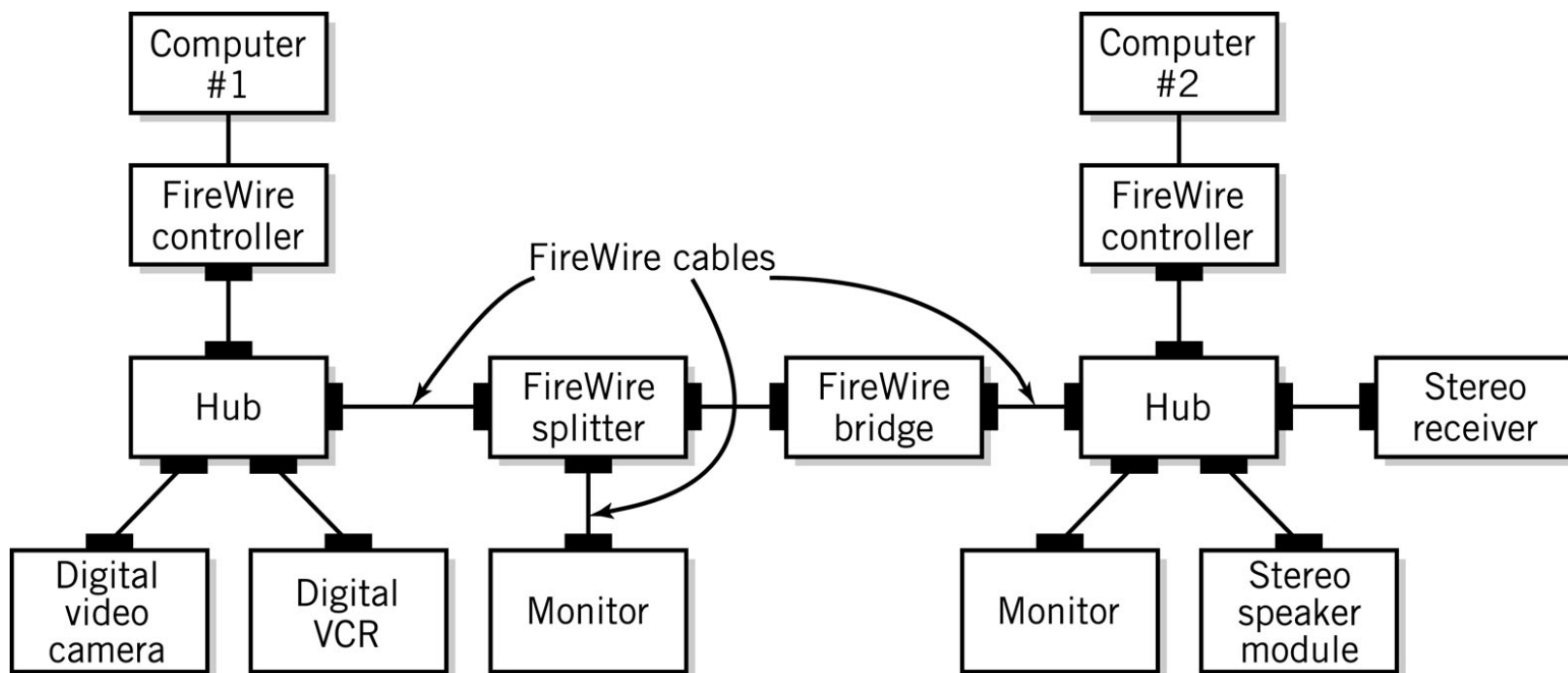
USB vs. FireWire

- USB: lassú a közepes sebességű adat átviteli alkalmazásokhoz, tárolóegységek
 - 12 Mbits/sec
- USB 2.0: nagysebességű adatátvitel
 - 480Mbits/sec
- FireWire: nagysebességű adatátvitel, videók hanggal
 - 400 Mbits/sec-től 3.2 Gbits/sec-ig



Tipikus FireWire beállítás

- Hálózatjellegű karakterisztika
- Független egységirányítás



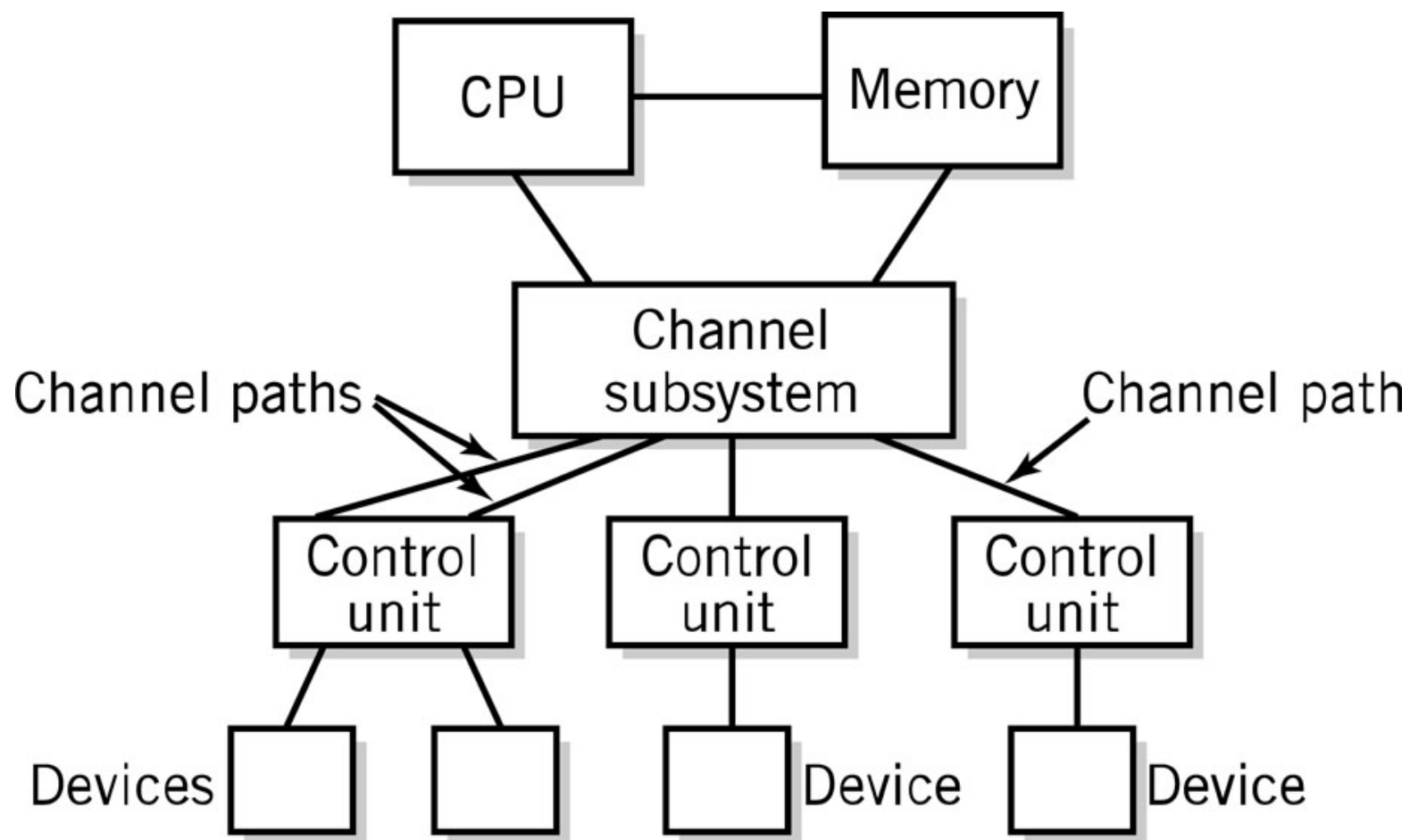


Csatorna architektúra

- Az IBM alapszámítógépeiben használják
- Csatorna alrendszer
 - leválasztott I/O processzor, ami CPU-ként működik az I/O műveletekhez
 - Csatornavezérlő parancsok
 - Program, ami adatokat szállít a memória és az I/O egységek között DMA-t használva
- Alcsatornák
 - Egy vagy két csatornán keresztül kapcsolódik egy irányítórendszer modulhoz
 - Hasonló a feladat az egységirányítóhoz



I/O csatorna architektúra





Copyright 2003 John Wiley & Sons

All rights reserved. Reproduction or translation of this work beyond that permitted in Section 117 of the 1976 United States Copyright Act without express permission of the copyright owner is unlawful. Request for further information should be addressed to the permissions Department, John Wiley & Sons, Inc. The purchaser may make back-up copies for his/her own use only and not for distribution or resale. The Publisher assumes no responsibility for errors, omissions, or damages caused by the use of these programs or from the use of the information contained herein.”



10. Fejezet

Számítógép-perifériák

**Hardver és Szoftver rendszerek
architektúrája:**

Egy Információ Technológiai Szemlélet

3. kiadás, Irv Englander

John Wiley and Sons ©2003

Wilson Wong, Bentley College

Linda Senne, Bentley College



Perifériák

- A számítógéptől különálló eszközök
 - Nem a CPU, memória, tápegység
- Csoportosíthatjuk input-ként, output-ként, és tárolóként
- Csatlakozhat:
 - Portokon keresztül
 - ▣ párhuzamos, USB, soros
 - Rendszerbuszokon keresztül
 - ▣ SCSI, IDE, PCMCIA



Tároló eszközök

- Elsődleges memória
- Kiterjesztett tároló
- Másodlagos tároló
 - Az adatokat és a programokat az elsődleges memóriába kell másolni, hogy a CPU elérhesse
 - Az adatok tárolhatósága
 - Közvetlen elérésű tároló egységek (DASD = Direct access storage devices)
 - Online tároló
 - Offline tároló – csak akkor töltődik be ha szükséges



Sebesség

- Az elérési idő és az adatátviteli arány segítségével mérhető
- Elérési idő (Access time): Az az átlagos időtartam, amely alatt a tároló megtalálja és beolvassa az adatot
 - millisecond = a másodperc ezred része
- Adat átviteli arány (data transfer rate): Egy másodperc alatt átvitt adatok mennyisége.



A tárolók hierarchiája

Eszköz	Tipikus Elérési idő		Átviteli arány
CPU Regiszterek			
Cache Memória (SRAM)	15 to 30	nanoseconds	
Átalakító Memória (DRAM)	50 to 100	nanoseconds	
Kiterjesztett Tároló (RAM)	75 to 500	nanoseconds	
Merevlemez	10 to 50	milliseconds	600 to 6,000 KB/sec
Floppy	95	milliseconds	100 to 200 KB/sec
CD-ROM	100 to 600	milliseconds	500 to 4,000 KB/sec
Szalagos meghajtó	.5 és e felett	seconds	2,000 KB/sec



Másodlagos tárolóeszközök

- Merevlemezek, floppy meghajtók
- CD-ROM és DVD-ROM meghajtók
- CD-R, CD-RW, DVD-RAM, DVD-RW
- Szalagos meghajtók
- Hálózati meghajtók
- Soros vagy közvetlen elérésű
- Váltakozó vagy lineáris



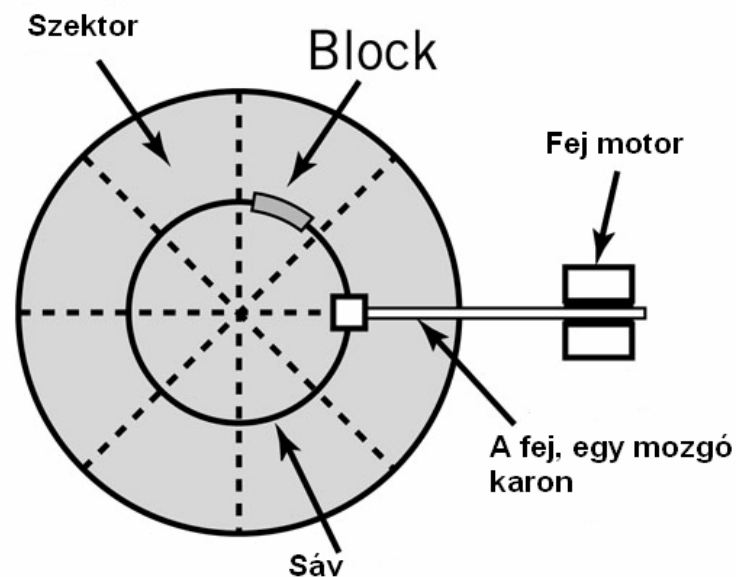
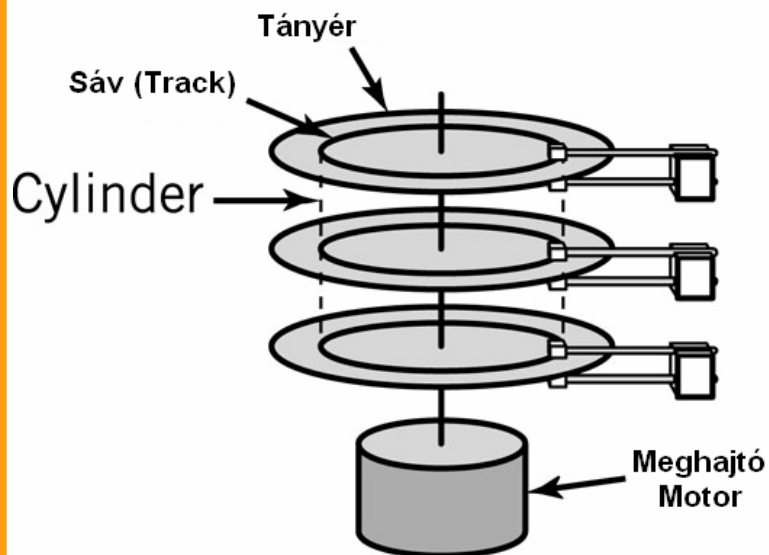
Mágnes lemezek

- Sáv – kör alakú
- Cylinder henger – az egymás felett elhelyezkedő sávok alkotják
- Block – a sávnak egy kis része
- Sector – a tányér egy torta szelet alakú része
- Head – az adatokat olvassa le a lemezről

- Író fejek
- Olvasó fejek
- A bit-ek mennyisége az egyes sávokon megegyezik!
- CAV (constant angular velocity) – Az állandó szögsebesség tartása, amely a lemez felületén kifelé haladva változó (növekvő) adatátviteli sebességet eredményez.
 - Az összes sáv ugyanazon a sebességen pörög
 - Merevlemezek – 3600 rpm – 7200 rpm
 - Floppy meghajtók – 360 rpm



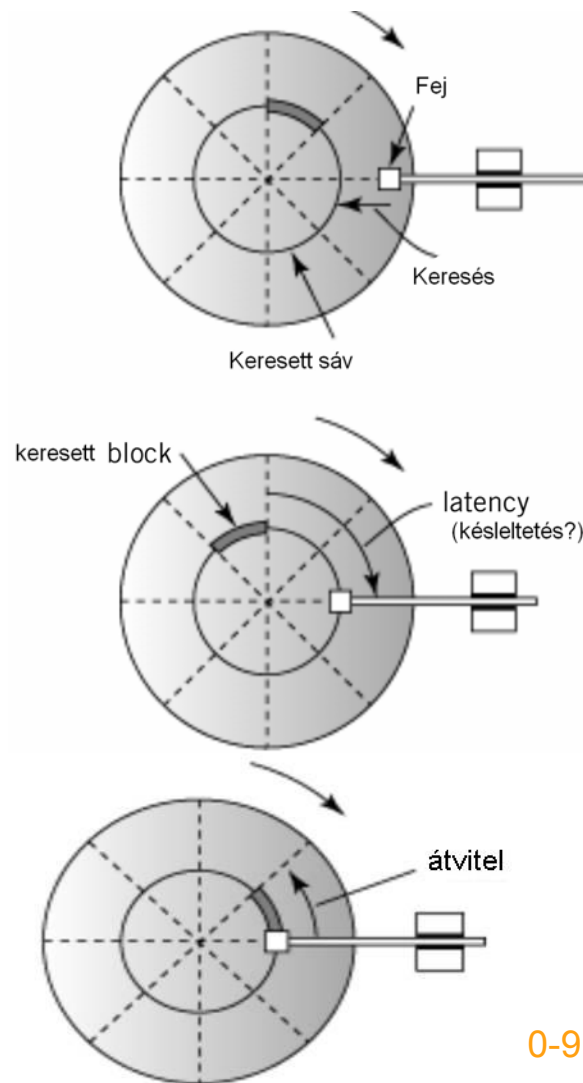
A merevlemez felépítése





Egy adatblokk megkeresése

- Átlagos keresési idő: Az egyik sávról a másikra való mozgáshoz szükséges időtartam
- Latency (késlekedés): Az az időtartam, amely ahhoz szükséges hogy a lemez a megfelelő szektor kezdetéhez forogjon.
- Átviteli idő: Az az időtartam, amely ahhoz szükséges hogy egy adat blokkot a lemezkezelő pufferbe juttasson.





A lemez elérési ideje

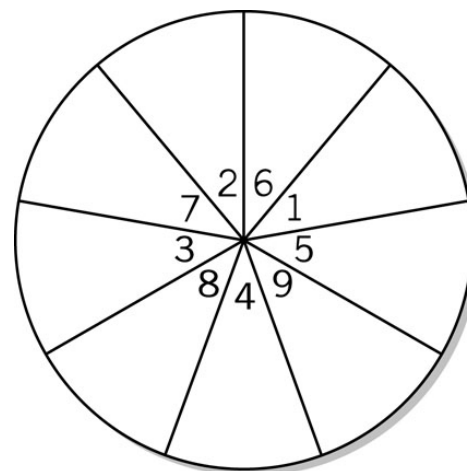
- Átlag keresési idő
 - Az egyik sávról a másik sávra mozgatáshoz szükséges idő átlagos mennyisége
- Átlag késlekedési idő
 - A szektor kezdetéhez forgatás átlagos idő mennyisége
 - Átlagos Latency idő = $\frac{1}{2} * \frac{1}{\text{forgási sebesség}}$
- Átviteli idő
 - $\frac{1}{(\text{szektorok száma} * \text{forgási sebesség})}$
- Egy lemez blokkjának eléréséhez szükséges idő
 - Átlag keresési idő + Átlag késlekedési idő + Átviteli idő



Mágnes lemezek

- Adatblokk Alak (Data Block Format)
 - Interblokk rés
 - Címzés (Header)
 - Adat
 - Lemez formázás
- Lemez Interleaving(??)
- Lemez Fajták
 - RAID – tükrözött, striped(??)
 - Logikai → hiba kezelő számítógépek

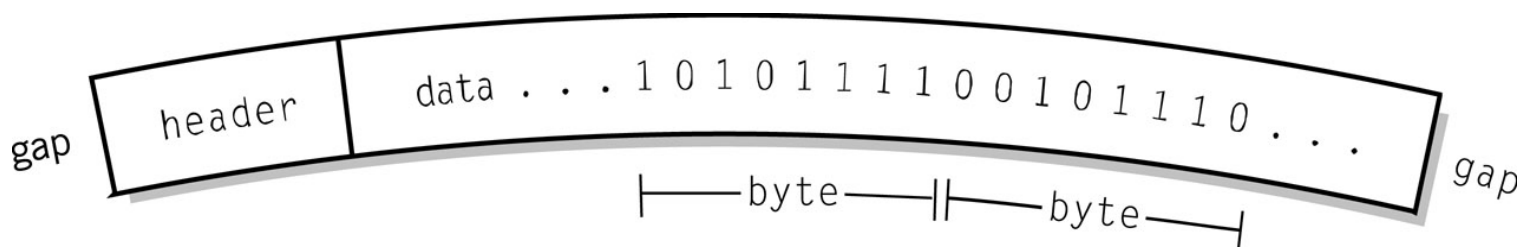
Disk Interleaving



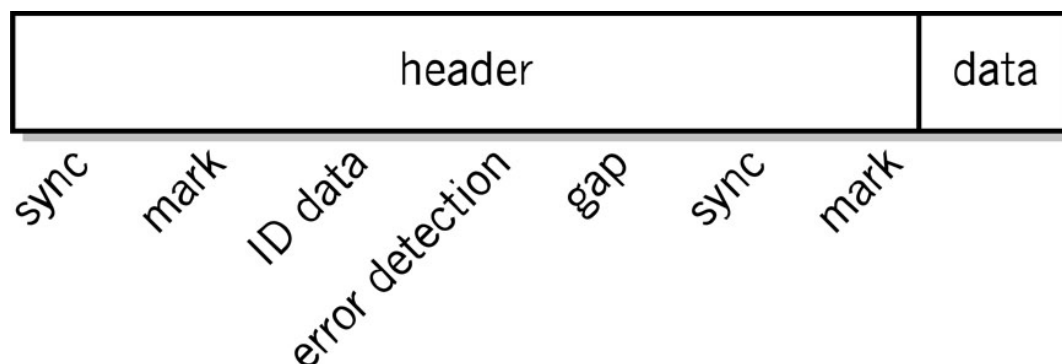


Lemezblokk formátuma

Egyszerű Adat Blokk



Windows lemez címkése





Alternatív lemeztechnológiák

- Hordozható merevlemezek
 - Disk pack – A lemez tányérjai egy műanyag tokban vannak, ami hordozható
 - Egy másik változat amikor a tokban a lemez fej és kar is megtalálható
- Fixed-head lemezmeghajtók
 - Egy fej / sáv
 - Kiküszöböli a keresési időt
- Bernoulli lemezmeghajtók
 - Hibrid technológia amely magába foglalja a floppy és merevlemez technológiákat is
 - Zip meghajtók



Mágnesszalag

- **Offline tárolás**
- **Archiválási célok**
- **Teljes adatvesztés elleni védelem**
- **Mágnesszalagos kazetták**
 - 20 – 144 sáv (oldalanként)
 - Soros olvasás
 - QIC – quarter inch cartridge (¼ inches kazetta, nagyobb méret)
 - DAT – digital audio tape (digitális audiószalag, kis méret)
 - Tipikus méret (2:1 arányú tömörítés)



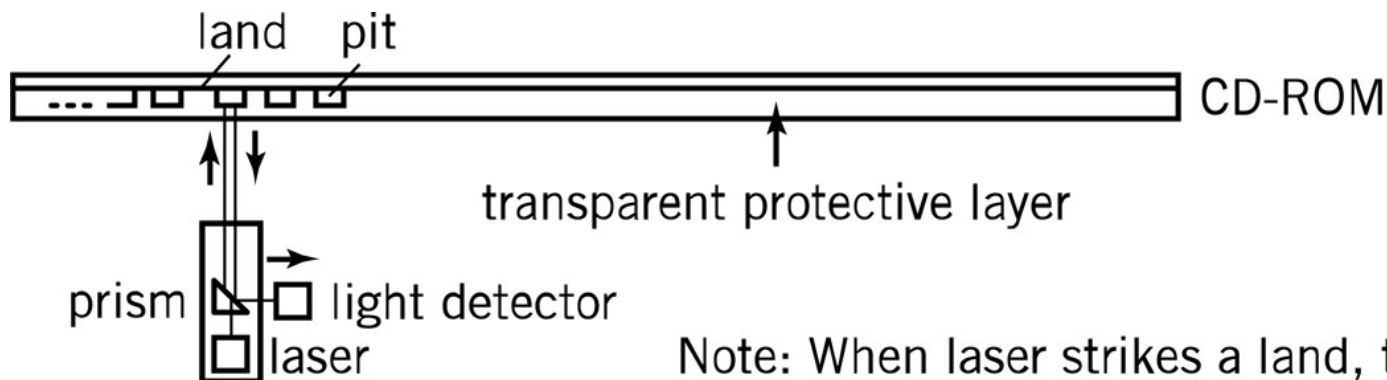
Optikai tárolók

- Fényvisszaverődés tükrözött vagy lyukacsos felületről
- CD-ROM
 - 3 mérföld hosszú spirál, 15 billió bit-et tartalmaz!
 - CLV – Az összes blokk fizikai hossza megegyezik
 - Block – 2352 byte
 - ▣ 2k adat (2048 byte)
 - ▣ 16 byte címzésnek (12 start, 4 id)
 - ▣ 288 byte fejlett hiba kezeléshez
- DVD-ROM
 - 4.7G rétegenként
 - Maximum 2 réteg / oldal, 2 oldal = 17G



Optikai tárolók

- Ha a lézer a felületre érkezik: a fény visszaverődik a detektorba
- Ha a lézer egy üregbe érkezik : a fény elnyelődik

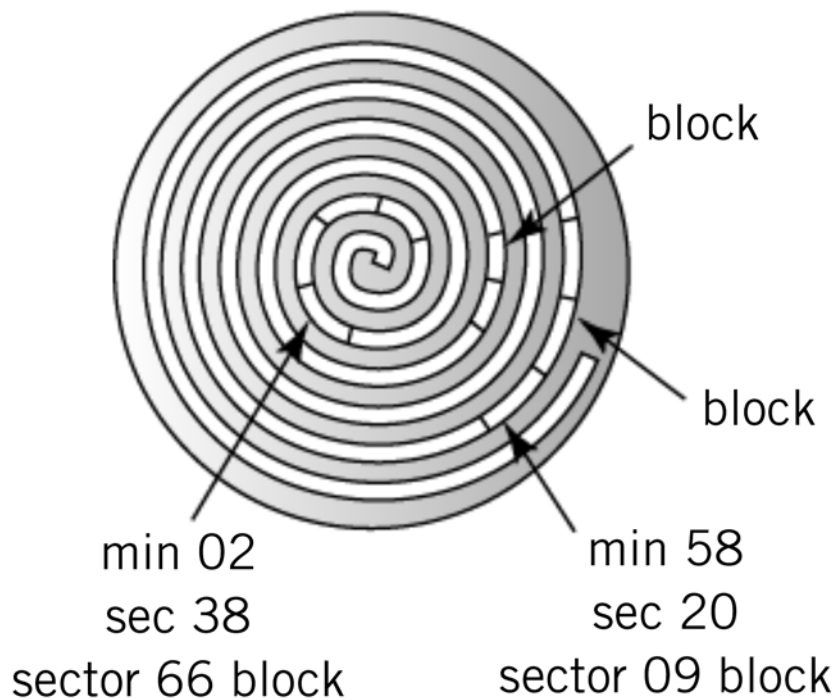


Note: When laser strikes a land, the light is reflected into the detector; when the light strikes a pit, it is scattered.

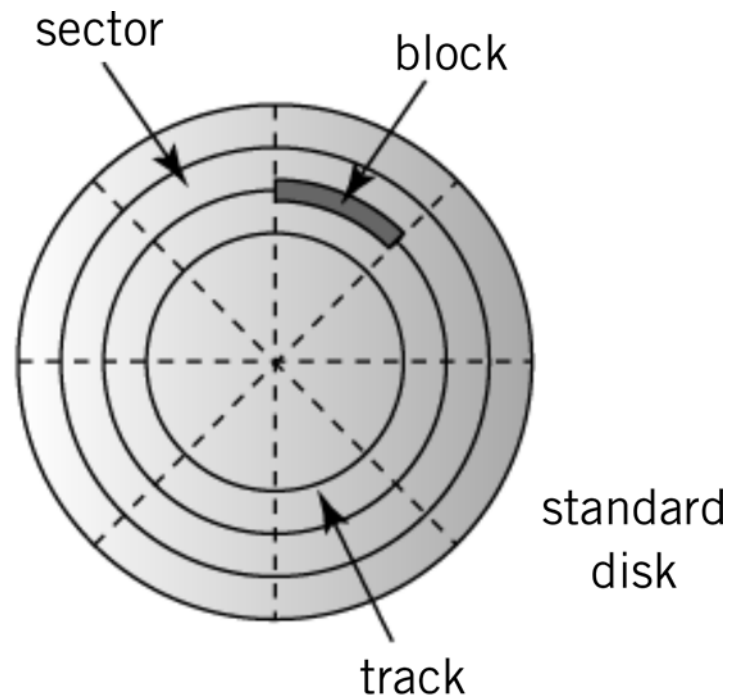


CD-ROM vs. Szabvány lemez

CD-ROM



Hard Disk





CD-ROM -ok

<i>General Speed</i>	<i>Seek Time</i>	<i>Data Transfer Rate</i>
	<i>(milliseconds)</i>	
Single-Speed	600	150K per second
2X	320	300K per second
3X	250	450K per second
4X	135-180	600K per second
6X	135-180	900K per second
8X	135-180	1.2 MBps
10X	135-180	1.6 MBps
12X	100-150	1.8 MBps
16X	100-150	2.4 MBps (maximum)
24X	100-150	3.6 Mbps (maximum)
32X	100-150	4.8 Mbps (maximum)



Optikai meghajtók típusai

- „WORM” Lemezek
 - Write-once-read-many times (Egyszer írható, sokszor olvasható)
 - Az adat koncentrikus sávokon tárolódik, szektor felépítése megegyezik a mágneses lemezekével
 - CAV
- Közepes erősségű lézertechnológiát használnak még:
 - CD-R, DVD-R, DVD-ROM
 - CD-RW, DVD-RW, DVD-RAM, DVD+RAM
- Mágneses-Optikai tárolók



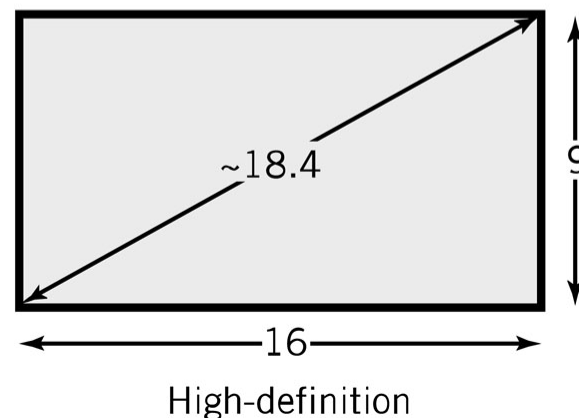
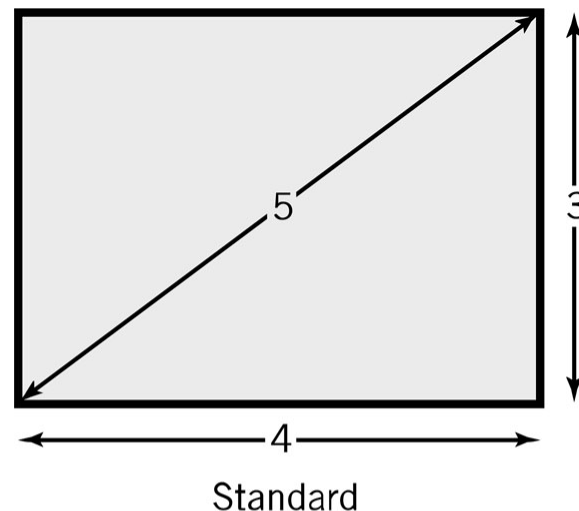
Kijelzők

- Pixel – **p**icture **e**lement (kép alkotóelem)
- Méret: A képernyő diagonális hossza (átlós hossz)
- Felbontás (pixelek a képernyőn)
 - VGA: 480 x 640
 - SVGA: 600 x 800
 - 768 x 1024
 - 1280 x 1024
- Kép méret kalkuláció
 - Felbontás * a szükséges bitek száma (színek száma a képen)
 - Például: 16 színű kép, 100*500pixel méretű
 $4 \text{ bites (16 színű)} * 100 * 50 = 20,000 \text{ bites kép}$



Kijelző

- Kijelző méret: diagonálisan mérhető (átlósan)
- Felbontás: minimális meghatározható pixel méret
- Aspect ratio: Arányszám (x:y)
4:3 a legtöbb számítógépen
 - 16:9 magas felbontású kijelzők esetén





Színek és a Kijelző

- A pixel színe 3 szín erősségéből áll össze – Vörös,Zöld,Kék (RGB)
- 4 bit/szín
 - $16 \times 16 \times 16 = 4096$ színű
- 24 bites szín (True Color)
 - 16.7 millió szín
- Videó-memória követelmény fontos!



CRT és Text Monitors

- CRTs (hasonlít a TV-khez)
 - 3 sor foszfor mindegyik színhez
 - 3 különálló elektronágyú mindegyik színhez
 - A fénysugár ereje → a szín árnyalata
 - Raster vizsgálat
 - ▣ 30 alkalom/másodperc
 - ▣ Interlaced vs. non-interlaced (progresszív vizsgálat)
- Text monitorok
 - 24 sor x 80 karakter
 - A karakter a képernyő legkisebb egysége
 - Kis memória szükséges csak

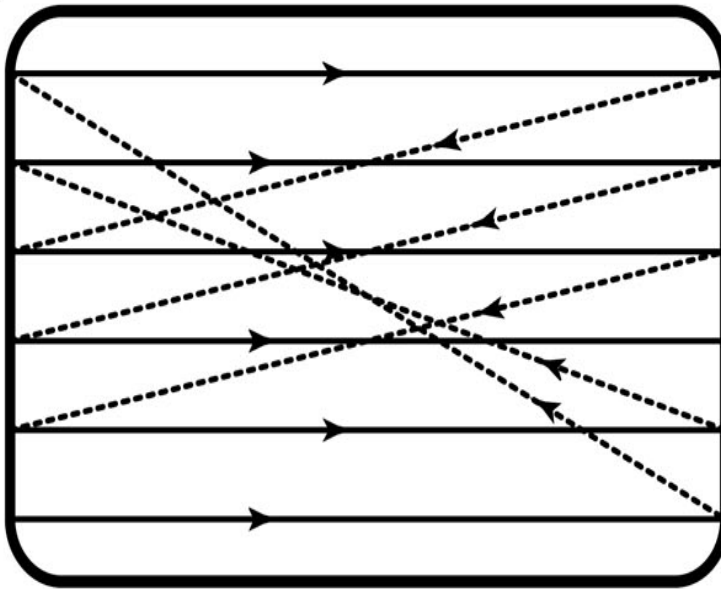


Interlaced vs. Non-interlaced

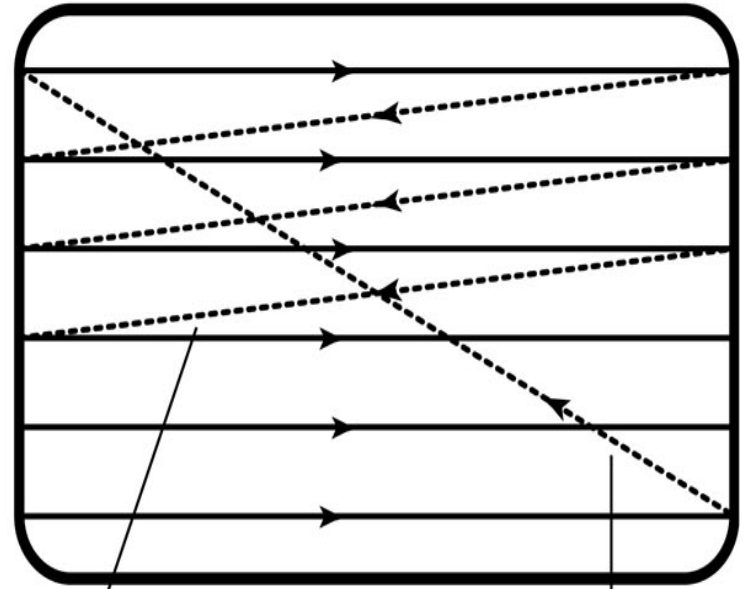
Pass

①

②



Interlaced scan



Noninterlaced scan

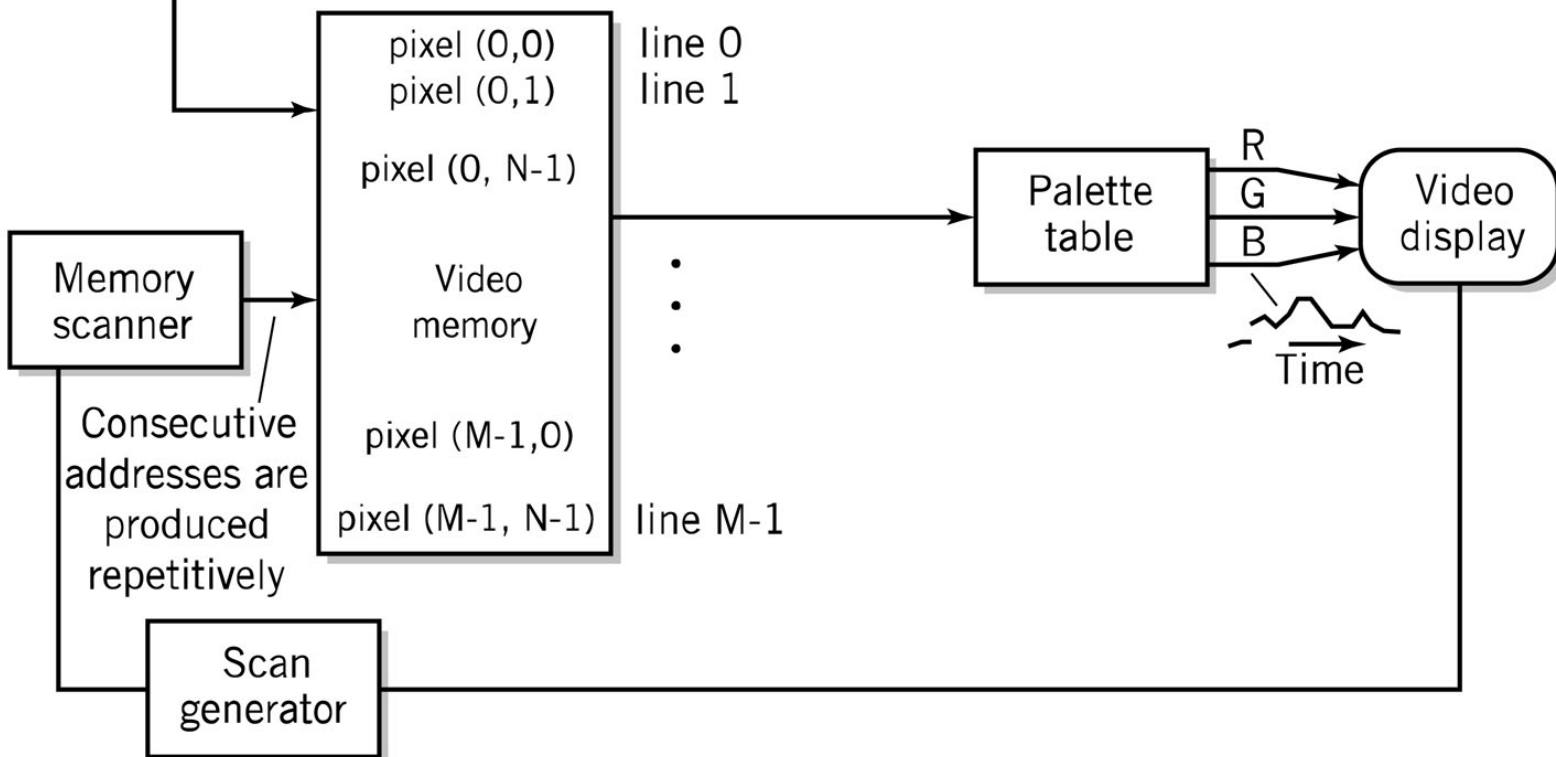
Horizontal
retrace

Vertical
retrace



Diagram a raster kijelzőről

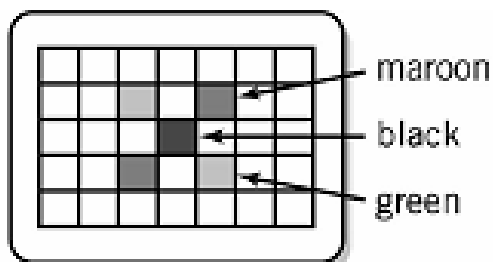
CPU program loads
memory with image to
be displayed





Kijelző példa

a. Desired display

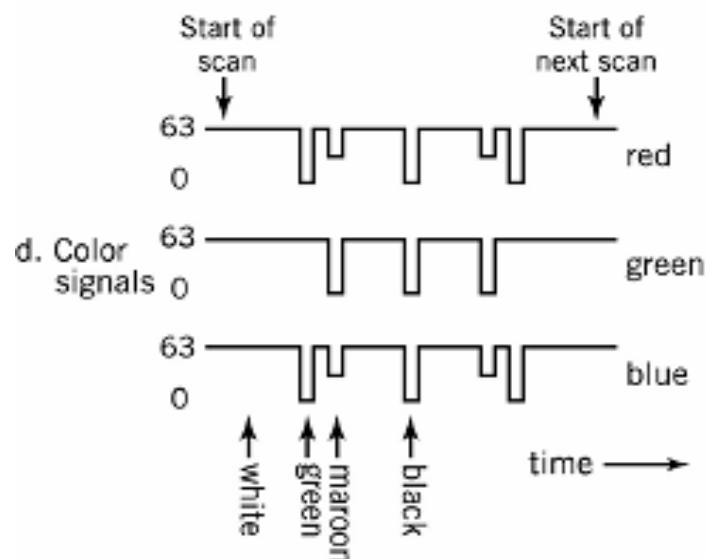


b. Video memory contents

0	1	2	3	4	5	6	Address
0	0	0	0	0	0	0	Value
7	8	9	10	11	12	13	Address
0	0	17	0	123	0	0	Value
14	15	16	17	18	19	20	Address
0	0	0	255	0	0	0	Value
21	22	23	24	25	26	27	Address
0	0	123	0	17	0	0	Value
28	29	30	31	32	33	34	Address
0	0	0	0	0	0	0	Value

c. Color palette table

Pixel value	red	green	blue
0	63	63	63
...			
17	0	63	0
...			
123	31	0	31
...			
255	0	0	0





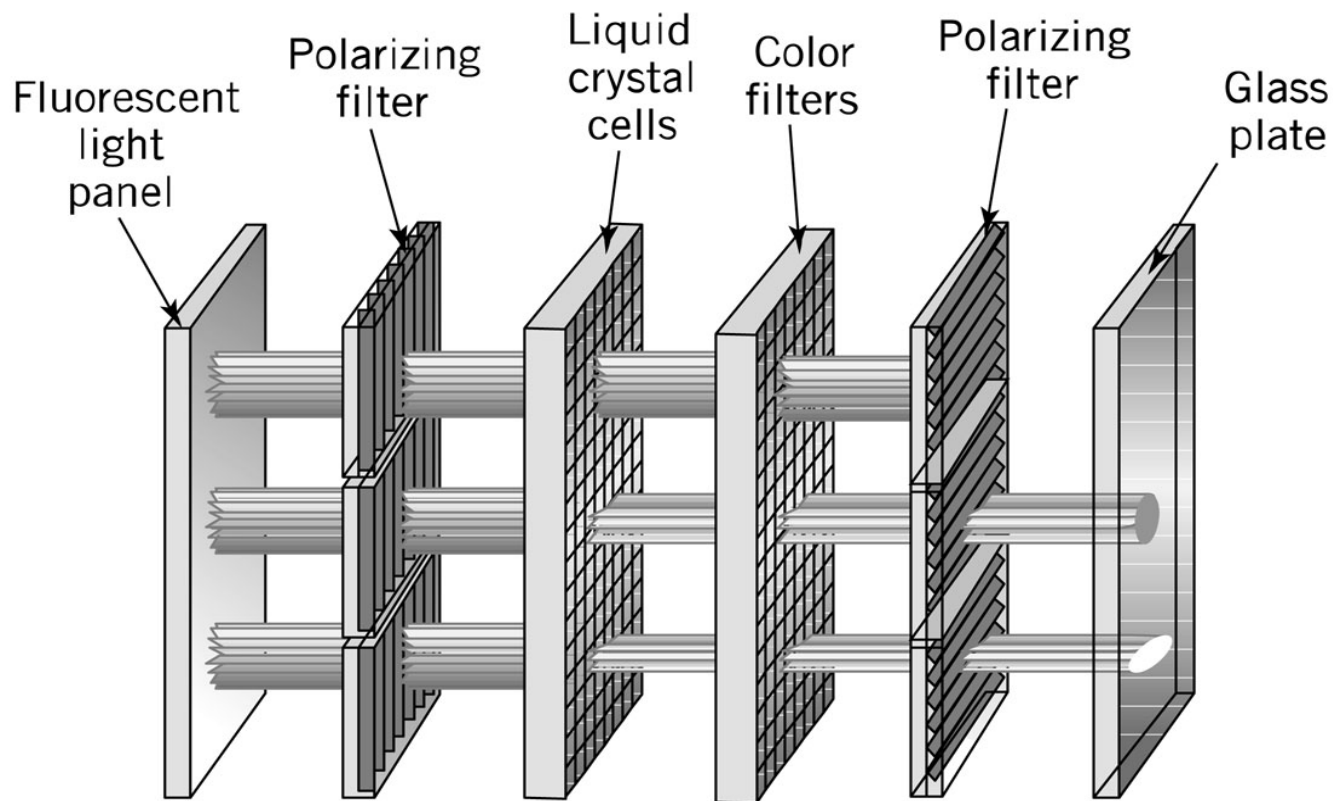
LCD – Liquid Crystal Display

Folyadék Kristályos Kijelző

- Fluoreszkáló fénypanel
- 3 szín cella/pixel
- Művelet
 - Az első szűrő polarizálja a fényt a megadott irányba
 - Elektromos töltés a molekulákat a folyadék kristály cellába mozgatja, ettől függ a szín ereje
 - Szín szűrők csak a vörös, zöld és kék színeket engedik át
 - Az utolsó szűrő a fény világosságát engedi át a polarizáció arányában



LCD Műveletek





LCD-k (folytatás)

- Aktívmátrixos
 - Cellánként egy tranzisztor
 - Sokkal drágább
 - Tisztább, élesebb kép
- Passzívmaátrixos
 - Egy tranzisztor soronként vagy oszloponként
 - A kijelző homályosabb mivel a pixelek kevésbé frekvenciáltak

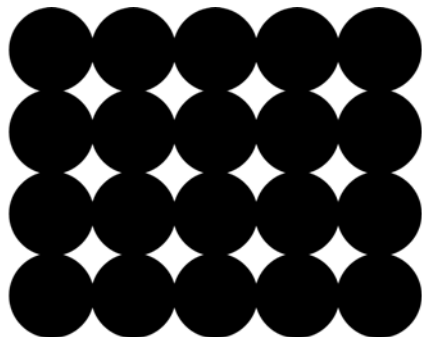


Nyomtatók

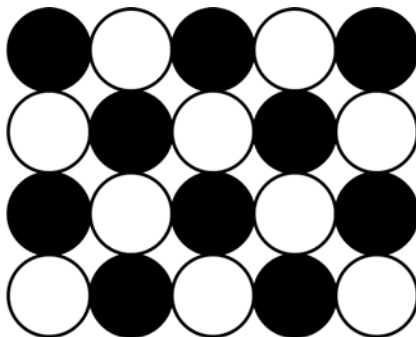
- Pontok vs. pixelek
 - 300-2400 dpi vs. 70-100 pixels per inch
- Típusok
 - Írógép / Margaréta fejes – elavult
 - Pontmátrix – általában 24 tűs
 - Tintasugaras – forró tinta cseppeket lövell a felületre
 - Lézer
 - Termálviaszos átvitel
 - Festék szublimálás



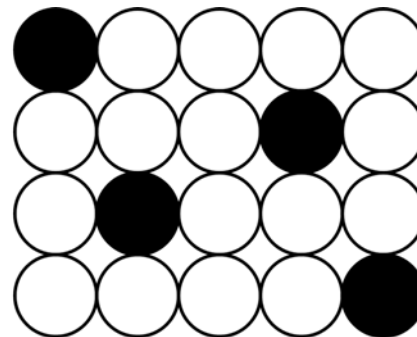
Szürke skála előállítás



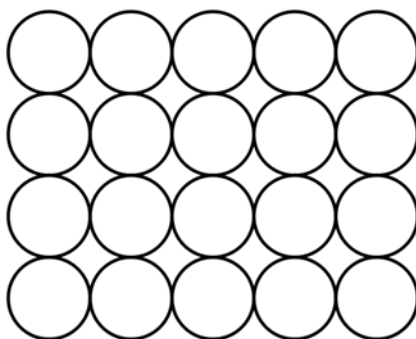
black



dark gray



light gray



white

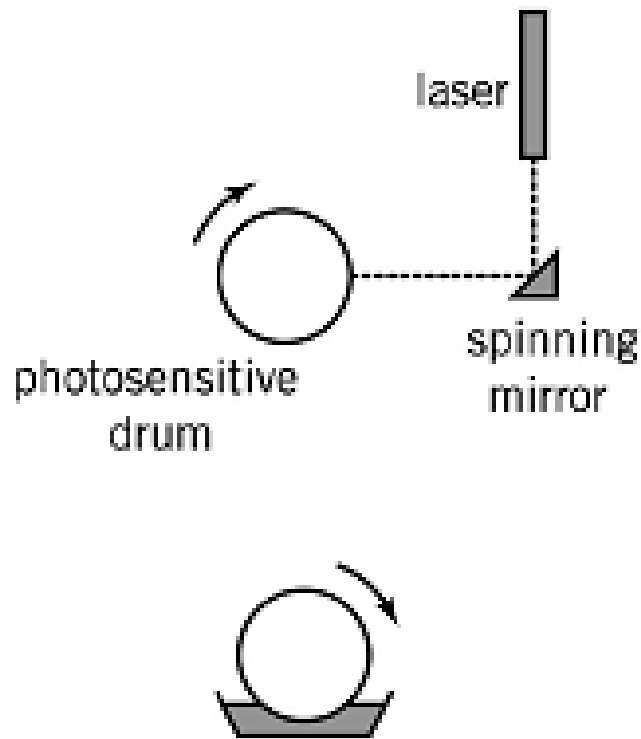


Laser Printer Operation

1. A henger elektromos töltést kap
2. A lézer megvilágítja a hengert
3. A henger átmegy a toner-en, ahol az elektromosan töltött részekre ráragadnak a festékszemcsék
4. Az elektromosan töltött papírt a henger felé továbbítja
5. A festék átkerül a papírra
6. A fűtőrendszer ráégeti a papírra a festéket
7. A „corona wire” elvezeti az elektromos töltést a hengerről



Laser Printer Operation



1. A laser is fired in correspondence to the dots that are to be printed. A spinning mirror causes the dots to be fanned out across the drum. The drum rotates to create the next line, usually $1/300$ th or $1/600$ th of an inch.

The drum is photosensitive. As a result of the laser light, the drum will become electrically charged wherever a dot is to be printed.

2. As the drum continues to rotate, the charged part of the drum passes through a tank of black powder called toner. Toner sticks to the drum wherever the charge is present. Thus, it looks like the image.



Laser Printer Operation

charge wire



3. A sheet of paper is fed toward the drum. A charge wire coats the paper with electrical charges. When it contacts the drum, it picks up the toner from the drum.



4. As the paper rolls from the drum, it passes over a heat and pressure area known as the fusing system. The fusing system melts the toner to the paper. The printed page then exits the printer.

At the same time, the surface of the drum passes over another wire, called a corona wire. This wire resets the charge on the drum, to ready it for the next page.



Egyéb Számítógép Perifériák

- Scanner-ek
 - Síkágys, lapolvasó, kézi olvasó
 - A fény visszaverődik a lapról
- Felhasználói beviteli perifériák
 - billentyűzet, egér, fényceruza, rajz tábla
- Kommunikációs eszközök
 - Telefonmodemek
 - Hálózati eszközök



Copyright 2003 John Wiley & Sons

All rights reserved. Reproduction or translation of this work beyond that permitted in Section 117 of the 1976 United States Copyright Act without express permission of the copyright owner is unlawful. Request for further information should be addressed to the permissions Department, John Wiley & Sons, Inc. The purchaser may make back-up copies for his/her own use only and not for distribution or resale. The Publisher assumes no responsibility for errors, omissions, or damages caused by the use of these programs or from the use of the information contained herein.”